

**TRƯỜNG ĐẠI HỌC THỦ DẦU MỘT
VIỆN CÔNG NGHỆ SỐ**



**BÁO CÁO TIỂU LUẬN HỌC PHẦN
KIẾN TRÚC VÀ THIẾT KẾ PHẦN MỀM**

**XÂY DỰNG WEB – APP QUẢN LÝ
DỊCH VỤ ĐỌC TRUYỆN TRANH ONLINE**

Chuyên ngành : Công Nghệ Thông Tin
Nhóm : Nhóm 17
Sinh viên: Trương Xuân Huy – 2324802010044
Phạm Huỳnh Ngọc Sơn – 2324802010339

Giảng viên hướng dẫn : ThS. Cao Thanh Xuân

TP. HỒ CHÍ MINH, THÁNG 5/2026

LỜI GIỚI THIỆU

Hiện nay, cộng đồng đọc truyện tranh (Manga/Manhwa/Comic) tại Việt Nam và quốc tế đang phát triển vô cùng mạnh mẽ. Tuy nhiên, các hệ thống website đọc truyện hiện tại thường xuyên gặp phải tình trạng máy chủ quá tải, tốc độ tải ảnh chậm và gián đoạn dịch vụ khi có lượng truy cập lớn. Bên cạnh đó, việc thu thập dữ liệu bị phân tán, kiểm duyệt nội dung (hình ảnh nhạy cảm, độc hại) vẫn phụ thuộc vào sức người, cùng với trải nghiệm người dùng chưa được cá nhân hóa khiến độc giả khó tiếp cận được những tác phẩm phù hợp.

Xuất phát từ thực trạng đó, nhóm chúng em quyết định chọn đề tài "Xây dựng Web – App Quản lý Dịch vụ đọc Truyện tranh Online". Mục đích cốt lõi của đề tài là kiến tạo một nền tảng đọc truyện thể hệ mới dựa trên kiến trúc Microservices mạnh mẽ, nhằm giải quyết triệt để các bài toán về hiệu suất, tự động hóa vận hành và khả năng chịu tải hàng ngàn lượt truy cập cùng lúc.

Giá trị thực tiễn mà dự án mang lại không chỉ là một trải nghiệm đọc mượt mà, tối ưu hóa lưu trữ ảnh, mà còn là sự tiên phong trong việc tích hợp AI (Swin Transformer, OCR, BERTopic) để tự động hóa kiểm duyệt và gợi ý truyện. Qua đó, hệ thống hướng tới việc xây dựng một cộng đồng độc giả tương tác thời gian thực lành mạnh, đồng thời tối ưu hóa chi phí và nguồn lực quản lý cho hệ thống.

Tên nhóm: Nhóm 17

STT	MSSV	Họ tên	Nội dung thực hiện	Mức độ hoàn thành
1	2324802010339	Phạm Huỳnh Ngọc Sơn	Vẽ usecase tổng quan + Ngữ cảnh dự án	100%
			Frontend hệ thống Web – app	100%
			Thiết kế các view kiến trúc	100%
			Thiết kế cơ sở dữ liệu	100%

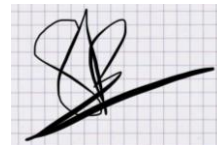
			Xác định các yêu cầu kiến trúc	100%
3	2324802010044	Trương Xuân Huy	Backend Hệ thống Web – app	100%
			Viết tài liệu các mẫu kiến trúc sử dụng trong dự án	100%
			Tài liệu xác thực kiến trúc	100%
			Viết cuốn báo cáo + làm slide + thuyết trình	100%
			Xây dựng app di động đa nền tảng Flutter	100%

Tp. Hồ Chí Minh, ngày 10 tháng 05 năm 2026

Nhóm sinh viên thực hiện



Trương Xuân Huy



Phạm Huỳnh Ngọc Sơn

**PHIẾU ĐÁNH GIÁ KIỂM TRA
HỌC PHẦN: KIẾN TRÚC VÀ THIẾT KẾ PHẦN MỀM
Học kỳ: 2 Năm học: 2025-2026 Khóa: 2022-2026**

I. THÔNG TIN CHUNG HỌC PHẦN

- Tên học phần: Kiến trúc và thiết kế phần mềm
- Mã học phần: KTPM034 Số tín chỉ: 2
- Ngành đào tạo: Công nghệ thông tin
- Hình thức kiểm tra: Tiểu luận

II. KẾT QUẢ ĐÁNH GIÁ

Danh sách sinh viên trong nhóm: Nhóm 17

MSSV	Họ và tên	Lớp	Ngành	Phần 1: Đánh giá tham gia trong nhóm (nếu có)	Phần 2: Đánh giá chủ đề	Điểm = Phần 1 * Phần 2
(1)	(2)	(3)	(4)	(5)	(6)	(7)
2324802010044	Trương Xuân Huy	D23CNTT02	CNTT	100%		
2324802010339	Phạm Huỳnh Ngọc Sơn	D23CNTT04	CNTT	100%		

Phần 1 (Chỉ sử dụng đối với bài làm theo nhóm):

Nội dung đánh giá tham gia trong nhóm của sinh viên được tính theo phần trăm (%) đóng góp (nếu có); đây là phần đánh giá cho cột (5)

(Dành cho sinh viên đánh giá)

Họ và tên SV	Thái độ tham gia nhóm	Ý kiến đóng góp hữu ích	Thời gian giao nộp sản phẩm đúng hạn	Chất lượng sản phẩm giao nộp	Tổng %	Ký tên
	20%	20%	30%	30%	100%	

Họ và tên SV	Thái độ tham gia nhóm	Ý kiến đóng góp hữu ích	Thời gian giao nộp sản phẩm đúng hạn	Chất lượng sản phẩm giao nộp	Tổng %	Ký tên
	20%	20%	30%	30%	100%	
Trương Xuân Huy	20%	20%	30%	30%	100%	
Phạm Huỳnh Ngọc Sơn	20%	20%	30%	30%	100%	

Phần 2: Nội dung đánh giá chủ đề, các tiêu chí được trích xuất từ rubric đánh giá; đây là phần đánh giá cho cột (6)

TT	Chỉ báo thực hiện	Tiêu chí đánh giá	Điểm tối đa	CELO x	Điểm đánh giá		
					CB chấm 1	CB chấm 2	Điểm thống nhất
1	Phân tích và thiết kế hệ thống	Xác định yêu cầu và giải pháp của bài toán.	0.5	CELO ₁	0.5	0.5	0.5
		Thiết kế hệ thống hoặc thuật toán rõ ràng, phù hợp và logic	0.5	CELO ₁	0.5	0.5	0.5
2	Sản phẩm	Triển khai giao diện (với sản phẩm ứng dụng) hoặc triển khai thuật toán chính xác.	3.0	CELO ₂	3.0	3.0	3.0
		Thực hiện thử nghiệm hoặc kiểm thử sản phẩm/thuật toán	2.0	CELO ₃	2.0	2.0	2.0
3		Đánh giá mức độ ứng dụng thực tế. Sử dụng ý tưởng hoặc công nghệ mới để giải quyết vấn đề	1.0	CELO ₃	1.0	1.0	1.0
4	Nội dung quyền báo cáo tiểu luận	Trình bày đúng quy cách, có tài liệu tham khảo rõ ràng đầy đủ	1.0	CELO ₄	1.0	1.0	1.0
5	Thuyết	Phong cách trình bày	1.0	CELO	1.0	1.0	1.0

TT	Chỉ báo thực hiện	Tiêu chí đánh giá	Điểm tối đa	CELO x	Điểm đánh giá		
					CB chấm 1	CB chấm 2	Điểm thống nhất
	trình	Giọng nói Tương tác với người nghe		4			
6	Thái độ tham gia lớp học/nhóm làm việc	Tham gia tích cực các buổi họp nhóm Phân chia công việc hợp lý	1.0	CELO 4	1.0	1.0	1.0
Điểm tổng cộng			10				

Thành phố Hồ Chí Minh, ngày 15 tháng 05 năm 2026

CÁN BỘ CHẤM 1

(Ký, ghi rõ họ tên)

Trần Văn Hữu

CÁN BỘ CHẤM 2

(Ký, ghi rõ họ tên)

Cao Thanh Xuân

MỤC LỤC	
DANH MỤC HÌNH	3
DANH MỤC BẢNG	6
DANH SÁCH CÁC KÝ TỰ, CÁC CHỮ VIẾT TẮT.....	8
CHƯƠNG 1: TÀI LIỆU KIẾN TRÚC	9
1.1. Sơ lược về tài liệu kiến trúc	9
1.2. Ngữ cảnh dự án	10
1.3. Những yêu cầu kiến trúc	11
1.3.1. Những mục tiêu chính của dự án	11
1.3.2. Những yêu cầu về chức năng của các bên liên quan	11
1.3.3. Những yêu cầu về chất lượng (phi chức năng) của các bên liên quan	13
1.4. Trình bày kiến trúc	15
1.4.1. Các mẫu kiến trúc được sử dụng trong dự án	15
1.4.2. Kiến trúc tổng quan của dự án	15
1.4.3. Thiết kế View kiến trúc Logic (Logical Views)	22
1.4.5. Thiết kế View kiến trúc Tiến trình (Process Views)	28
1.4.6. Thiết kế View kiến trúc triển khai/Vật lý (Deployment Views)	36
1.5. Tài liệu xác thực kiến trúc	38
1.5.1. Xác thực yêu cầu về Tính sẵn dùng (Availability)	38
1.5.2. Xác thực yêu cầu về Khả năng cập nhật (Modifiability)	43
1.5.3. Xác thực yêu cầu về Tính bảo mật (Security)	47
1.5.4. Xác thực yêu cầu về Khả năng mở rộng (Scalability)	50
1.5.5. Xác thực yêu cầu về Độ tin cậy (Reliability)	56

1.5.6. Xác thực yêu cầu về Hiệu suất (Performance)	61
CHƯƠNG 2: THIẾT KẾ SẢN PHẨM CỦA ĐỀ TÀI	69
2.1. Thiết kế cơ sở dữ liệu	69
2.2. Thiết kế giao diện của hệ thống	91
2.2.1. Phân Hệ User	91
2.2.2. Phân Hệ Admin.....	114
2.2.3. Phân hệ bổ sung	118
CHƯƠNG 3: ĐÁNH GIÁ VÀ HƯỚNG PHÁT TRIỂN	119
3.1 Các nội dung đã thực hiện được.....	119
3.2 Những hạn chế trong việc cụ thể hóa cài đặt so với bản thiết kế.....	120
3.3 Hướng phát triển của của đề tài	120
TÀI LIỆU THAM KHẢO	122

DANH MỤC HÌNH

Hình 1. Kiến trúc Microservice tổng quan của dự án.....	15
Hình 2. Mẫu kiến trúc client - server của dự án	18
Hình 3. Mẫu kiến trúc dùng để giao tiếp giữa các microservice.....	19
Hình 4. Mẫu Pipeline để ETL dữ liệu	20
Hình 5. Mẫu kiến trúc Master – Slave tách biệt luồng đọc – ghi để giảm tải cho DB.....	21
Hình 6. Biểu đồ usecase tổng thể	22
Hình 7. Biểu đồ lớp tổng thể	25
Hình 8. Biểu đồ gói	26
Hình 9. Biểu đồ thành phần.....	27
Hình 10. Tiến trình Tìm kiếm & Lọc truyện.....	29
Hình 11. Tiến trình Đọc truyện	30
Hình 12. Tiến trình Theo dõi truyện & Xử lý thông báo	31
Hình 13. Tiến trình Upload & Xử lý AI ngầm.....	32
Hình 14. Tiến trình Đăng bình luận & Lọc từ ngữ.....	33
Hình 15. Tiến trình Chat Real-time	34
Hình 16. Tiến trình Crawl & Đồng bộ dữ liệu ETL (Pipe-Filter).....	35
Hình 17. Tiến trình Cấp quyền và Xác thực (Identity Workflow)	36
Hình 18. Các thành phần triển khai vật lý	37
Hình 19. Danh sách các container của prototype và port sử dụng	61
Hình 20. Kết quả K6 test prototype 1 trường hợp 1	62
Hình 21. Kết quả K6 test prototype 1 trường hợp 2.....	63
Hình 22. Kết quả K6 test prototype 2 trường hợp 1	64
Hình 23. Kết quả K6 test prototype 2 trường hợp 2.....	65
Hình 24. Kết quả K6 test prototype 3 trường hợp 1	67
Hình 25. Kết quả K6 test prototype 3 trường hợp 2.....	67
Hình 26. ERD của DB nghiệp vụ	88
Hình 27. Danh sách các document sau khi thu thập từ Mangadex API.....	89

Hình 28. ERD của DB Staging.....	90
Hình 29. Giao diện authentication của app	91
Hình 30. Quy trình Dùng OTP cho xác thực tài khoản	92
Hình 31. Hệ thống được setup SMTP service thủ công	92
Hình 32. Giao diện trang chủ	93
Hình 33. Giao diện xem chi tiết manga.....	94
Hình 34. Giao diện xem danh sách manga theo Author/Artist	94
Hình 35. Thêm/Xóa manga trong list tại trang chi tiết manga	95
Hình 36. Giao diện xem danh sách chapter có tùy chỉnh ngôn ngữ.....	96
Hình 37. Giao diện đọc truyện có thể tùy chỉnh layout.....	97
Hình 38. Các setting cơ bản trong lúc đọc truyện	98
Hình 39. Demo chức năng dịch trang truyện	99
Hình 40. Giao diện comment và report	100
Hình 41. Giao diện trang Related Manga (Hệ tư vấn)	101
Hình 42. Giao diện cho phép tải manga về máy	102
Hình 43. Giao diện quản lý danh sách cá nhân của người đọc	103
Hình 44. Giao diện search/follow list của user khác	103
Hình 45. Giao diện quản lý manga trong list	104
Hình 46. Các tiện ích hỗ trợ quản lý manga trong list	104
Hình 47. Giao diện xem lịch sử đọc	105
Hình 48. Giao diện trang cá nhân user, có thể thay đổi avatar.....	106
Hình 49. Giao diện phân tích thói quen đọc dựa trên dữ liệu lịch sử.....	107
Hình 50. Giao diện quản lý các dịch vụ chat.....	108
Hình 51. Giao diện trang chat có thể gửi/xem/tải ảnh.....	108
Hình 52. Giao diện hệ tư vấn theo cá nhân sử dụng lọc cộng tác	109
Hình 53. Hỗ trợ nhiều option linh hoạt cho chức năng search.....	113
Hình 54. Hỗ trợ nhiều layout hiển thị cho kết quả search.....	113
Hình 55. Giao diện dashboard của Admin	114

Hình 56. Giao diện quản lý độc giả.....	115
Hình 57. Giao diện quản lý comment.....	116
Hình 58. Giao diện quản lý manga của hệ thống	117
Hình 59. Giao diện của người đăng tải.....	118
Hình 60. Giao diện quản lý log của admin.....	118

DANH MỤC BẢNG

Bảng 1. Nhóm Identity (Tài khoản & Phân quyền)	16
Bảng 2. Nhóm Catalog (Thông tin truyện tĩnh)	16
Bảng 3. Nhóm Reader & Media (Truyền tải ảnh & Upload)	17
Bảng 4. Nhóm Interaction & Chat (Tương tác xã hội).....	17
Bảng 5. Nhóm Library (Thư viện cá nhân của người dùng)	18
Bảng 6. Xác thực yêu cầu Availability.....	43
Bảng 7. Xác thực yêu cầu Modifiability	46
Bảng 8. Xác thực yêu cầu Security.....	50
Bảng 9. Xác thực yêu cầu Scalability.....	56
Bảng 10. Xác thực yêu cầu Reliability	61
Bảng 11. Kết quả tổng hợp số liệu sau khi chạy K6 test prototype 1	63
Bảng 12. Kết quả tổng hợp số liệu sau khi chạy K6 test prototype 2	65
Bảng 13. Bảng User.....	69
Bảng 14. Bảng Comment	70
Bảng 15. Bảng CommentReaction	71
Bảng 16. Bảng Rating.....	71
Bảng 17. Bảng ReadingHistory	72
Bảng 18. Bảng Report	72
Bảng 19. Bảng Manga	73
Bảng 20. Bảng MangaAltTitle	74
Bảng 21. Bảng MangaAvailableLanguage.....	74
Bảng 22. Bảng MangaDescription	75
Bảng 23. Bảng MangaLink.....	75
Bảng 24. Bảng MangaRelated.....	76
Bảng 25. Bảng MangaStatistics.....	77
Bảng 26. Bảng Tag	77
Bảng 27. Bảng MangaTag.....	77

Bảng 28. Bảng Chapter.....	78
Bảng 29. Bảng Creator	79
Bảng 30. Bảng CreatorRelationship	79
Bảng 31. Bảng List	80
Bảng 32. Bảng ListManga	81
Bảng 33. Bảng ListFollower.....	81
Bảng 34. Bảng Covers	82
Bảng 35. Bảng MangaCovers	83
Bảng 36. Bảng ChatMessage.....	84
Bảng 37. Bảng ChatRoom.....	85
Bảng 38. Bảng ChatRoomMember	86
Bảng 39. Bảng Friendship	86
Bảng 40. Bảng UserPresence	87
Bảng 41. Ví dụ ma trận User – Item.....	110

DANH SÁCH CÁC KÝ TỰ, CÁC CHỮ VIẾT TẮT

Từ viết tắt	Giải thích
UC	Use Case
CSDL (DB)	Cơ sở dữ liệu (Database)
PK	Primary Key (Khóa chính)
AI	Artificial Intelligence (Trí tuệ nhân tạo)
API	Application Programming Interface (Giao diện lập trình ứng dụng)
ETL	Extract, Transform, Load (Trích xuất, Biến đổi, Tải dữ liệu)
JWT	JSON Web Token (Mã thông báo web JSON dùng để xác thực)
NSFW	Not Safe For Work (Nội dung nhạy cảm/không an toàn)
OCR	Optical Character Recognition (Nhận dạng ký tự quang học)
OOM	Out of Memory (Lỗi tràn/hết bộ nhớ RAM)
UI/UX	User Interface / User Experience (Giao diện người dùng / Trải nghiệm người dùng)

CHƯƠNG 1: TÀI LIỆU KIẾN TRÚC

1.1. Sơ lược về tài liệu kiến trúc

Tài liệu Kiến trúc phần mềm/ Software Architecture Document

Phiên bản: 1.0

Tên dự án: Xây dựng Web – App Quản lý Dịch vụ đọc Truyện tranh Online (Manga)

Nhóm thực hiện: Nhóm 17

Ngày phát hành: 23/03/2026

Lịch sử sửa đổi (Revision History)

Date	Version	Description	Authors
12/01/2026	0.1	- Xác định các bên liên quan - Xác định yêu cầu cho tất cả chức năng mà hệ thống có thể có	Nhóm 17
02/03/2026	0.2	- Tên của mẫu kiến trúc chính được chọn, gồm các mẫu kiến trúc hỗ trợ, bổ sung cho hệ thống. - Vẽ biểu đồ mô tả trực quan các kiến trúc đó dưới dạng hình khối. - Liệt kê đầy đủ tên các thành phần (component) trong mỗi kiến trúc đó. Mô tả rõ chức năng của mỗi thành phần (component) của mỗi kiến trúc. - Gọi tên các công nghệ được chọn (nếu có, bao gồm phần cứng và phần mềm) để sử dụng trong các thành phần trong mỗi kiến trúc.	Nhóm 17
09/03/2026	0.3	- Thiết kế View kiến trúc Logic (Logical Views) - Thiết kế View kiến trúc Cài đặt (Implementation Views)	Nhóm 17

		– Thiết kế View kiến trúc Tiến trình (Process Views) – Thiết kế View kiến trúc triển khai/Vật lý (Deployment Views)	
18/03/2026	0.4	Xác thực kiến trúc	Nhóm 17
23/03/2026	1.0	Tổng hợp, hoàn thiện và đóng gói tài liệu SAD cuối cùng.	Nhóm 17

Tên dự án: *Xây dựng Web – App Quản lý Dịch vụ đọc Truyện tranh Online*

1.2. Ngữ cảnh dự án

Hiện nay, cộng đồng đọc truyện tranh (Manga/Manhwa/Comic) tại Việt Nam và quốc tế rất lớn. Tuy nhiên, các hệ thống website đọc truyện hiện tại (hệ thống cũ hoặc của các bên đối thủ) đang gặp phải nhiều vấn đề nhức nhối:

- Hiệu năng kém và gián đoạn dịch vụ: Máy chủ thường xuyên bị quá tải (Crash/Downtime) khi có một bộ truyện hot ra chapter mới. Tốc độ tải ảnh chậm, gây ức chế cho người dùng.
- Quản lý dữ liệu phân tán thủ công: Quá trình thu thập (crawl) truyện từ các nguồn khác (Mangadex, Bato...) thiếu tự động hóa, dễ bị lỗi format dữ liệu, dẫn đến rác dữ liệu trong Database.
- Thiếu kiểm duyệt nội dung: Hình ảnh nhạy cảm (NSFW), bạo lực hoặc bình luận mang tính chất toxic/spoiler không được kiểm soát tốt do dùng sức người.
- Trải nghiệm người dùng nghèo nàn: Chưa có hệ thống gợi ý truyện thông minh (Recommend system) được cá nhân hóa; thiếu các tính năng tương tác thời gian thực (Real-time chat).

Dự án này ra đời cung cấp một nền tảng đọc truyện thể hệ mới, giải quyết triệt để các bài toán về hiệu suất và tự động hóa. Dự án đáp ứng các kỳ vọng:

- Trải nghiệm mượt mà: Tốc độ tải trang dưới 2 giây, hỗ trợ đọc offline, lật trang/cuộn đọc không độ trễ.
- Khả năng mở rộng (Scalability): Chịu tải hàng chục ngàn Concurrent Users trong giờ cao điểm mà không sập hệ thống.
- Tự động hóa vận hành: Tích hợp AI để tự động kiểm duyệt ảnh (Swin Transformer), nhận diện chữ (OCR) hỗ trợ dịch thuật, và sử dụng ETL Pipeline tự động crawl/chuẩn hóa dữ liệu.
- Tương tác thời gian thực: Trở thành một mạng xã hội thu nhỏ cho giới yêu truyện với tính năng chat, thông báo đẩy tức thì khi có chương mới.

1.3. Những yêu cầu kiến trúc

1.3.1. Những mục tiêu chính của dự án

- Xây dựng nền tảng (Web/App) cho phép người dùng đọc truyện, tương tác và tải truyện ngoại tuyến.
- Xây dựng hệ thống Backend mạnh mẽ với kiến trúc Microservices để dễ dàng bảo trì, mở rộng và tích hợp AI.
- Tối ưu hóa chi phí lưu trữ ảnh thông qua Object Storage (MinIO).

1.3.2. Những yêu cầu về chức năng của các bên liên quan

Đối với Độc giả:

- Tìm kiếm & Trải nghiệm đọc: Tìm kiếm/lọc truyện theo thể loại, tác giả, đánh giá, ngôn ngữ. Hỗ trợ chế độ đọc linh hoạt (cuộn đọc, lật trang, zoom panel cho ảnh màu/đen trắng) và dịch tự động text trực tiếp trên trang truyện.
- Quản lý cá nhân: Tạo, quản lý (CRUD) danh sách đọc truyện cá nhân (Public/Private); theo dõi (Follow/Unfollow) danh sách của user khác. Cung cấp công cụ tìm kiếm lịch sử đọc theo khoảng thời gian/metadata.

- Tương tác & Cộng đồng: Thêm/sửa/xóa, like/dislike và báo cáo bình luận; đánh giá điểm số. Tích hợp tính năng Chat real-time với các người dùng (và Admin) trong hệ thống.
- Cập nhật & Ngoại tuyến: Bookmark, theo dõi updates, nhận thông báo Push khi có chapter mới và cho phép tải chapter về đọc ngoại tuyến.
- Gợi ý truyện (Recommend): Hệ thống tự động gợi ý truyện cá nhân hóa dựa trên lịch sử đọc và gợi ý các bộ truyện tương tự với bộ đang xem.

Đối với Người Upload bản dịch (Uploaders):

- Quản lý đăng tải: Cung cấp công cụ upload batch bản dịch các chapter với các định dạng (JPEG, PNG, WebP) một cách dễ dàng.
- Quản lý danh sách: Quản lý danh sách các bản dịch đã đăng tải và hiển thị thông tin public của uploader để độc giả có thể theo dõi.

Đối với Quản trị viên:

- Quản lý User & Content: Dashboard theo dõi xu hướng/lịch sử của user, thực hiện block/unblock tài khoản hoặc block những user đăng nội dung vi phạm. Quản lý, duyệt hoặc xóa các chapter vi phạm bản quyền/chuẩn mực. Xử lý các comment bị báo cáo.
- Vận hành hệ thống: Quản lý toàn bộ danh sách truyện trong hệ thống, bao gồm tùy chọn duyệt/cập nhật trực tiếp từ API bên thứ 3 (Mangadex, Bato...). Xem Dashboard báo cáo Analytics (Traffic, User growth, Popularity).

Đối với Developers / Maintainers:

- Bảo trì & Mở rộng: Cung cấp RESTful API cho bên thứ 3 (Mobile App) tích hợp. Quản lý cấu hình CI/CD và giám sát hệ thống.
- AI Training: Cung cấp công cụ/luồng hỗ trợ định kỳ train lại tất cả các mô hình AI trong hệ thống dựa trên tập dữ liệu mới.

Đối với Data Source bên thứ 3 (Hệ thống ETL): Tự động crawl và thực hiện ETL dữ liệu chapter truyện và thông tin Metadata từ Mangadex API hoặc cào thủ công từ bato, truyệnqq...

1.3.3. Những yêu cầu về chất lượng (phi chức năng) của các bên liên quan

Ràng buộc về Hiệu suất:

- Tốc độ phản hồi: Thời gian tải trang hiển thị phải dưới 2 giây. Các truy xuất biểu đồ analytics phức tạp phải hoàn thành dưới 15 giây (yêu cầu tích hợp ELK Stack để tối ưu truy vấn logs/dữ liệu).
- Xử lý tài nguyên tĩnh: Ảnh trong chapter phải được load bất đồng bộ (Lazy-load/Asynchronous) để đảm bảo không gián đoạn thao tác đọc các ảnh kích thước lớn. Quá trình xử lý dịch text bằng AI phải trả về kết quả dưới 15 giây/ảnh.

Khả năng mở rộng và Chịu tải:

- Backend phải chịu tải được hàng nghìn request cùng lúc, bắt buộc sử dụng Apache Kafka + Redis; ưu tiên khả năng mở rộng theo chiều ngang (Horizontal Scaling).
- Dữ liệu hình ảnh và phi cấu trúc phải được lưu trữ trên Object Storage giống S3 (S3-like storage: MinIO (trong quá trình development vì miễn phí) hoặc AWS S3 (trên production nếu có dư tiền, không thì lại quay về dùng MinIO tiếp)) để giảm tải băng thông cho server chính. Dữ liệu đọc offline được cache qua một DB trung gian (Redis) hoặc load trực tiếp từ MinIO Data lake.

Độ tin cậy và Tính sẵn dùng:

- Rollback tự động: Chức năng tự động cập nhật truyện toàn hệ thống (qua Mangadex API) phải có cơ chế tự động Rollback lại toàn bộ dữ liệu sạch trước đó nếu server bị down giữa chừng.
- Backup: Hệ thống tự động lập lịch backup toàn bộ DB đẩy lên Cloud (AWS S3 nếu đến lúc đó còn đủ tiền trả cho dịch vụ, không thì dùng tạm GG Big Query) mà không cần sự can thiệp thủ công.

Ràng buộc về Bảo mật (Security & Privacy):

- Thiết kế RESTful API với chuẩn Auth (OAuth/JWT). Hệ thống có cơ chế tự động Block hoặc tạm thời giới hạn (Rate Limiting) truy cập từ các IP spam bất thường/DDoS.
- User bị Block sẽ bị cưỡng chế đăng xuất khỏi hệ thống ngay lập tức.
- Dữ liệu lịch sử đọc cá nhân có thể được cấp quyền (Toggle) hiển thị/ẩn với người dùng khác. Bình luận có bộ lọc từ cấm tự động và tính năng toggle ẩn/hiện văn bản chứa Spoiler.

Ràng buộc Kiến trúc Công nghệ & AI (Technical & AI Constraints):

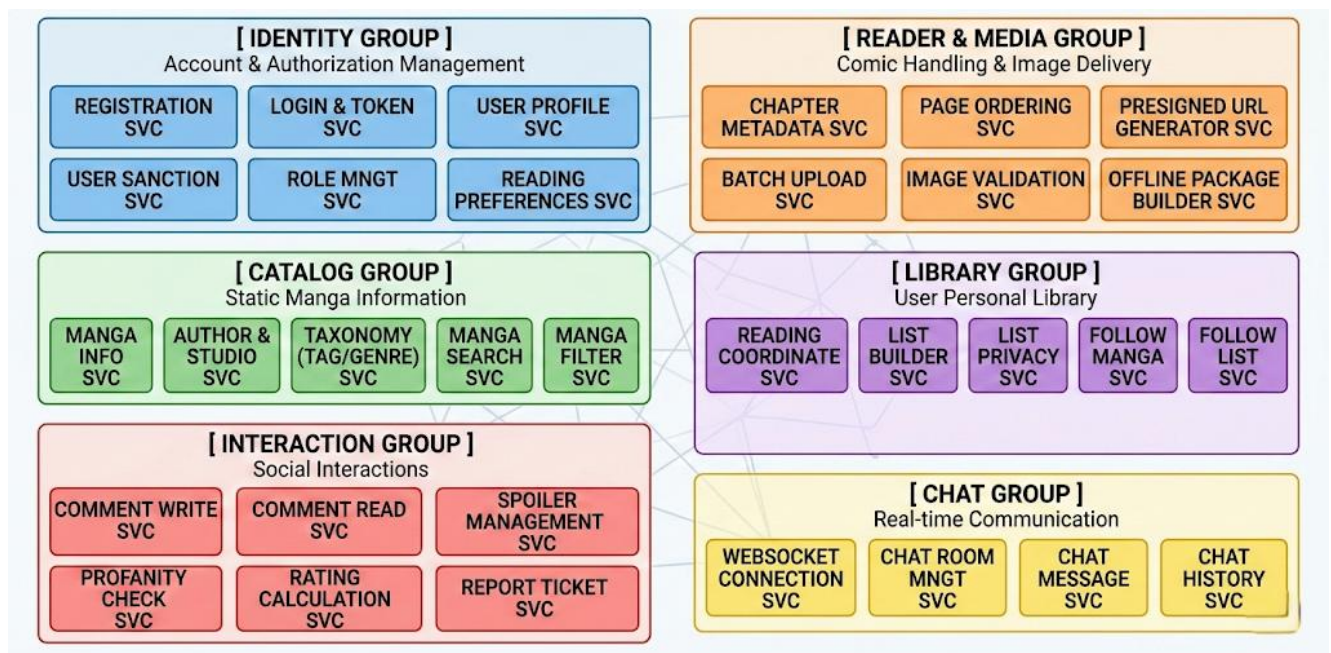
- Cấu trúc Database: Bắt buộc sử dụng UUID v4 làm khóa chính cho mọi bảng trong CSDL để dễ dàng đồng bộ định dạng với dữ liệu cào từ Mangadex API.
- AI Kiểm duyệt: Sử dụng mô hình Swin Transformer v2 quét ảnh NSFW. Yêu cầu thời gian quét tối đa < 5 phút cho mỗi chương. Vi phạm sẽ tự đẩy ticket cho Admin.
- AI Dịch thuật: Kết hợp OCR Scan và Google AI Mode (qua thư viện Playwright nếu cần), trong quá trình dev thì dùng tạm Google Translate API, nếu thấy ổn thì khởi xài Google AI Mode.
- AI Gợi ý: Thuật toán gợi ý cá nhân hóa dựa trên query tĩnh (lấy max 20 truyện rating cao chứa 5 tags xem gần nhất). Thuật toán gợi ý truyện tương tự dùng BERTopic để phân cụm các comment/review đã được tiền xử lý (lọc nhiễu). Model AI cần được train định kỳ 6 tháng 1 lần.
- ETL Pipeline & Automation: Chạy Job lập lịch thu thập metadata vào "*Tối chủ nhật đầu tiên của quý*" và nội dung chapter vào "*Tối thứ 2 đầu tiên của quý*". Áp dụng cơ chế Full Load cho quý đầu tiên và Incremental Load cho các chu kỳ sau. Push notification (Kafka/WebSocket) phải alert thời gian thực khi có update.
- Nghiệp vụ độ trễ: Admin phải đợi 5 phút giữa 2 lần thao tác Block/Unblock trên cùng một user. Báo cáo comment bị Admin ignore sẽ có thời gian chờ 24 tiếng trước khi người dùng có thể report lại comment đó.

1.4. Trình bày kiến trúc

1.4.1. Các mẫu kiến trúc được sử dụng trong dự án

- Kiến trúc chính: Microservices Architecture Pattern. Lý do: Mỗi dịch vụ xử lý một nghiệp vụ nhỏ và sở hữu Database (hoặc bảng/schema) riêng lẻ. Đảm bảo một service lỗi không làm sập các tính năng khác.
- Kiến trúc hỗ trợ 1: Client – Server Architecture Pattern. Lý do: Trình duyệt web (Client) sẽ đóng vai trò thiết lập giao diện và trực tiếp gửi request (HTTP/REST) tới các cụm máy chủ Microservices (Server) để xin cấp phát tài nguyên (ảnh, text).
- Kiến trúc hỗ trợ 2: Broker Architecture Pattern. Lý do: Giải quyết bài toán giao tiếp giữa hàng chục service nhỏ. Sử dụng Broker để điều phối các sự kiện.
- Kiến trúc hỗ trợ 3: Pipe-Filter Architecture Pattern. Lý do: Xử lý một chuỗi transformation dữ liệu từ nơi này sang nơi khác để lưu và dùng.

1.4.2. Kiến trúc tổng quan của dự án



Hình 1. Kiến trúc Microservice tổng quan của dự án

Service	Chức năng
Registration Svc Login Svc	Quản lý tạo tài khoản mới và cấp phát JWT
User Profile Svc	CRUD avatar, tên hiển thị, bio
User Sanction Svc	Lưu trữ trạng thái Block/Unblock, quản lý bộ đếm lùi 5 phút cấm thao tác, kick session.
Role Mngt Svc	Xác định ai là Admin, ai là Uploader.
Reading Preferences Svc	Lưu cấu hình đọc ví dụ "chế độ zoom panel" của độc giả.

Bảng 1. Nhóm Identity (Tài khoản & Phân quyền)

Service	Chức năng
Manga Info Svc	CRUD thông tin cơ bản: Tên, tóm tắt
Author & Studio Svc taxonomy Svc	Chuẩn hóa tên tác giả và hệ thống Tag, Ngôn ngữ (Anh/Việt)
Manga Search Svc Manga Filter Svc	Chuyên trị query tìm kiếm text và lọc theo điều kiện phức tạp.

Bảng 2. Nhóm Catalog (Thông tin truyện tĩnh)

Service	Chức năng
Chapter Metadata Svc	Lưu thông tin chap mấy, thuộc volume nào, tên chapter.
Page Ordering Svc	Quản lý số thứ tự các trang ảnh trong một chapter để UI render không bị lộn xộn.

Batch Upload Svc	Mở port cho uploader đẩy file hàng loạt lên RAM máy chủ.
Image Validation Svc	Check MIME type (JPEG, PNG, WebP) và dung lượng file.
Presigned URL Generator Svc	Sinh ra các link tạm thời từ kho ảnh MinIO cho UI load trực tiếp (không qua server Backend).
Offline Package Builder Svc	Đóng gói các ảnh của chapter thành file nén trả về cho UI.

Bảng 3. Nhóm Reader & Media (Truyền tải ảnh & Upload)

Service	Chức năng
Comment Write Svc Comment Read Svc	Tách biệt luồng ghi và luồng đọc comment.
Spoiler Management Svc	Xử lý cờ (flag) ẩn/hiện comment spoil nội dung truyện.
Profanity Check Svc	Lọc từ cấm trước khi đẩy comment vào DB.
Rating Calculation Svc Report Ticket Svc	Tính trung bình sao; Quản lý ticket báo cáo, quản lý timer chờ 24h nếu bị ignore.
Chat Svc (WS, Room, Message)	Mở luồng WebSocket, phân luồng phòng chat, lưu tin nhắn real-time giữa users.

Bảng 4. Nhóm Interaction & Chat (Tương tác xã hội)

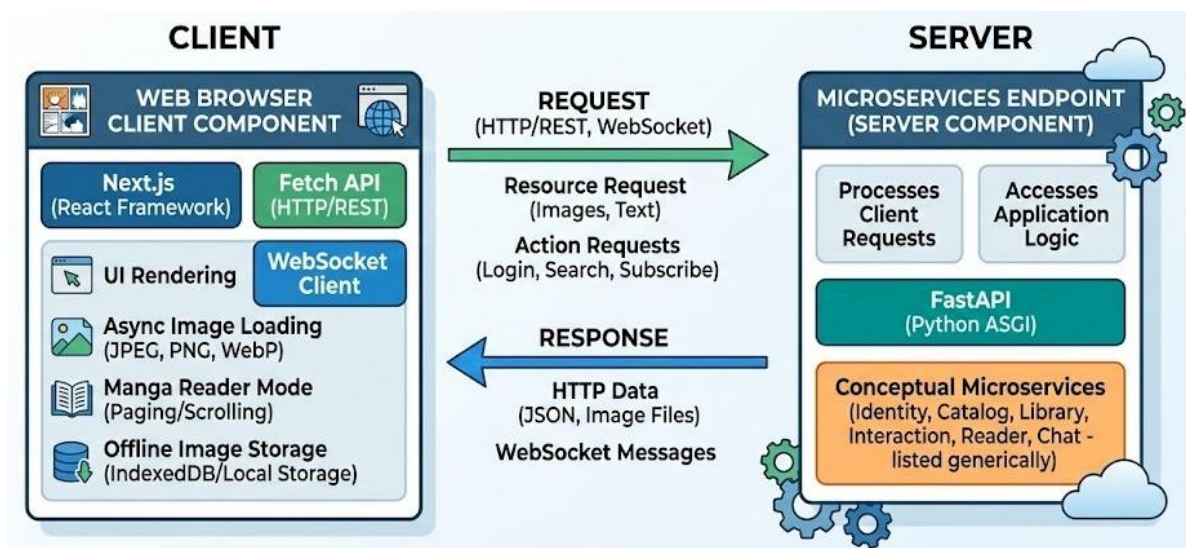
Service	Chức năng
Reading Coordinate Svc	Liên tục ghi nhận (Append-only) người dùng đang đọc tới

	trang nào, hình nào (Yêu cầu 12).
List Builder Svc List Privacy Svc	Tạo các danh sách Manga tùy chỉnh, cấu hình Private/Public.
Follow Manga Svc Follow List Svc	Ghi nhận đăng ký theo dõi truyện hoặc list người khác.

Bảng 5. Nhóm Library (Thư viện cá nhân của người dùng)

Công nghệ sử dụng:

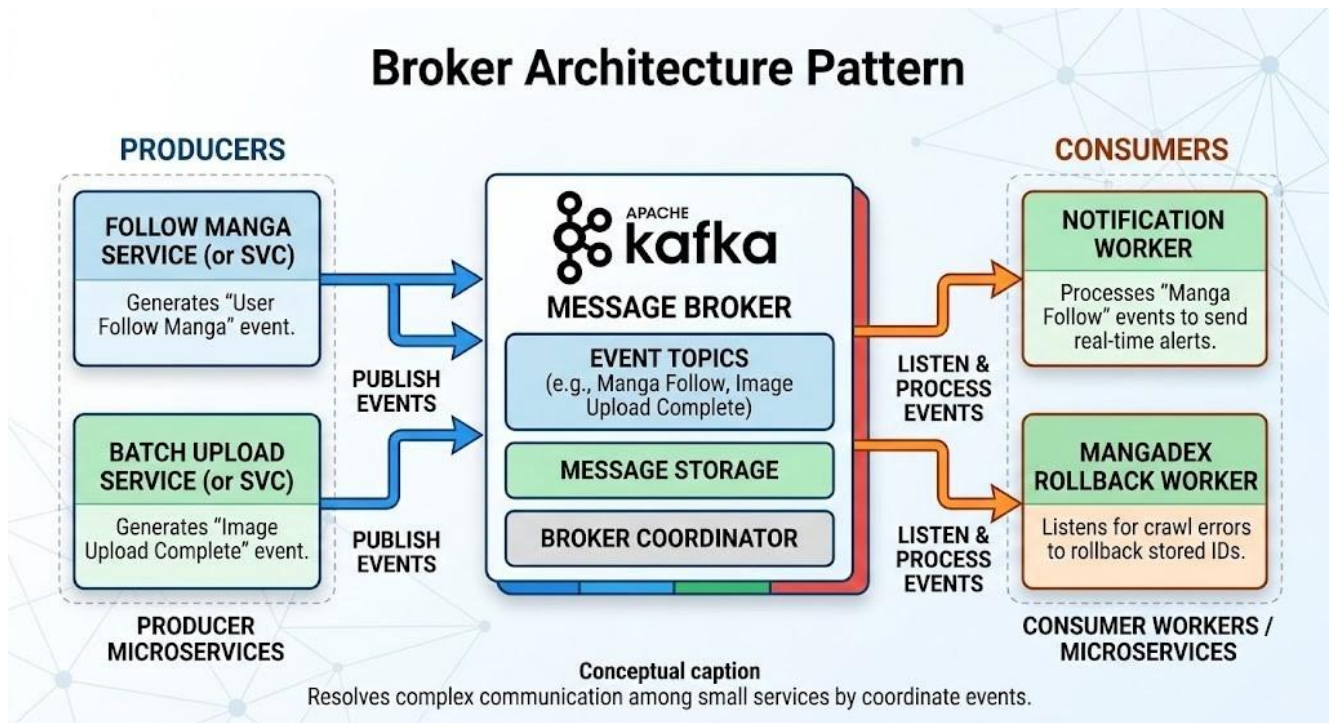
- Framework: FastAPI.
- ORM: SQLAlchemy.
- WebSockets: Sử dụng FastAPI WebSockets module.



Hình 2. Mẫu kiến trúc client - server của dự án

Web Browser Client Component: Render giao diện. Xử lý logic tải ảnh bất đồng bộ, xử lý chế độ đọc lật trang/cuộn đọc, lưu trữ file ảnh zip ngoại tuyến vào IndexedDB/Local storage trình duyệt.

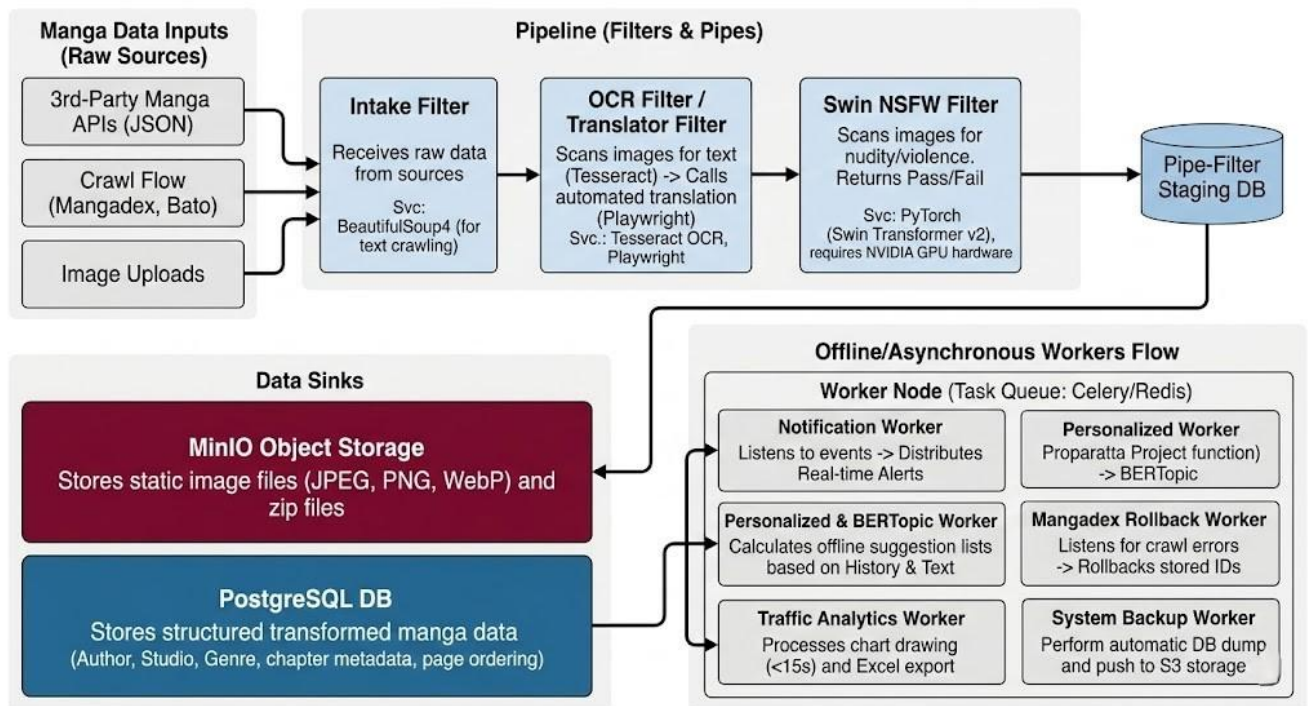
Công nghệ sử dụng: Next.js (React framework), giao tiếp qua Fetch API và WebSocket.



Hình 3. Mẫu kiến trúc dùng để giao tiếp giữa các microservice

Apache Kafka Broker giao tiếp sự kiện. Khi Follow Manga Svc ghi nhận user bấm theo dõi, hoặc Batch Upload Svc báo nhận ảnh xong → Tạo sự kiện đẩy vào Kafka.

Công nghệ sử dụng: Apache Kafka: Xử lý chịu tải message. Dùng thư viện aiokafka để kết nối từ FastAPI.

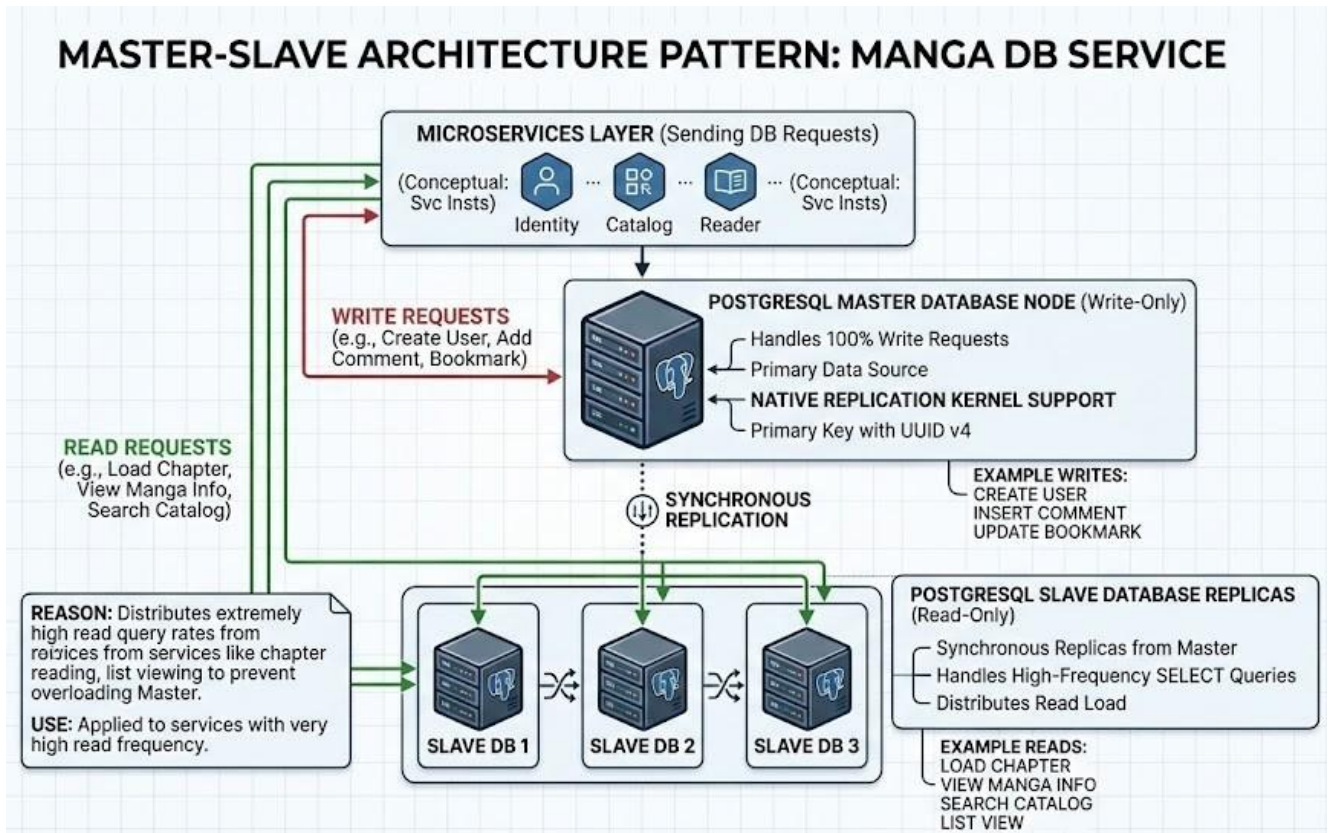


Hình 4. Mẫu Pipeline để ETL dữ liệu

- Intake Filter: Nhận data thô từ luồng Crawl (Mangadex) hoặc Upload.
- OCR Filter / Translator Filter: Quét chữ trên ảnh (Tesseract) → Gọi Playwright dịch.
- Swin NSFW Filter: Quét ảnh khỏa thân/bạo lực. Trả kết quả pass/fail.
- Notification Worker: Lắng nghe Kafka, phân phát Alert thời gian thực.
- Personalized & BERTopic Worker: Tính toán offline danh sách truyện gợi ý dựa trên Lịch sử & Text.

Công nghệ sử dụng:

- Task Queue: Celery chạy cùng Redis broker.
- AI (Hình ảnh & NLP): PyTorch (cho mô hình Swin Transformer v2), BERTopic.
- ETL & Dịch thuật: BeautifulSoup4 (Cào text), Playwright (Cào tự động Google Translate, GG AI Mode), Tesseract OCR (Trích xuất chữ từ ảnh).
- DBT



Hình 5. Mẫu kiến trúc Master – Slave tách biệt luồng đọc – ghi để giảm tải cho DB

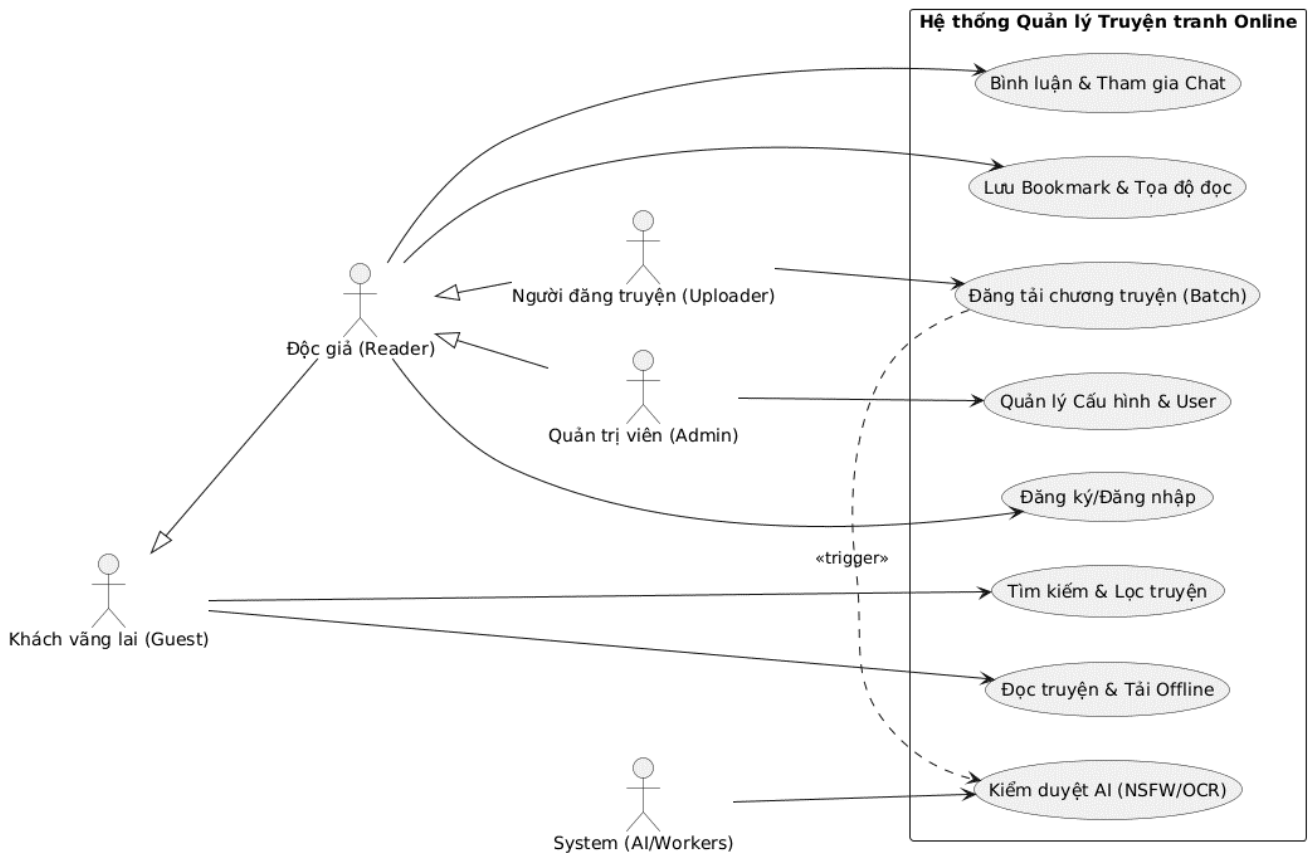
- PostgreSQL Master: Nút cơ sở dữ liệu xử lý 100% các request Write (Tạo user, comment, bookmark...). Khóa chính dùng UUID v4.
- PostgreSQL Slaves: Bản sao đồng bộ từ Master. Mọi Microservices khi cần thực hiện lệnh SELECT (như load chương, tìm truyện) đều query vào Slaves.
- Mangadex Rollback Worker: Lắng nghe Kafka, nếu cào bị lỗi sẽ rollback toàn bộ ID đã lưu.
- Traffic Analytics & System Backup Worker: Xử lý vẽ biểu đồ (<15s), xuất Excel, dump DB đẩy lên S3 tự động.

Công nghệ sử dụng:

- PostgreSQL: Quản lý Master-Slave native replication.
- Redis: Phân tán cache, lưu session chat.

- MinIO: Object Storage lưu trữ ảnh tĩnh chuẩn S3.

1.4.3. Thiết kế View kiến trúc Logic (Logical Views)

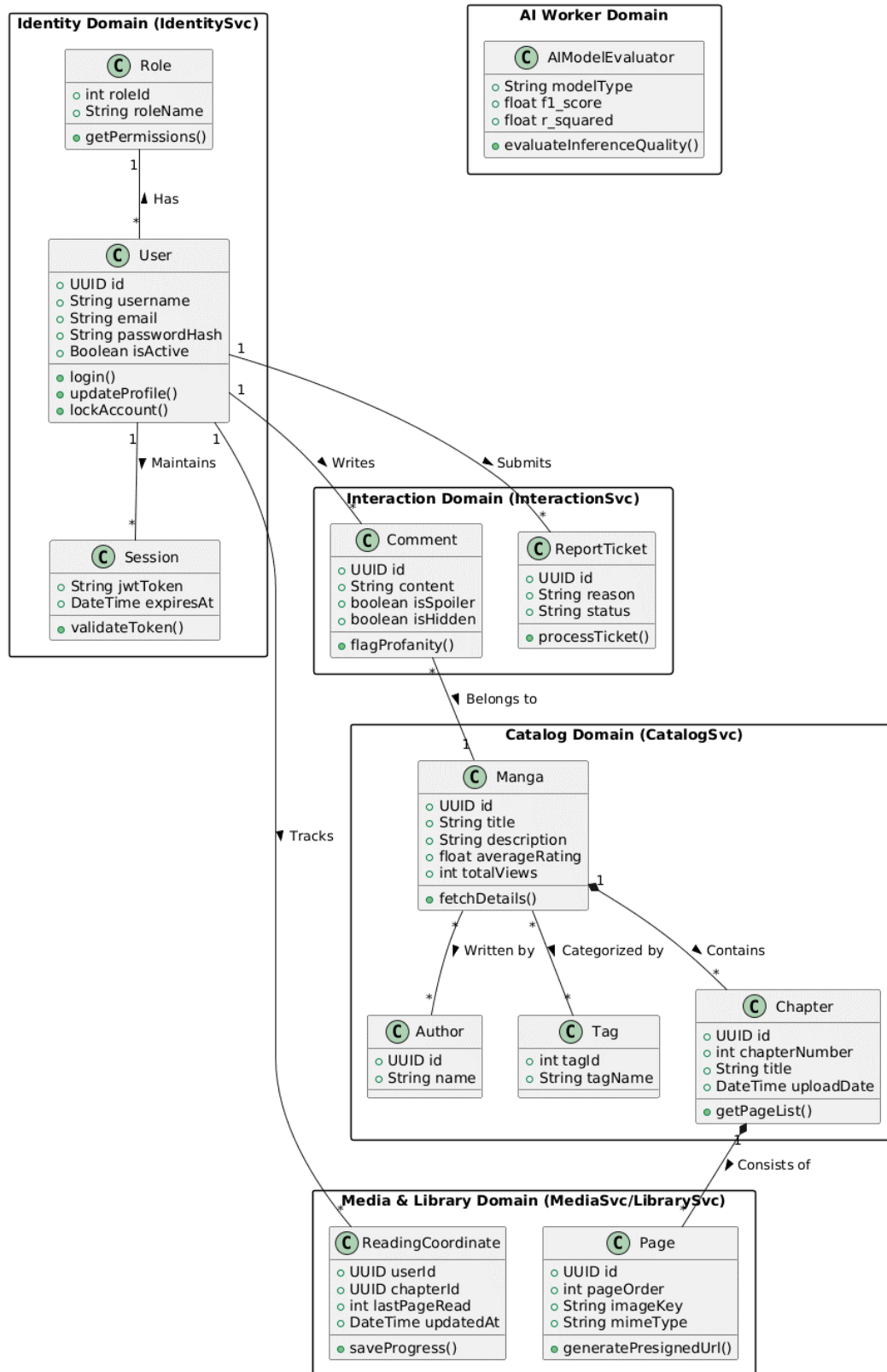


Hình 6. Biểu đồ usecase tổng thể

Giải thích các Use case tổng thể:

- Đăng ký/Đăng nhập: Quản lý định danh, cấp phát token JWT, xác thực quyền hạn.
- Tìm kiếm & Lọc truyện: Truy vấn danh sách truyện theo thể loại, tác giả, trạng thái, (tối ưu hóa trên các Slave Database).
- Đọc truyện & Tải Offline: Tải metadata của chương, nhận Presigned URL từ MinIO để lấy ảnh trực tiếp, đóng gói ảnh tải ngoại tuyến thành file .zip gửi cho độc giả.

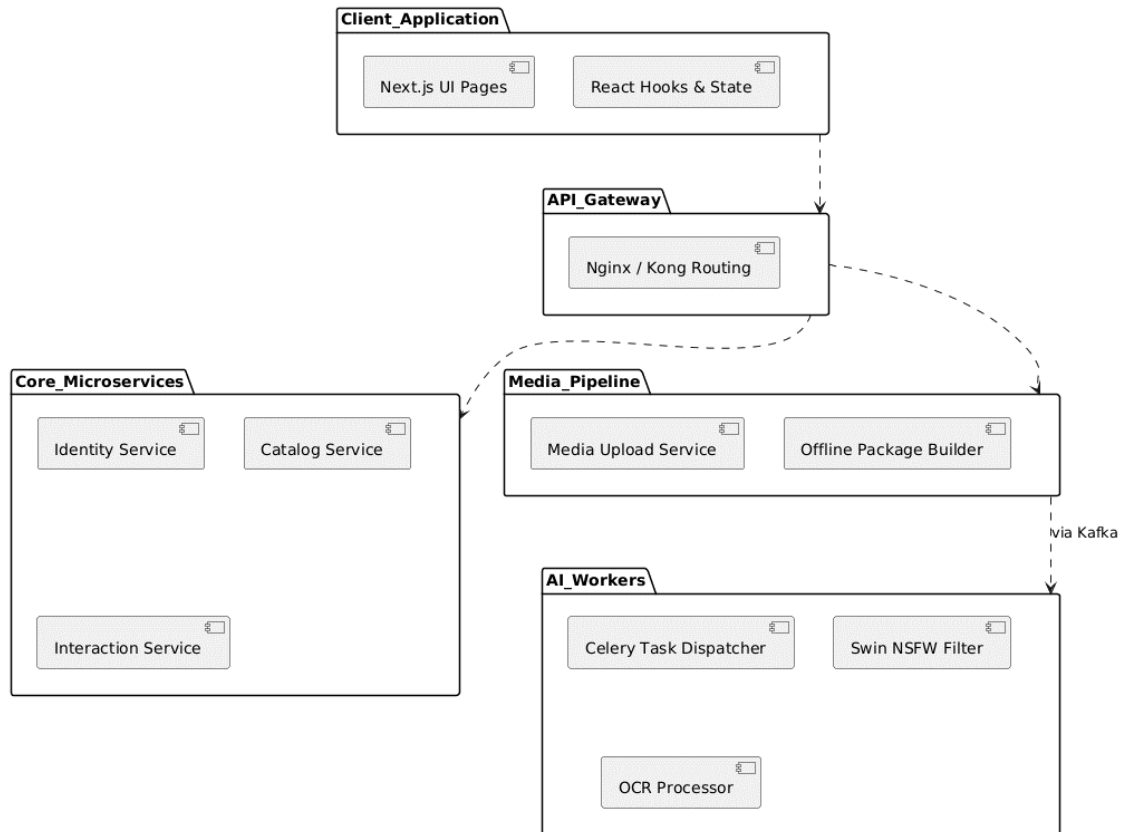
- Bình luận & Tham gia Chat: Đăng bình luận, đánh dấu spoiler, và duy trì luồng giao tiếp thời gian thực qua WebSocket.
- Lưu Bookmark & Tọa độ đọc: Đánh dấu truyện yêu thích và tự động ghi nhận vị trí trang đang xem.
- Đăng tải chương truyện (Batch): Cho phép Uploader đẩy hàng loạt hình ảnh lên server thông qua luồng upload tối ưu.
- Kiểm duyệt AI (NSFW/OCR): Hệ thống ngầm tự động quét ảnh nhạy cảm và nhận diện ký tự để dịch thuật hoặc tìm kiếm text trên ảnh.
- Quản lý Cấu hình & User: Phân quyền hệ thống, khóa tài khoản vi phạm, cấu hình banner.



Hình 7. Biểu đồ lớp tổng thể

- Package Identity Domain: Chứa các lớp User, Role, Session. Chịu trách nhiệm quản lý thông tin định danh, phân quyền và vòng đời của chuỗi mã hóa JWT.
- Package Catalog Domain: Chứa các lớp Manga, Author, Tag, Chapter. Đây là core của dịch vụ tra cứu, các bộ truyện được gắn tag và liên kết với danh sách các chương.
- Package Media & Library Domain: Chứa các lớp Page và ReadingCoordinate. Lớp Page đảm nhận ánh xạ file vật lý trên MinIO thành các Presigned URL, trong khi ReadingCoordinate lưu vết lịch sử đọc của từng độc giả.
- Package Interaction Domain: Chứa các lớp Comment, ReportTicket. Quản lý dữ liệu tương tác, che nội dung spoiler và xử lý các báo cáo vi phạm.
- Package AI Worker Domain: Chứa lớp AIModelEvaluator. Quản lý các dữ liệu đo lường chất lượng của các mô hình học máy (như mô hình gợi ý, nhận diện ảnh). Bổ sung thêm các metric để kiểm tra mô hình bên cạnh những metric cũ (F1 score, ...) ví dụ như R^2 và các metric liên quan nhằm đảm bảo độ tin cậy của AI.

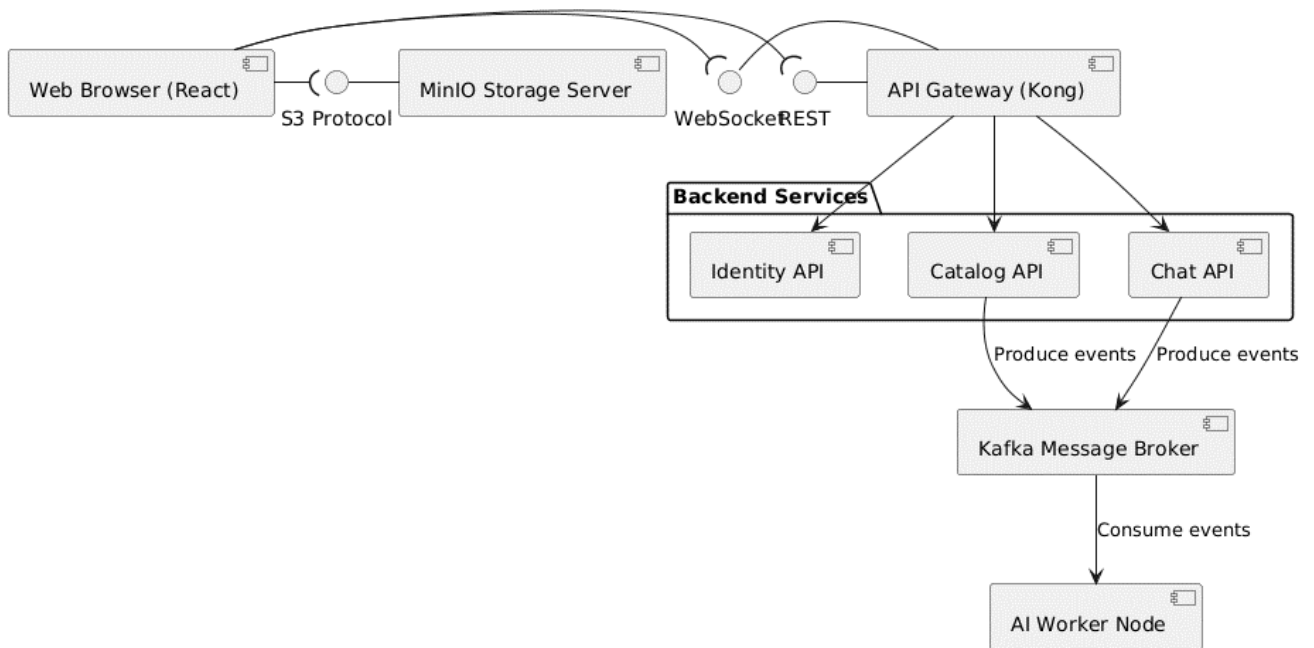
1.4.4. Thiết kế View kiến trúc Cài đặt (Implementation Views)



Hình 8. Biểu đồ gói

- [Next.js UI Pages]: Gói mã nguồn chứa các file giao diện (.tsx/.jsx), chịu trách nhiệm hiển thị HTML tĩnh và định tuyến phía trình duyệt.
- [React Hooks & State]: Gói chứa các hàm logic frontend để quản lý trạng thái tải trang, dữ liệu tạm thời và bộ nhớ đệm.
- [Nginx / Kong Routing]: Gói cấu hình file (.conf), quy định các luật định tuyến, chặn DDoS và giới hạn băng thông cho toàn hệ thống.
- [Identity Service]: Mã nguồn Python (FastAPI) chuyên biệt cho việc mã hóa mật khẩu, tạo token và kiểm tra quyền truy cập.
- [Catalog Service]: Mã nguồn Python xử lý truy vấn tìm kiếm phức tạp, bộ lọc truyện và chuẩn hóa dữ liệu từ database.

- [Interaction Service]: Mã nguồn Python quản lý luồng dữ liệu WebSocket cho tính năng chat thời gian thực và ghi nhận bình luận.
- [Media Upload Service]: Gói mã nguồn mở cổng tiếp nhận file, kiểm tra MIME type, giải nén file zip ảnh tải lên từ Uploader.
- [Offline Package Builder]: Gói mã nguồn đóng gói hàng loạt file ảnh thành các cục nén để Client tải về thiết bị đọc offline.
- [Celery Task Dispatcher]: Gói mã nguồn quản lý hàng đợi, phân phát các tác vụ tốn thời gian từ Kafka đến các node Worker nhận rồi.
- [Swin NSFW Filter]: Gói mã nguồn PyTorch chứa trọng số mô hình Swin Transformer, làm nhiệm vụ phân tích ma trận ảnh để phát hiện nội dung độc hại.
- [OCR Processor]: Gói mã nguồn tích hợp Tesseract để trích xuất văn bản từ khung thoại trong ảnh truyện tranh.



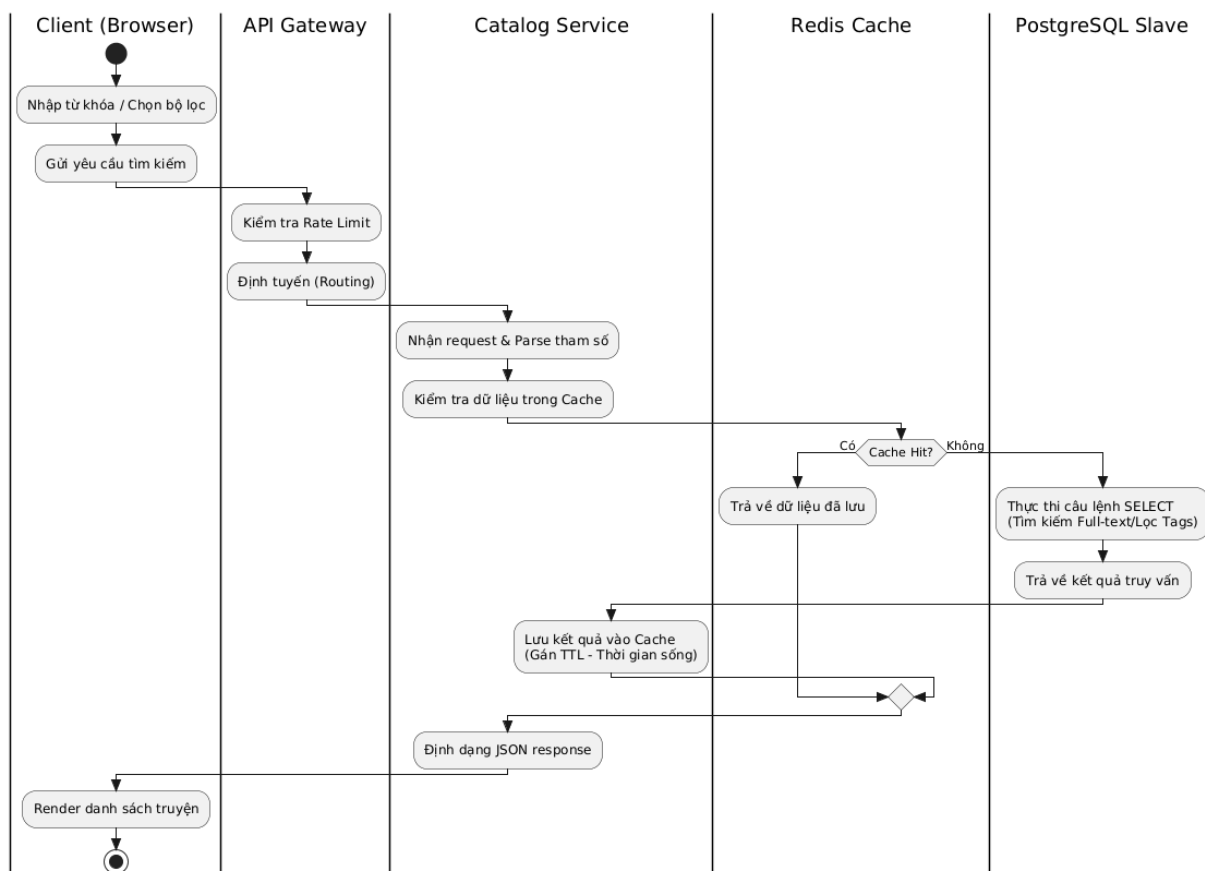
Hình 9. Biểu đồ thành phần

- **Web Browser (React)**: Thành phần thực thi chạy trên máy người dùng cuối, đóng vai trò hiển thị giao diện và trực tiếp gọi các giao thức mạng.

- API Gateway: Thành phần container độc lập, lắng nghe tại cổng 80/443, phân giải URL để trỏ chính xác vào các Service backend bên trong mạng nội bộ.
- Identity API: Thành phần container chạy uvicorn server, sở hữu DB riêng. Xử lý các endpoint bắt đầu bằng /api/v1/auth.
- Catalog API: Thành phần container chịu tải chính cho việc đọc dữ liệu truyện. Xử lý các endpoint bắt đầu bằng /api/v1/manga.
- Chat API: Thành phần container duy trì hàng nghìn kết nối TCP mở (WebSocket) cho tính năng chat. Xử lý qua /ws/chat.
- Kafka Message Broker: Thành phần cụm máy chủ trung gian (Cluster) lưu trữ các thông điệp sự kiện vào các Topic phân tán, đảm bảo không mất mát dữ liệu khi hệ thống quá tải.
- AI Worker Node: Thành phần thực thi chạy ngầm (Daemon), liên tục pull tin nhắn từ Kafka để chạy các thuật toán AI nặng mà không phản hồi trực tiếp cho HTTP request.
- MinIO Storage Server: Thành phần máy chủ lưu trữ Object Storage, thay thế cho ổ cứng cục bộ để lưu trữ và phân phối file ảnh tĩnh (JPEG, PNG) với hiệu suất cao trực tiếp cho Client.

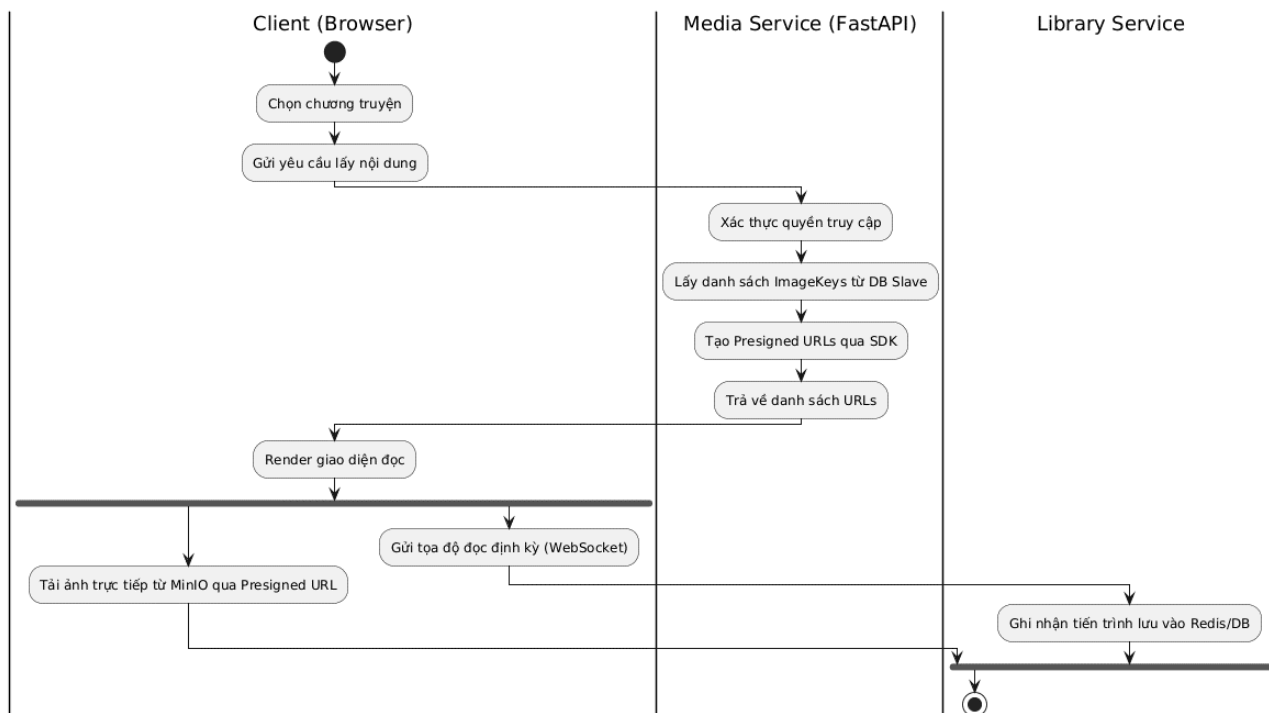
1.4.5. Thiết kế View kiến trúc Tiến trình (Process Views)

Dưới đây là biểu đồ hoạt động (Activity diagram) cho một số chức năng của dự án:



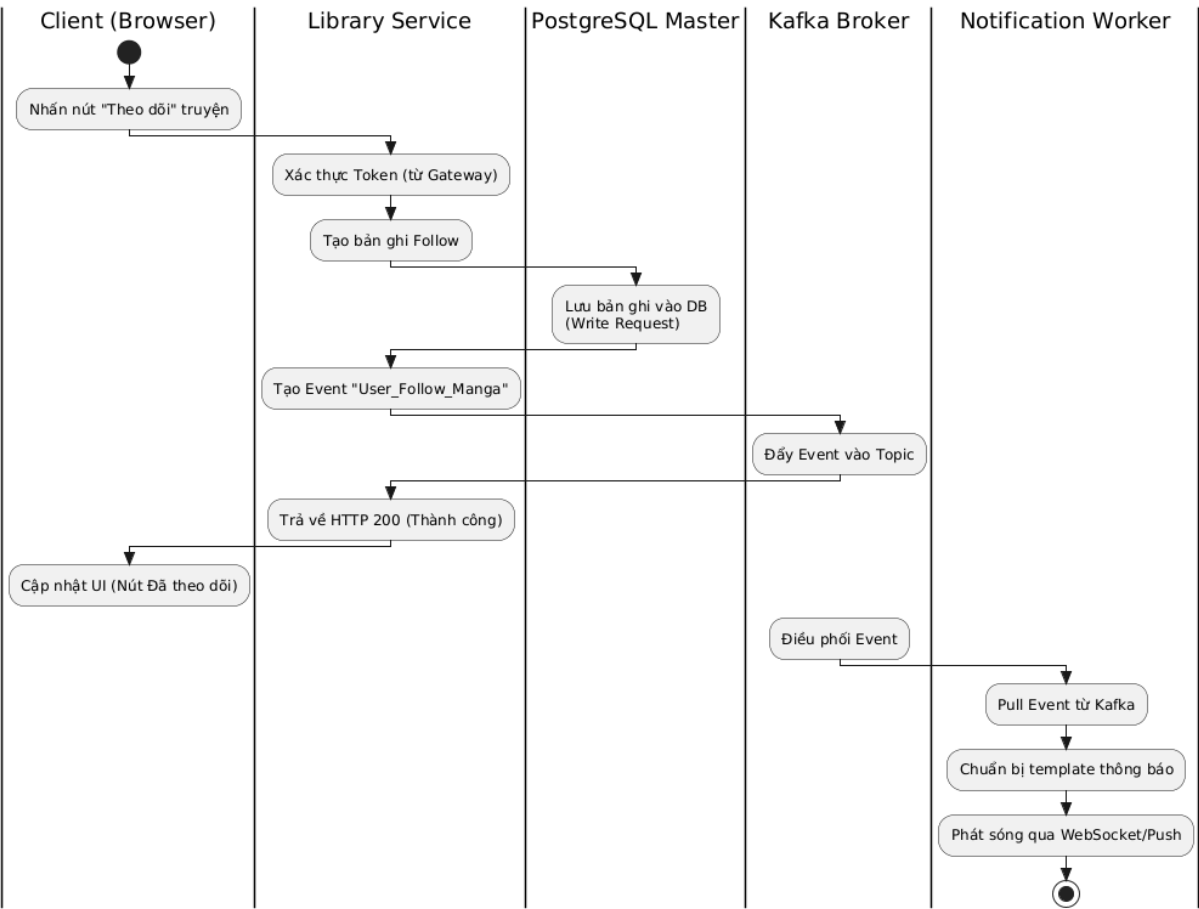
Hình 10. Tiến trình Tìm kiếm & Lọc truyện

Hệ thống tối ưu hóa cực đại cho thao tác Đọc. Khi người dùng tìm kiếm, Catalog Service sẽ ưu tiên kiểm tra trong Redis Cluster trước. Nếu không có dữ liệu (Cache Miss), nó sẽ thực hiện câu lệnh SELECT thảng vào cụm PostgreSQL Slave (chịu trách nhiệm 100% luồng đọc), giúp PostgreSQL Master hoàn toàn rảnh rỗi để xử lý các lệnh Ghi. Dữ liệu sau đó được lưu ngược lại Redis để phục vụ các yêu cầu tương tự tiếp theo.



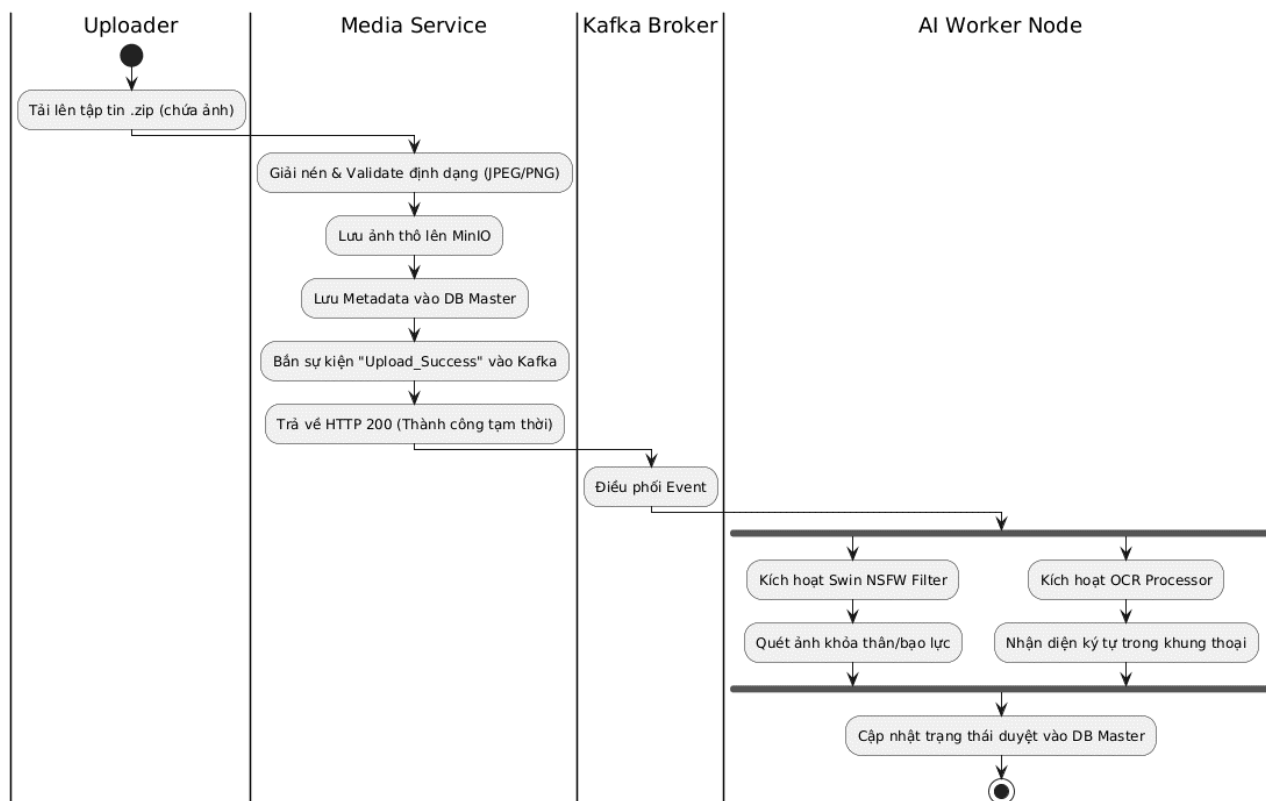
Hình 11. Tiến trình Đọc truyện

Tiến trình này tối ưu hóa việc truyền tải. Thay vì Backend phải đọc file ảnh và gửi về (gây nghẽn băng thông), Media Service chỉ tính toán và sinh ra các *Presigned URLs* (đường dẫn có chữ ký tạm thời). Client sử dụng các đường dẫn này để kéo ảnh trực tiếp từ máy chủ lưu trữ Object Storage. Song song đó, tiến trình đọc liên tục bắn tọa độ về hệ thống để lưu lại vị trí trang hiện tại.



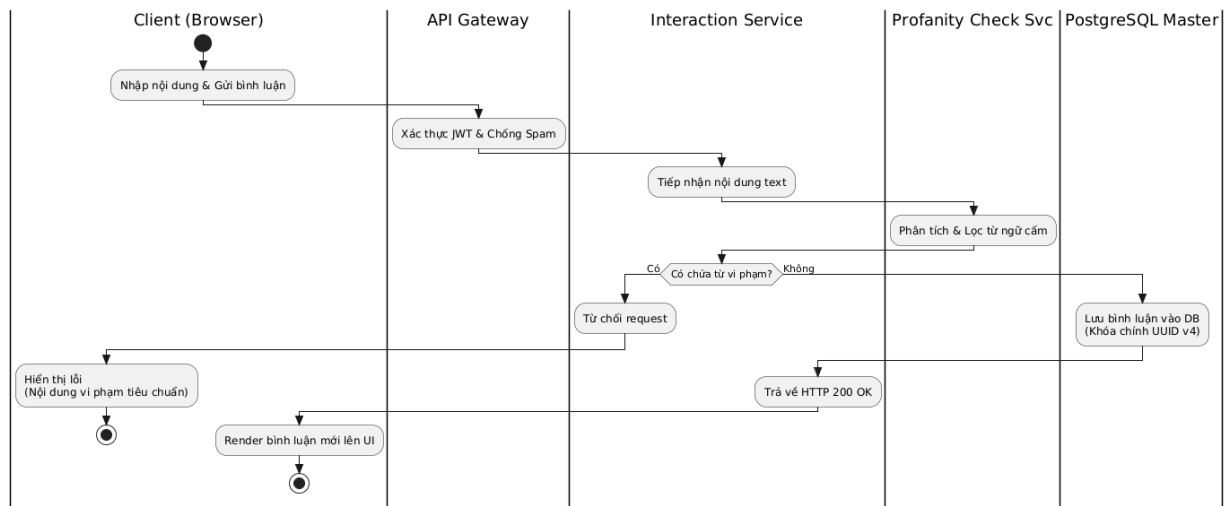
Hình 12. Tiến trình Theo dõi truyện & Xử lý thông báo

Áp dụng xử lý bất đồng bộ (Asynchronous). Khi độc giả bấm theo dõi, Library Service chỉ cần ghi nhận vào Master DB và đẩy một sự kiện vào Apache Kafka Broker rồi lập tức phản hồi thành công cho Client. Người dùng không phải chờ đợi hệ thống xử lý thông báo. Một Notification Worker chạy ngầm sẽ hứng sự kiện này từ Kafka và đảm nhiệm việc broadcast thông báo đầy thời gian thực cho độc giả.



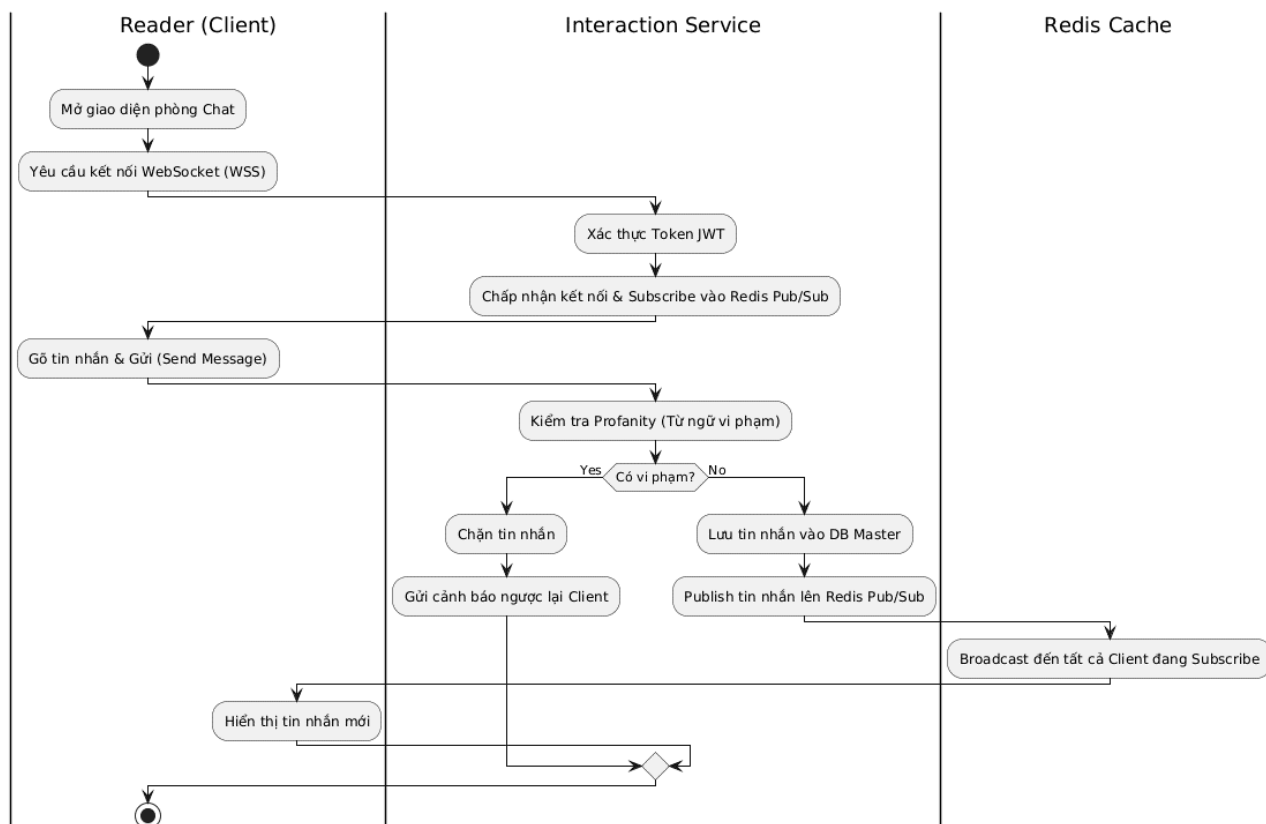
Hình 13. Tiến trình Upload & Xử lý AI ngầm

Áp dụng mẫu kiến trúc Pipe-Filter và Background Jobs. Khi người dùng tải ảnh lên, Backend không bắt người dùng chờ AI xử lý xong. Nó chỉ lưu file tạm thời và báo thành công, sau đó ném sự kiện vào Kafka. Các tiến trình AI ngầm sẽ lấy ảnh từ MinIO xuống để chạy mô hình Swin NSFW (kiểm duyệt) và OCR (nhận diện chữ), sau đó tự động cập nhật trạng thái hiển thị của chương truyện.



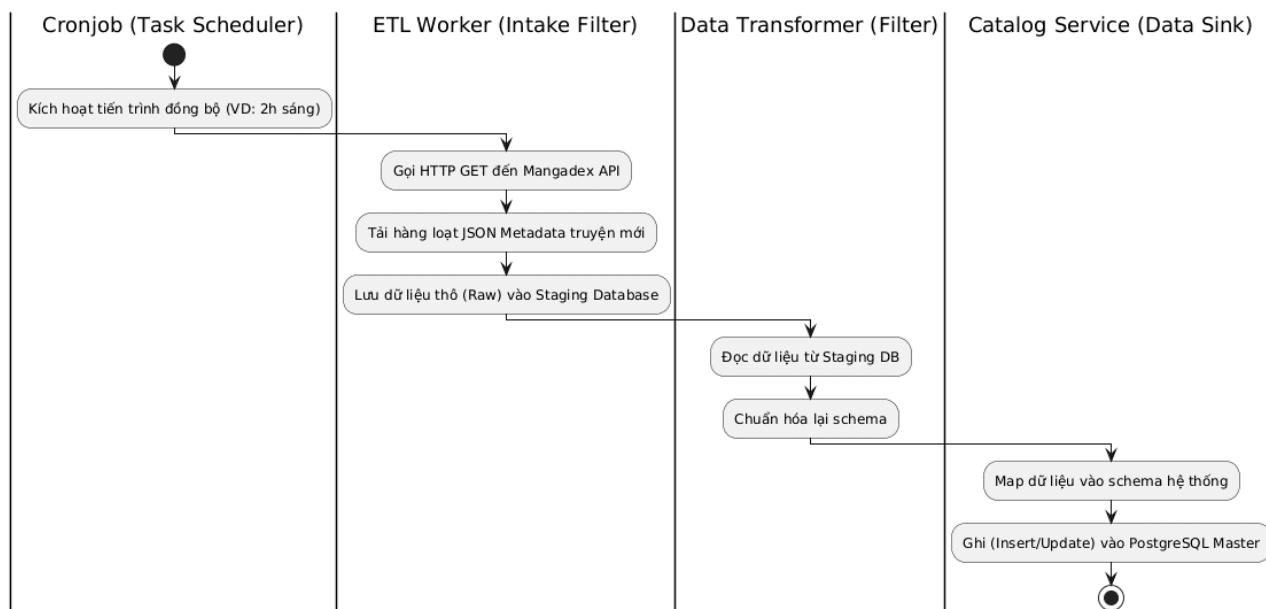
Hình 14. Tiến trình Đăng bình luận & Lọc từ ngữ

Đây là một tiến trình Ghi dữ liệu trực tiếp điển hình. Request của người dùng sau khi qua API Gateway sẽ được Interaction Service tiếp nhận. Trước khi ghi xuống DB, nó phải gọi qua Profanity Check Service để làm sạch và kiểm duyệt từ ngữ độc hại. Chỉ khi dữ liệu hoàn toàn "sạch", nó mới được insert vào PostgreSQL Master Database (sử dụng UUID v4 làm khóa chính). Nếu có lỗi, luồng sẽ bị chặn đứng ngay lập tức để bảo vệ cộng đồng.



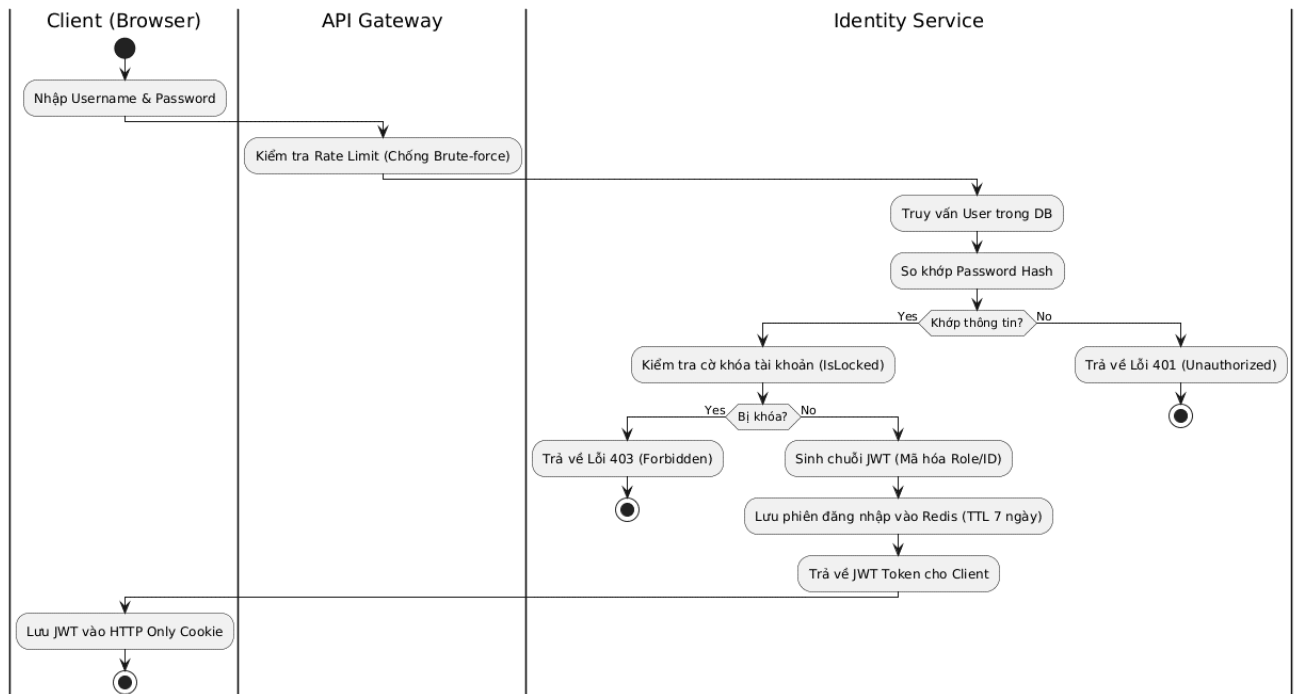
Hình 15. Tiến trình Chat Real-time

Sử dụng giao thức WebSocket kết hợp Redis Pub/Sub để duy trì kết nối luồng kép. Khi một tin nhắn được gửi, nó đi qua bộ lọc từ ngữ trước. Nếu an toàn, tin nhắn được lưu xuống DB và đẩy vào Redis Pub/Sub. Cơ chế Pub/Sub của Redis sẽ chịu trách nhiệm phát sóng (Broadcast) tin nhắn đó đến toàn bộ các Server FastAPI đang giữ kết nối WebSocket với các người dùng khác trong cùng phòng.



Hình 16. Tiến trình Crawl & Đồng bộ dữ liệu ETL (Pipe-Filter)

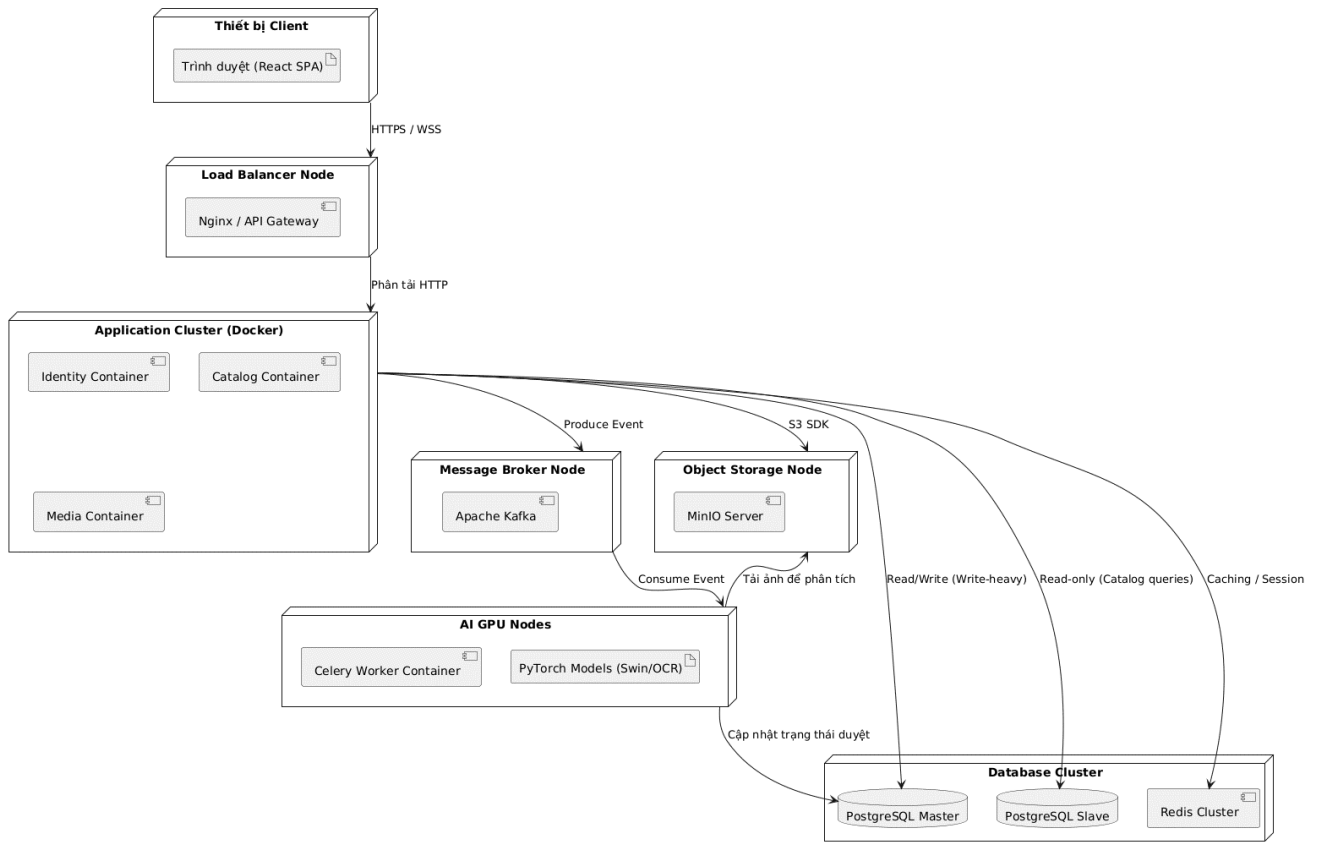
Hệ thống sử dụng một bộ lập lịch (Cronjob) để kích hoạt tự động. Thay vì lưu trực tiếp dữ liệu từ nguồn ngoài vào DB chính gây rủi ro toàn vẹn, luồng xử lý áp dụng ETL (Extract – Transform – Load). Dữ liệu JSON thô đi qua màng lọc (Intake Filter) đổ vào Staging DB, sau đó qua màng lọc chuẩn hóa và dịch thuật, cuối cùng mới được Service chịu trách nhiệm ghi vào DB Master.



Hình 17. Tiến trình Cấp quyền và Xác thực (Identity Workflow)

Mọi yêu cầu xác thực trước tiên phải đi qua API Gateway để đếm số lần thử (Rate Limiting), ngăn chặn tấn công dò mật khẩu. Tại 'Identity Service', sau khi xác minh Hash mật khẩu, hệ thống sinh ra JWT để Client mang theo trong các Request tiếp theo. Phiên đăng nhập được đẩy vào Redis Cache cho phép hệ thống thu hồi token ngay lập tức nếu phát hiện bất thường mà không cần truy vấn lại DB.

1.4.6. Thiết kế View kiến trúc triển khai/Vật lý (Deployment Views)



Hình 18. Các thành phần triển khai vật lý

- Thiết bị Khách: Điện thoại hoặc PC chạy trình duyệt web tải ứng dụng React (SPA).
- Load Balancer Node: Máy chủ vật lý hoặc VM đứng mũi chịu sào (ví dụ Nginx), nhận mọi traffic từ Internet, cung cấp SSL và phân tán tải trọng mạng.
- Application Cluster: Cụm máy chủ chạy Docker Swarm (Nếu còn dư dả thời gian build thì chuyển qua xài K8S ngon hơn). Mỗi máy chủ sẽ host nhiều Container chứa các Microservices (FastAPI). Có khả năng scale tự động.
- AI GPU Nodes: Máy chủ chuyên dụng có trang bị Card đồ họa (GPU). Cài đặt môi trường CUDA để chạy mô hình AI PyTorch (Swin/OCR) qua Celery Worker.
- Message Broker Node: Cụm máy chủ chạy Apache Kafka. Đây là trục xương sống giao tiếp bất đồng bộ, chịu tải sự kiện.

- Object Storage Node: Cụm máy chủ lưu trữ ổ cứng dung lượng cao (HDD/SSD). Chạy MinIO phân tán, chịu trách nhiệm lưu trữ và phục vụ tải ảnh trực tiếp không cần thông qua Application Cluster.
- Database Cluster: Cụm máy chủ cơ sở dữ liệu áp dụng mẫu Master – Slave. Máy chủ Master hứng chịu toàn bộ lệnh Ghi (Write). Máy chủ Slave đồng bộ liên tục từ Master và xử lý 100% các lệnh Đọc (Read - VD: Tải danh sách truyện). Redis Cluster được tách riêng để xử lý truy xuất tức thời vào RAM.

1.5. Tài liệu xác thực kiến trúc

Trước khi giả định, em cần nhấn mạnh lại một số kiến trúc hoặc công nghệ highlight của nhóm đã chọn lựa trước đó. Vì nhóm em chọn nó để giải quyết phần nào các tình huống thực tế có thể xảy ra (nên sẽ được trích dẫn lại rất nhiều trong phản hồi của bảng bên dưới): Microservices, Event – Driven với Kafka, Master – Slave Database, Pipe – Filter cho AI/ETL, Redis Cluster, Minio.

1.5.1. Xác thực yêu cầu về Tính sẵn dùng (Availability)

Thuộc tính chất lượng (Quality Attribute)	Sự cố giả định (Stimulus)	Phản hồi của hệ thống (Response)
1. Tính sẵn dùng (Availability) <i>(Khả năng hệ thống duy trì hoạt động hoặc phục hồi nhanh khi có lỗi)</i>	Sự cố 1: Máy chủ chứa Catalog đột ngột bị sập do tràn RAM (Out of Memory).	Em đặt vấn đề không phải lỗi linh tinh thông thường mà là lỗi OOM vì nó rất khó chịu trên những container chạy Linux, đặc biệt là những người mới dùng Docker. Cần giải thích một chút là lỗi OOM trên các hệ điều hành Linux (ở đây

		chứa Docker) sẽ kích hoạt cơ chế OOM Killer bắn chết hết các tiến trình của container đó ngay lập tức (Exit code 137). Một số dev mới hay có thói quen để mặc định check trạng thái “UNHEALTHY” của container bất kỳ trong một cụm rồi mới xử lý lỗi. Nhưng đằng này nó chuyển hẳn sang trạng thái “EXITED” luôn. Nếu chỉ dùng Docker thông thường thì phải cài thêm policy để nó còn biết tự khởi động lại Container nếu exit đột ngột. Nhóm em lại không thích rườm rà như vậy nên chọn một giải pháp đơn giản mà hiệu quả hơn: Cụm Application Cluster được quản lý bởi công cụ điều phối Docker Swarm. Khi máy chủ Catalog API bị tràn RAM, rõ ràng lúc đó hệ điều hành sẽ kill sạch tiến trình, làm container bị ngắt đột ngột. Trình điều phối Swarm Manager sẽ lập tức phát hiện trạng thái thực tế bị thiếu hụt so với cấu hình (desired
--	--	--

		state: Ví dụ em khai báo lúc đầu là service này luôn phải có 3 replica chạy khỏe, nếu vắng mất 1 2 cái thì sẽ tự động khởi tạo container mới trên node bất kỳ để đảm bảo đúng yêu cầu cấu hình). Nó sẽ ra lệnh tự động spin up một container thay thế, đồng thời Load Balancer tự động loại bỏ node chết khỏi danh sách định tuyến, chuyển traffic sang các node còn lại. Quá trình tự phục hồi diễn ra tự động mà người dùng gần như không cảm nhận được gián đoạn.
	Sự cố 2: Cụm Database Master (PostgreSQL) bị hỏng ổ cứng, ngừng hoạt động hoàn toàn.	Cần nhắc lại rằng “ghi” là của một mình ông DB Master xử lý, còn luồng đọc là do bọn DB Slave đảm nhận 100%. Trong sự cố này thì luồng ghi bị ảnh hưởng do DB Master chết đột ngột. Cũng cần lưu ý luồng ghi sẽ gồm 2 kiểu dữ liệu ứng với 2 thao tác: 1. Với thao tác Ghi trực tiếp (Ví dụ: Đổi mật khẩu, Viết bình luận): Hệ thống API sẽ không kết nối được tới Master, sinh ra lỗi

		Connection Timeout. API Gateway có thể trả về cho Frontend mã lỗi 503 Service Unavailable hoặc hiển thị thông báo "Hệ thống đang bảo trì tính năng này, vui lòng thử lại sau". 2. Với thao tác Upload truyện (Nhờ Kafka gánh vác): Khi uploader đẩy ảnh lên, Media Upload Service nhận file, lưu vào MinIO và sinh ra một Event ném vào Apache Kafka. Lúc này dù Master DB có chết, tin nhắn vẫn nằm an toàn trong hàng đợi của Kafka. Khi DB được sửa xong, các AI Worker sẽ từ từ lấy tin nhắn ra và ghi vào DB. => Không mất mát dữ liệu. Do đó, nhờ áp dụng mẫu kiến trúc Master – Slave, toàn bộ các lệnh "Đọc" (người dùng xem danh sách truyện, đọc chương truyện) hiển nhiên vẫn diễn ra bình thường vì chúng được query từ Slave DB và
--	--	--

		Redis Cache. Các lệnh "Ghi" (đăng truyen, bình luận) sẽ tạm thời báo lỗi hoặc được đưa vào hàng đợi. Hệ thống không thể sống mãi mà không có Master. Quá trình khắc phục diễn ra như sau: 1. Alerting: Các hệ thống giám sát (Prometheus/Grafana) hoặc Notification Worker liên tục ping DB Master. Khi rớt kết nối, nó bắn cảnh báo khẩn cấp (qua Telegram/Email) cho Quản trị viên (DevOps/DBA). 2. Promotion: Quản trị viên can thiệp (hoặc dùng tool tự động như Patroni). Họ gỡ bỏ chế độ Read – Only của một node Slave khỏe nhất, promote nó lên làm Master mới. 3. Reconfiguration: Cập nhật lại chuỗi kết nối (Connection String / DNS) để các Microservices trở IP Master mới về cái máy vừa được thăng cấp.
--	--	---

		4. Khôi phục hoàn toàn: Tính năng "Ghi" hoạt động trở lại bình thường. Quản trị viên có thể mua ổ cứng mới, cài lại một Node mới và gán nó làm Slave bổ sung vào cụm sau.
--	--	---

Bảng 6. Xác thực yêu cầu Availability

1.5.2. Xác thực yêu cầu về Khả năng cập nhật (Modifiability)

Thuộc tính chất lượng (Quality Attribute)	Sự cố giả định (Stimulus)	Phản hồi của hệ thống (Response)
2. Khả năng cập nhật (Modifiability) <i>(Khả năng thay đổi, nâng cấp tính năng mà không làm ảnh hưởng hệ thống cũ)</i>	Sự cố 1: Đội ngũ Data Science muốn thay thế mô hình AI Swin NSFW cũ bằng một mô hình mới nặng hơn, yêu cầu thư viện Pytorch phiên bản khác.	Thông thường nếu xài kiến trúc nguyên khối, việc nâng cấp PyTorch có thể gây xung đột thư viện (dependency) với các phần khác của backend (như thư viện xử lý ảnh, database driver). Khi đó toàn bộ hệ thống buộc phải restart lại, gây gián đoạn dịch vụ Nhưng nhờ kiến trúc Pipe-Filter & Workers và việc tách rời thông qua Kafka, mấy ông DS chỉ cần tạo một AI Worker Node mới chứa mô hình mới và deploy nó. Các service khác

		như Upload Service hay Catalog Service hoàn toàn không can hệ gì và cũng không cần sửa một dòng code nào, cũng không cần restart luôn. Ở đây mình có thể hình dung đơn giản như sau: ➤ Pipe: Là Kafka, làm nhiệm vụ vận chuyển dữ liệu (sự kiện ảnh cần kiểm duyệt) từ trạm này sang trạm khác. ➤ Filter: Là các AI Worker (Swin NSFW Filter, OCR Processor). Mỗi Filter là một module độc lập, chỉ làm đúng một việc: Nhận dữ liệu đầu vào từ Pipe → Xử lý (chạy AI) → Đẩy kết quả ra một Pipe khác hoặc cập nhật vào Database. → Việc thay đổi mô hình AI thực chất chỉ là việc tháo một Filter cũ ra khỏi đường ống và lắp một Filter mới vào, hoàn toàn không làm vỡ cấu trúc của toàn bộ quy trình kiến trúc được setup lúc đầu.
	Sự cố 2: Nền tảng muốn tích hợp thêm cổng thanh toán	Thông thường đối với kiến trúc nguyên khối cũ, nếu muốn cập nhật

	(Momo/ZaloPay) để độc giả mua chương truyện Premium.	một chức năng mới cho một hệ thống đã hoàn thiện, cần sửa source code thêm mới module “Payment” vào, và sửa các bảng trong DB. Nhưng nhờ kiến trúc Microservices, hệ thống của nhóm em cho phép tạo một Payment Service hoàn toàn mới, tích hợp các cổng thanh toán và độc lập với Database riêng. Quy trình triển khai thì y hệt như những service khác: Đóng gói Payment Service thành một Docker Image và ra lệnh cho Docker Swarm chạy nó lên thành các Container mới (ví dụ: chạy 2 replicas). Lúc này, Payment Service đã chạy ngầm trong mạng nội bộ, nhưng người dùng bên ngoài chưa hề biết đến sự tồn tại của nó. Các service đọc truyện cũ vẫn hoạt động bình thường, không hề bị ảnh hưởng. Quản trị viên lúc này chỉ cần thêm vài dòng cấu hình để định tuyến API Gateway cho phép người dùng trở tới Payment Service vừa tạo. App/Web Frontend của người dùng
--	---	--

		đã có thể gọi API thanh toán. Cuối cùng là cấu hình các giao tiếp nội bộ với các service khác. Giả sử: Khi Momo gọi Webhook báo thanh toán thành công về Payment Service. Payment Service ghi nhận vào DB của nó, sau đó ném một sự kiện <code>PREMIUM_UNLOCKED {user_id, chapter_id}</code> vào Kafka Message Broker. Catalog Service (đang lắng nghe Kafka) nhận được tin nhắn này, cập nhật trạng thái quyền đọc của user đối với chapter đó trong Database Slave/Redis của riêng nó. Sự liên kết lỏng lẻo này giúp 2 service không gọi trực tiếp lẫn nhau, tránh việc một bên sập kéo theo bên kia sập. Toàn bộ các quy trình trên có thể được thực hiện ngay trong lúc các service khác đang chạy mà không lo gián đoạn dịch vụ hay phải bảo trì hệ thống.
--	--	--

Bảng 7. Xác thực yêu cầu Modifiability

1.5.3. Xác thực yêu cầu về Tính bảo mật (Security)

Thuộc tính chất lượng (Quality Attribute)	Sự cố giả định (Stimulus)	Phản hồi của hệ thống (Response)
3. Tính bảo mật (Security) <i>(Khả năng ngăn chặn, chống lại các cuộc tấn công và truy cập trái phép)</i>	Sự cố 1: Hacker sử dụng tool tự động gửi 10,000 request/giây vào endpoint Đăng nhập (/api/v1/auth) nhằm dò rỉ mật khẩu (Brute-force).	Nhóm em đã tính toán cho hai cơ chế bảo mật dưới đây: Cơ chế Rate Limiting: API Gateway được cấu hình để đếm số lượng request đến từ một địa chỉ IP (hoặc một dải IP) trong một khung thời gian. Cấu hình chỉ cho phép tối đa 5 request đăng nhập / 1 phút / 1 IP. Khi tool của hacker gửi đến request thứ 6, API Gateway sẽ lập tức chặn đứng request này lại. Nó không thêm forward request đó vào mạng nội bộ (nơi chứa các Microservices), mà lập tức trả thẳng về cho hacker mã lỗi HTTP 429 Too Many Requests hoặc 403 Forbidden. Kết quả, Identity Service và cơ sở dữ liệu PostgreSQL ở bên trong hoàn toàn bình yên vô sự. CPU và RAM của hệ thống không bị lãng phí để xử lý 10,000 cái request rác kia, nhờ đó hệ thống không bị đánh sập (chống được luôn cả DDoS tầng Application). Mặc khác, đối với các hacker chuyên

		ngiệp có thể dùng Mạng Botnet hoặc Proxy đổi IP liên tục để qua mặt API Gateway. Giả sử kịch bản tồi tệ nhất xảy ra: Hacker dò được một số mật khẩu yếu, hoặc hệ thống bị dính lỗi SQL Injection khiến toàn bộ Database bị tải trộm ra ngoài. Lúc này, lớp phòng thủ thứ hai tại Identity Service sẽ phát huy tác dụng: Hệ thống KHÔNG BAO GIỜ lưu mật khẩu dưới dạng văn bản thô (Plain-text) như 123456 hay password. Trước khi lưu xuống Database, Identity Service đã sử dụng thuật toán băm (Hashing) một chiều cường độ cao (Bcrypt và Argon2) kết hợp với Salt (chuỗi ngẫu nhiên sinh ra cho từng user). Mật khẩu 123456 sẽ biến thành một chuỗi vô nghĩa như \$2b\$12\$eImiTXuWVxfM37uY4JANj... Về lý thuyết, thuật toán Hash được thiết kế bằng toán học sao cho: Từ mật khẩu suy ra chuỗi Hash thì rất dễ, nhưng từ chuỗi Hash không thể nào dịch ngược lại ra mật khẩu gốc. Kết quả, Hacker cầm trong tay toàn bộ Database cũng đành bó tay chịu chết. Chúng không thể
--	--	--

		biết mật khẩu thật của độc giả là gì để đi đăng nhập trái phép vào hệ thống, bảo vệ an toàn tuyệt đối cho tài sản số và thông tin cá nhân của người dùng.
	Sự cố 2: Kẻ tấn công cố gắng đánh sập máy chủ bằng cách liên tục request tải các hình ảnh dung lượng cao của truyện tranh.	Cần lưu ý rằng, hệ thống của nhóm tách biệt độc lập backend với nơi lưu trữ ảnh. Do đó, hệ thống Backend sẽ không bị sập. Theo tiến trình đọc truyện (đã vẽ trước đó), hệ thống chỉ trả về Presigned URLs. Request tải ảnh thực tế được đẩy thẳng sang MinIO Object Storage Cluster. Khi hacker (hoặc độc giả) request xem một chương truyện (gọi vào Catalog API), Backend FastAPI tuyệt đối không đi tìm file ảnh để trả về. Thay vào đó, Backend chỉ query Database để lấy danh sách tên file, sau đó gọi thư viện mã hóa sinh ra các Presigned URLs (Đường dẫn có chữ ký giới hạn thời gian). Chuỗi JSON Backend trả về cực kỳ nhẹ (chỉ vài Kilobyte). Quá trình này tiêu tốn cực ít CPU và RAM của Backend. Ở trình duyệt của hacker, sau khi nhận được mảng URL trên, sẽ sử dụng các URL đó để tải ảnh. Đáng chú ý, các

		URL này không trở về Backend FastAPI, mà trở thẳng đến MinIO Object Storage Cluster. MinIO được thiết kế (bằng ngôn ngữ Go) chuyên biệt tốt cho việc phân phối nội dung tĩnh. Nó có khả năng mở rộng hàng chục Node và sử dụng cơ chế truyền tải Zero-copy của hệ điều hành để đẩy dữ liệu qua mạng với tốc độ tối đa của card mạng vật lý, mà không tốn CPU tính toán. Khi hacker tung hàng chục ngàn request tải ảnh, Application Cluster Hoàn toàn bình yên vô sự. CPU rảnh rỗi, RAM trống trơn. Các API đăng nhập, chat, tìm kiếm truyện vẫn hoạt động tốt. MinIO Cluster Hứng trọn lượng traffic khổng lồ này. Nếu traffic vượt quá ngưỡng của cổng mạng, những file ảnh tải về có thể bị chậm đi, nhưng bản thân phần mềm MinIO sẽ không sập.
--	--	---

Bảng 8. Xác thực yêu cầu Security

1.5.4. Xác thực yêu cầu về Khả năng mở rộng (Scalability)

Thuộc tính chất lượng (Quality)	Sự cố giả định (Stimulus)	Phản hồi của hệ thống (Response)
---------------------------------	---------------------------	----------------------------------

Attribute)		
4. Khả năng mở rộng (Scalability) <i>(Khả năng đáp ứng khi lượng truy cập hoặc dữ liệu tăng đột biến)</i>	Sự cố 1: Bộ truyện "One Piece" ra chương mới, lượng người dùng truy cập vào xem cùng lúc tăng gấp 20 lần bình thường.	Có ba lớp để scale cho vấn đề: Lớp 1: Đỡ tải Database bằng Redis Cluster (In-memory Cache) Thay vì bắt Database đọc dữ liệu từ ổ cứng, hệ thống của nhóm dùng Redis - cơ sở dữ liệu lưu trữ hoàn toàn trên RAM. Khi Uploader vừa đăng chương mới của "One Piece", Catalog Service đã chủ động nạp sẵn (Cache Warming) các thông tin metadata (Tên truyện, số trang, danh sách URL ảnh) lên RAM của Redis. Khi luồng traffic khổng lồ x20 lần ập đến, các request chỉ đi đến RAM (tốc độ phản hồi tính bằng micro-giây) và cache hit về ngay lập tức. Cụm Database ở phía sau hầu như không cảm nhận được lượng traffic khổng lồ này, duy trì sự ổn định cho các tính năng khác. Lớp 2: Gánh tải bằng thông bằng MinIO (Object Storage) Như đã giải thích ở kịch bản bảo mật, Catalog API chỉ trả về chuỗi text rất nhẹ (Presigned URLs). Trách nhiệm truyền tải hàng Terabyte hình ảnh được

		giao phó hoàn toàn cho cụm MinIO Cluster. Khác với máy chủ Backend thông thường, máy chủ MinIO được thiết kế chuyên dụng cho I/O mạng (sẵn tiện, I/O luôn là một trong những thao tác gây hao tổn performance nặng nề nhất cho bất kỳ hệ thống nào). Nếu lưu lượng quá lớn, nhóm chỉ cần gắn thêm các ổ cứng hoặc thêm các node MinIO vật lý mới vào cụm, băng thông tải ảnh sẽ được phân tán đều. Kiến trúc này cho phép scale theo chiều ngang cực kỳ thuận tiện, cho bất kỳ tình huống tải lớn nào. Lớp 3: Co giãn CPU bằng Docker Swarm (Auto-scaling) Dù Redis và MinIO đã gánh phần nặng nhất, bản thân Catalog API vẫn phải dùng CPU để phân tích (parse) các HTTP Request, kiểm tra JWT Token của người dùng, và định tuyến. Khi traffic x20 lần, CPU của container Catalog API hiện tại có thể chạm ngưỡng rất cao. Lúc này, cơ sở hạ tầng Docker Swarm sẽ nhận được cảnh báo từ hệ thống giám sát
--	--	--

		(Prometheus/Grafana). Cơ chế Horizontal Pod Autoscaling (HPA - Mở rộng theo chiều ngang) được kích hoạt. Thay vì chỉ có 2 container Catalog API chạy như bình thường, Swarm sẽ tự động "nhân bản" thành 5, 10, hoặc 20 container Catalog API nằm rải rác trên các máy chủ vật lý khác nhau trong Application Cluster. API Gateway lập tức chia đều 100,000 request đó cho 20 container mới này. Kết quả, tải CPU trên mỗi container giảm xuống mức an toàn. Toàn bộ quá trình này diễn ra tự động trong vài giây mà không cần kỹ sư hệ thống phải thao tác thủ công để cắm thêm RAM hay CPU. Quá trình thu hồi tài nguyên (Scale-in): Khả năng mở rộng thực sự tốt là phải đi kèm với Tính đàn hồi (Elasticity) - tức là có nở ra được thì phải co lại được để tiết kiệm tiền server. Sau 2-3 tiếng, khi lượng người đọc truyện đã đọc xong và out ra, traffic trở về mức bình thường. Docker Swarm nhận thấy CPU của 20 container Catalog API đang rảnh rỗi. Nó sẽ tự động ra lệnh terminate bớt các container dư thừa, đưa hệ thống về lại
--	--	--

		trạng thái 2 container như ban đầu, tối ưu chi phí.
	Sự cố 2: Các tool Crawl (ETL) đồng loạt đẩy 50,000 file ảnh chương truyện mới lên hệ thống vào lúc nửa đêm.	Để giải quyết bài toán này, Nhóm đã áp dụng mẫu kiến trúc Broker Architecture kết hợp với Background Jobs (Celery). Bước 1: Fast Intake Khi tool Crawl đẩy 50,000 ảnh lên, Media Upload Service chỉ làm đúng 2 việc rất nhẹ: Nhận file và cất tạm vào MinIO. Gửi một tin nhắn (Event/Message) có nội dung kiểu "Ê, có ảnh mới tên là ABC ở đường dẫn XYZ, cần kiểm duyệt nhé!" vào Apache Kafka. Toàn bộ quá trình này chỉ mất vài chục mili-giây. Sau đó, Service lập tức trả về phản hồi 200 OK cho tool Crawl. Kết quả, Tool Crawl đẩy xong 50,000 ảnh trong nháy mắt mà Media Upload Service không hề bị quá tải. Bước 2: Apache Kafka đóng vai trò Buffer Kafka là một hệ thống phân phối tin nhắn cực kỳ mạnh, được thiết kế để chịu tải hàng triệu tin nhắn mỗi giây. 50,000 sự kiện vừa được tạo ra sẽ nằm

		gọn gàng trong một Topic (hàng đợi) của Kafka. Kafka đóng vai trò như một hồ chứa, hứng toàn bộ tải lưu lượng này, bảo vệ các thành phần phía sau khỏi bị ngập. Bước 3: Asynchronous Processing Phía sau Kafka là các AI Worker Nodes (chạy Celery). Khác với API, các Worker này không bị ai hỏi thúc cả. Chúng đóng vai trò là "Consumer", từ từ pull từng tin nhắn từ Kafka về để xử lý (chạy mô hình AI). Nếu hệ thống chỉ có 2 AI Worker (năng lực xử lý ví dụ 10 ảnh/giây), thì 50,000 ảnh sẽ được giải quyết từ từ trong khoảng vài tiếng khá lâu, nhưng tuyệt nhiên không bao giờ sập vì lượng tải này. Vấn đề ở đây là dù có 50,000 hay 500,000 ảnh, CPU và RAM của AI Worker cũng chỉ hoạt động ở mức trần thiết kế (không bao giờ vượt quá 100%). Hệ thống không bao giờ bị nghẽn cổ chai (bottleneck) dẫn đến crash. Mặc khác, nếu muốn xử lý ảnh nhanh hơn, quản trị viên chỉ cần cấp thêm vài
--	--	--

		máy chủ AI GPU Nodes mới vào cụm Docker Swarm. Nhờ cơ chế Consumer Group của Kafka, các tin nhắn trong hàng đợi sẽ lập tức được chia đều cho cả các Worker cũ và Worker mới. Quá trình Scale-out diễn ra tự nhiên mà không cần sửa bất kỳ dòng code nào.
--	--	--

Bảng 9. Xác thực yêu cầu Scalability

1.5.5. Xác thực yêu cầu về Độ tin cậy (Reliability)

Thuộc tính chất lượng (Quality Attribute)	Sự cố giả định (Stimulus)	Phản hồi của hệ thống (Response)
5. Độ tin cậy (Reliability) (Khả năng hoạt động chính xác, đảm bảo toàn vẹn dữ liệu và không mất mát thông tin)	Sự cố 1: Trong lúc tiến trình ngầm (AI Worker) đang quét ảnh kiểm duyệt nội dung thì máy chủ AI Node đột nhiên bị cúp điện tắt phụt.	Về bản chất của Message và Broker, khi một file ảnh được tải lên, Media Upload Service tạo ra một sự kiện (Message) đưa vào hàng đợi của Kafka. Điểm quan trọng của Kafka là nó lưu trữ thông điệp trên ổ cứng (Persistent Storage), không phải chỉ trên RAM. Khi một tin nhắn đã vào Kafka, nó an toàn tuyệt đối. Các AI Worker chỉ đóng vai trò là Consumer (người lấy tin nhắn ra đọc và làm việc). Đồng thời, Kafka cung cấp cơ chế

		Acknowledgment – ACK giúp đảm bảo độ tin cậy trong quá trình giao tiếp bất đồng bộ: Bước 1: Worker nhận việc (Pulling): AI Worker A (Celery) kết nối đến Kafka và nói "Cho tôi xin một việc". Kafka gửi cho nó sự kiện "Hãy quét bức ảnh xyz.jpg". Bước 2: Kafka chuyển trạng thái (Unacknowledged): Lúc này, Kafka không xóa sự kiện đó khỏi hàng đợi. Nó chỉ đánh dấu sự kiện đó là đang được xử lý (in-flight/unacknowledged) bởi Worker A và tạm thời che nó đi không cho các Worker khác thấy. Bước 3: Worker Processing: Worker A tải ảnh từ MinIO xuống, load mô hình PyTorch Swin NSFW lên GPU và bắt đầu chạy suy luận. Lúc này sự cố xảy ra! Máy chủ AI Node chứa Worker A bị cúp điện tắt phụp. Do bị sập, Worker A không bao giờ gửi được tín hiệu ACK (Tôi đã hoàn thành) về cho Kafka. Kafka phát hiện lỗi Timeout: Mỗi sự kiện giao cho Worker đều có một khoảng thời gian
--	--	--

		chờ (Timeout, ví dụ: 60 giây). Sau 60 giây không thấy Worker A phản hồi ACK (hoặc Kafka phát hiện Worker A đã ngắt kết nối mạng - Session Timeout), Kafka sẽ kết luận: "Worker A đã chết hoặc gặp lỗi". Ngay lập tức, Kafka gỡ bỏ trạng thái đang xử lý của sự kiện đó, và đưa nó hiện thị trở lại trong hàng đợi. Worker khác (nếu đang rảnh) sẽ đứng ra tiếp quản. Giả sử một AI Worker B (đang chạy trên một máy chủ khác vẫn có điện) sẽ lập tức nhìn thấy sự kiện này. Nó kéo sự kiện về và thực hiện quét bức ảnh xyz.jpg lại từ đầu. Khi Worker B chạy xong, update Database thành công, nó gửi tín hiệu ACK về Kafka. Lúc này, Kafka mới thực sự đánh dấu hoàn thành (Commit offset) và không giao sự kiện này cho ai nữa. Toàn bộ quá trình trên trính là diễn giải cho nguyên lý "At-least-once delivery" của Message Broker như trong lý thuyết đã học. Mặc dù hệ thống có thể mất vài phút trễ nải (do phải đợi timeout và làm lại), nhưng quan trọng nhất là Tính toàn vẹn của
--	--	---

		dữ liệu được bảo vệ tuyệt đối. Không có bất kỳ một bức ảnh nào tải lên mà không qua màng lọc kiểm duyệt (Swin NSFW Filter) dù máy chủ chạy AI có bị sập cháy. Điều này đáp ứng hoàn hảo yêu cầu phi chức năng về quản lý chất lượng nội dung của nền tảng truyền tranh.
	Sự cố 2: Trong quá trình chạy tiến trình Crawl & Đồng bộ dữ liệu (ETL) từ nguồn bên thứ 3 (Mangadex), API của họ đột nhiên trả về file JSON bị lỗi cấu trúc (corrupted).	Như đã mô tả trong những mục trước đó (Tiến trình Crawl & Đồng bộ dữ liệu ETL), dữ liệu từ bên ngoài không bao giờ được phép đi thẳng vào nhà chính (Master DB). Nó bắt buộc phải đi qua một filter. Bước 1: Extract (Trích xuất) & Intake Filter Tiến trình Crawl (chạy định kỳ) gọi API Mangadex để lấy hàng ngàn object JSON. Dữ liệu này đi qua màng lọc đầu tiên là Intake Filter. Nó không lưu vào Master DB, mà lưu toàn bộ vào một vùng đệm gọi là Staging DB (Cơ sở dữ liệu tạm). Ví dụ: Nó tạo ra một lô (batch) dữ liệu mang mã số BATCH_1024 trong Staging DB. Bước 2: Transform (Biến đổi & Xác

		thực) Tiếp theo, dữ liệu trong BATCH_1024 đi qua các màng lọc kiểm tra tính hợp lệ (Validation Filters) và màng lọc chuẩn hóa (Translator Filter - như mục trước đã đề cập). Tại đây, màng lọc phát hiện ra JSON bị lỗi cấu trúc (corrupted) – ví dụ UUID bị sai định dạng, hoặc thiếu thông tin chương truyện. Màng lọc lập tức ném ra một Exception và đánh dấu lô BATCH_1024 này là FAILED. Nhờ cơ chế Automated Rollback qua Kafka, khi lô dữ liệu bị đánh dấu FAILED, hệ thống không để rác nằm im đó, mà kích hoạt cơ chế tự phục hồi: Tiến trình ETL lập tức bắn một sự kiện <code>CRAWL_BATCH_FAILED</code> {batch_id: "1024"} vào Apache Kafka. Một con Worker chuyên dụng chạy ngầm mang tên Mangadex Rollback Worker (được nhóm thiết kế riêng ở Bài 2) đang lắng nghe trên Kafka sẽ chụp lấy sự kiện này. Worker này chạy các lệnh SQL để Rollback (Xóa sạch) toàn bộ các dòng dữ liệu, ID truyện, ID chương thuộc về
--	--	--

		BATCH_1024 khỏi Staging DB. Cuối cùng, quản trị viên được gửi một thông báo (qua Telegram/Email) về việc Mangadex API đang bị lỗi để theo dõi hoặc sửa lại code tool crawl.
--	--	---

Bảng 10. Xác thực yêu cầu Reliability

1.5.6. Xác thực yêu cầu về Hiệu suất (Performance)

Để minh chứng cho tính khả thi và khả năng đáp ứng yêu cầu hiệu năng cao của các mẫu kiến trúc đã thiết kế (bao gồm Kiến trúc Microservices, Event-Driven và cơ chế Caching), nhóm đã tiến hành xây dựng 3 nguyên mẫu giả lập (Prototype).

Môi trường thực nghiệm được đóng gói và cô lập hoàn toàn bằng công nghệ Containerization (Docker), đảm bảo tính khách quan. Công cụ k6 được sử dụng để giả lập tải trọng lớn (Load Testing) với kịch bản truy cập đồng thời (Concurrent Users - VUs).¹

✓	proto1_minio	-	-	-	0.15%	50 minutes ago
	proto-minio-1	c37618a6bcab	minio/minio	9011.9000 ↗	0.03%	50 minutes ago
	proto-api-1	93fa7b8c6adf	python.3.10-slim	8011.8000 ↗	0.12%	50 minutes ago
✓	proto2_redis	-	-	-	0.24%	48 minutes ago
	proto-redis-1	b64bb926ec47	redis.alpine		0.12%	48 minutes ago
	proto-api-1	ee4ea875f1b6	python.3.10-slim	8012.8000 ↗	0.12%	48 minutes ago
✓	proto3_celery	-	-	-	0.4%	46 minutes ago
	proto-redis-broker-1	184a78986a6d	redis.alpine		0.14%	46 minutes ago
	proto-worker-1	897f98257b21	python.3.10-slim		0.14%	46 minutes ago
	proto-api-1	911a8101b814	python.3.10-slim	8013.8000 ↗	0.12%	46 minutes ago

Hình 19. Danh sách các container của prototype và port sử dụng

Thực nghiệm 1: Xác thực kiến trúc tối ưu luồng phân phối tệp tĩnh (Media Offloading với MinIO)

¹ Toàn bộ source code prototype đã được đính kèm chung với sản phẩm demo trên github. Nên phần trình bày này sẽ không nhắc lại code.

Mục đích thực nghiệm: Kiểm chứng hiệu năng của giải pháp "Ủy quyền truy cập" (Presigned URL) trong việc phân phối hình ảnh truyền tranh. Giải pháp này nhằm chứng minh việc tách rời luồng I/O đọc tệp tĩnh ra khỏi API Backend sẽ giúp loại bỏ nút thắt cổ chai (bottleneck) về băng thông và RAM trên máy chủ ứng dụng.

Kịch bản kiểm thử:

- Tham số tải: Giả lập 50 người dùng đồng thời (50 VUs) liên tục yêu cầu tải một tệp hình ảnh có dung lượng 5MB trong thời gian 10 giây.
- Trường hợp 1 (Luồng xử lý đồng bộ - Anti-pattern): API trực tiếp đọc tệp từ hệ thống lưu trữ, nạp vào RAM và truyền luồng (stream) trả về cho Client.
- Trường hợp 2 (Luồng xử lý ủy quyền - Kiến trúc đề xuất): API chỉ đóng vai trò xác thực và sinh ra một chuỗi Presigned URL (chứa mã thông báo giới hạn thời gian). Client sử dụng URL này để tải trực tiếp từ hệ thống Object Storage (MinIO).

Kết quả và Đánh giá Hệ thống k6 ghi nhận số liệu cụ thể như sau:

```
TOTAL RESULTS
checks_total.....: 303      28.291865/s
checks_succeeded...: 100.00% 303 out of 303
checks_failed.....: 0.00%   0 out of 303

✓ status was 200

HTTP
http_req_duration.....: avg=1.73s min=759.64ms med=1.68s max=2.5s p(90)=2.25s p(95)=2.31s
{ expected_response:true }...: avg=1.73s min=759.64ms med=1.68s max=2.5s p(90)=2.25s p(95)=2.31s
http_req_failed.....: 0.00%   0 out of 303
http_reqs.....: 303      28.291865/s

EXECUTION
iteration_duration.....: avg=1.74s min=759.64ms med=1.68s max=2.5s p(90)=2.25s p(95)=2.31s
iterations.....: 303      28.291865/s
vus.....: 50      min=50      max=50
vus_max.....: 50      min=50      max=50

NETWORK
data_received.....: 1.6 GB 148 MB/s
data_sent.....: 27 kB 2.5 kB/s
```

Hình 20. Kết quả K6 test prototype 1 trường hợp 1

```
TOTAL RESULTS
checks_total.....: 18630 1859.283608/s
checks_succeeded...: 100.00% 18630 out of 18630
checks_failed.....: 0.00% 0 out of 18630

✓ status was 200

HTTP
http_req_duration.....: avg=26.79ms min=16.6ms med=25.82ms max=55.71ms p(90)=33.51ms p(95)=36.33ms
{ expected_response:true }...: avg=26.79ms min=16.6ms med=25.82ms max=55.71ms p(90)=33.51ms p(95)=36.33ms
http_req_failed.....: 0.00% 0 out of 18630
http_reqs.....: 18630 1859.283608/s

EXECUTION
iteration_duration.....: avg=26.85ms min=16.6ms med=25.88ms max=56.24ms p(90)=33.56ms p(95)=36.41ms
iterations.....: 18630 1859.283608/s
vus.....: 50 min=50 max=50
vus_max.....: 50 min=50 max=50

NETWORK
data_received.....: 8.4 MB 839 kB/s
data_sent.....: 1.7 MB 171 kB/s
```

Hình 21. Kết quả K6 test prototype 1 trường hợp 2

Trường hợp test	Tổng số request đáp ứng	Độ trễ trung bình
Luồng đồng bộ	303 requests (Thông lượng 28.29 req/s)	1.73 giây (Có thời điểm lên tới 2.5 giây)
Sử dụng Presigned URL	18,630 requests (Thông lượng 1,859.28 req/s)	26.79 mili-giây

Bảng 11. Kết quả tổng hợp số liệu sau khi chạy K6 test prototype 1

Từ bảng kết quả trên cho thấy: Độ trễ trung bình khi sử dụng Presigned URL Giảm hơn 64 lần so với phương pháp cũ (một con số tối ưu đáng kinh ngạc). Kết quả thực nghiệm đã chứng minh rõ rệt tính đúng đắn của thiết kế. Việc giao phó toàn bộ tác vụ phân phối tệp tĩnh cho MinIO thông qua cơ chế Presigned URL giúp Server API gần như không chịu tải về I/O mạng, qua đó tăng thông lượng hệ thống lên xấp xỉ 65 lần.

Thực nghiệm 2: Xác thực kiến trúc bộ nhớ đệm (Caching Architecture với Redis)

Mục đích thực nghiệm: Kiểm chứng khả năng bảo vệ Cơ sở dữ liệu chính (Database) khỏi tình trạng quá tải khi xảy ra hiện tượng "Bão Traffic" (ví dụ: một chương

truyện HOT vừa ra mắt thu hút lượng lớn độc giả truy cập cùng lúc), thông qua cơ chế Cache Warming và In-memory Caching.

Kịch bản kiểm thử:

- Tham số tải: Giả lập 100 người dùng đồng thời (100 VUs) liên tục truy vấn thông tin chương truyện trong thời gian 10 giây. Truy vấn gốc vào cơ sở dữ liệu được giả lập có độ trễ I/O là 0.5 giây.
- Trường hợp 1 (Không sử dụng Cache): Mọi request đều truy vấn trực tiếp xuống Database.
- Trường hợp 2 (Sử dụng Redis Cache): Áp dụng chiến lược Cache-Aside. API kiểm tra dữ liệu trên RAM (Redis) trước, nếu Cache Hit sẽ trả về ngay lập tức.

Kết quả và Đánh giá thu được số liệu như sau:

```
TOTAL RESULTS

checks_total.....: 1900      180.451504/s
checks_succeeded...: 100.00% 1900 out of 1900
checks_failed.....: 0.00%   0 out of 1900

✓ status was 200

HTTP
http_req_duration.....: avg=552.27ms min=545.8ms med=548.62ms max=601.43ms p(90)=562.38ms p(95)=575.08ms
{ expected_response:true }...: avg=552.27ms min=545.8ms med=548.62ms max=601.43ms p(90)=562.38ms p(95)=575.08ms
http_req_failed.....: 0.00%   0 out of 1900
http_reqs.....: 1900      180.451504/s

EXECUTION
iteration_duration.....: avg=552.4ms min=545.8ms med=548.67ms max=602.96ms p(90)=562.78ms p(95)=575.23ms
iterations.....: 1900      180.451504/s
vus.....: 100      min=100      max=100
vus_max.....: 100      min=100      max=100

NETWORK
data_received.....: 7.9 MB 748 kB/s
data_sent.....: 163 kB 16 kB/s
```

Hình 22. Kết quả K6 test prototype 2 trường hợp 1

```

TOTAL RESULTS

checks_total.....: 15231  1514.231976/s
checks_succeeded...: 100.00% 15231 out of 15231
checks_failed.....: 0.00%  0 out of 15231

✓ status was 200

HTTP
http_req_duration.....: avg=65.76ms min=2.07ms med=59.81ms max=681.3ms p(90)=74.79ms p(95)=79.65ms
{ expected_response:true }...: avg=65.76ms min=2.07ms med=59.81ms max=681.3ms p(90)=74.79ms p(95)=79.65ms
http_req_failed.....: 0.00% 0 out of 15231
http_reqs.....: 15231  1514.231976/s

EXECUTION
iteration_duration.....: avg=65.83ms min=2.07ms med=59.85ms max=683.56ms p(90)=74.83ms p(95)=79.74ms
iterations.....: 15231  1514.231976/s
vus.....: 100 min=100 max=100
vus_max.....: 100 min=100 max=100

NETWORK
data_received.....: 63 MB  6.3 MB/s
data_sent.....: 1.3 MB 133 kB/s

```

Hình 23. Kết quả K6 test prototype 2 trường hợp 2

Trường hợp test	Tổng số request xử lý	Độ trễ trung bình
Không Cache	1,900 requests (Thông lượng 180.45 req/s)	552.27 mili-giây
Có Redis Cache	15,231 requests (Thông lượng 1,514.23 req/s)	65.76 mili-giây

Bảng 12. Kết quả tổng hợp số liệu sau khi chạy K6 test prototype 2

Kết quả thực nghiệm xác nhận Kiến trúc Caching (đã được mô tả trong các thiết kế trước đó) đã hoạt động chính xác. Bằng cách lưu trữ dữ liệu tần suất đọc cao vào bộ nhớ trong (In-memory), hệ thống đã cắt giảm triệt để chi phí tính toán của Database, tối ưu hóa độ trễ phản hồi nhanh hơn 8.4 lần và bảo vệ tính sẵn sàng của hệ thống dưới tải nặng.

Thực nghiệm 3: Xác thực Kiến trúc Bất đồng bộ định hướng Sự kiện (Event-Driven & Message Broker)

Mục đích thực nghiệm: Minh chứng tính hiệu quả của mẫu kiến trúc Hướng sự kiện (sử dụng Message Broker) trong việc xử lý các tác vụ tiêu tốn nhiều tài nguyên (Heavy-computation tasks) như: Trích xuất nội dung văn bản (OCR), hoặc AI kiểm duyệt hình ảnh tải lên. Mục tiêu là triệt tiêu hiện tượng nghẽn luồng (Blocking) trên Main Thread của API.

Kịch bản kiểm thử:

- Cơ chế tác vụ ngầm: Hệ thống tích hợp một Worker Pool độc lập (giả lập mất 5 giây để xử lý xong luồng AI cho 1 hình ảnh).
- Tham số tải: Giả lập 10 VUs liên tục gửi request Upload lên hệ thống trong 10 giây.
- Trường hợp 1 (Xử lý đồng bộ - Sync): API đứng chờ tiến trình AI phân tích xong bản ảnh rồi mới trả về kết quả (HTTP 200).
- Trường hợp 2 (Xử lý bất đồng bộ - Async): Kiến trúc phân tán theo mô hình Publisher-Subscriber. API (Publisher) tiếp nhận ảnh, đóng gói thành bản tin (Event) đẩy vào hàng đợi Message Broker (Redis/Celery), sau đó phản hồi ngay lập tức cho Client. Cụm Worker (Subscriber) sẽ tự động Pull bản tin về xử lý ngầm (Background Task).

```

TOTAL RESULTS

checks_total.....: 20      1.829279/s
checks_succeeded...: 100.00% 20 out of 20
checks_failed.....: 0.00%  0 out of 20

✓ status was 200

HTTP
http_req_duration.....: avg=5.46s min=5.45s med=5.46s max=5.46s p(90)=5.46s p(95)=5.46s
{ expected_response:true }...: avg=5.46s min=5.45s med=5.46s max=5.46s p(90)=5.46s p(95)=5.46s
http_req_failed.....: 0.00%  0 out of 20
http_reqs.....: 20      1.829279/s

EXECUTION
iteration_duration.....: avg=5.46s min=5.45s med=5.46s max=5.47s p(90)=5.47s p(95)=5.47s
iterations.....: 20      1.829279/s
vus.....: 10      min=10      max=10
vus_max.....: 10      min=10      max=10

NETWORK
data_received.....: 3.5 kB 324 B/s
data_sent.....: 2.0 kB 185 B/s

```

Hình 24. Kết quả K6 test prototype 3 trường hợp 1

```

TOTAL RESULTS

checks_total.....: 8154      815.097884/s
checks_succeeded...: 100.00% 8154 out of 8154
checks_failed.....: 0.00%  0 out of 8154

✓ status was 200

HTTP
http_req_duration.....: avg=12.18ms min=3.75ms med=11.57ms max=139.82ms p(90)=14.64ms p(95)=16.16ms
{ expected_response:true }...: avg=12.18ms min=3.75ms med=11.57ms max=139.82ms p(90)=14.64ms p(95)=16.16ms
http_req_failed.....: 0.00%  0 out of 8154
http_reqs.....: 8154      815.097884/s

EXECUTION
iteration_duration.....: avg=12.25ms min=3.75ms med=11.62ms max=145.58ms p(90)=14.69ms p(95)=16.18ms
iterations.....: 8154      815.097884/s
vus.....: 10      min=10      max=10
vus_max.....: 10      min=10      max=10

NETWORK
data_received.....: 1.5 MB 147 kB/s
data_sent.....: 832 kB 83 kB/s

```

Hình 25. Kết quả K6 test prototype 3 trường hợp 2

Kết quả và Đánh giá Phân tích trải nghiệm phía người dùng (Client-side via k6):

- Trường hợp 1 (Sync): Thông lượng chỉ đạt 1.8 req/s, người dùng bị treo kết nối và phải chờ trung bình 5.46 giây mới nhận được phản hồi. Cả hệ thống chỉ xử lý được 20 tấm ảnh trong 10 giây.
- Trường hợp 2 (Async): Tốc độ phản hồi đạt mức tối đa, độ trễ trung bình chỉ vỏn vẹn 12.18 mili-giây. Hệ thống nhanh chóng ghi nhận 8,154 requests thành công, mang lại trải nghiệm mượt mà, không bị gián đoạn (Non-blocking UI).

Mặt khác, Phân tích giám sát hệ thống lỗi (Backend Worker Logs): Trích xuất logs từ cụm Worker độc lập cho thấy Hệ thống không hề bỏ qua việc kiểm duyệt ảnh. Trình quản lý tiến trình đã tự động điều phối cơ chế Đa luồng (Concurrency pool = 20 workers: ForkPoolWorker-1 đến ForkPoolWorker-20). Các worker này lặng lẽ hoạt động ở Background, gánh vác khối lượng tính toán nặng (mỗi task mất succeeded in 5.00s).

Tổng kết lại, prototype này là bằng chứng rõ ràng cho sức mạnh của Kiến trúc Tiến trình Upload & Xử lý ngầm mà nhóm em đề xuất. Bằng việc phân tách ranh giới giữa Service tiếp nhận (FastAPI) và Service xử lý (Celery Worker) thông qua Message Queue, hệ thống đạt được khả năng mở rộng đàn hồi (Elastic Scalability), chịu lỗi cao (Fault-tolerant) và đảm bảo năng lực đáp ứng theo thời gian thực (Real-time response) ở mặt tiền giao diện. Đây cũng chính là những yêu cầu kiến trúc cốt lõi mà bất kỳ hệ thống nào cũng cần nhắm đến.

Cuối cùng, sau khi đã test xong tất cả prototype, em dùng lệnh ‘*docker compose down -v*’ trong từng folder project prototype tương ứng để dọn sạch container, network ảo và data volume mà thư mục đó vừa sinh ra.

CHƯƠNG 2: THIẾT KẾ SẢN PHẨM CỦA ĐỀ TÀI

2.1. Thiết kế cơ sở dữ liệu ²

Table: User				
Field Name	Data Type	Null	Key	Description
UserId	UNIQUEIDENTIFIER	No	PK	ID duy nhất của người dùng (tự động UUID).
Username	NVARCHAR(100)	No		Tên đăng nhập, duy nhất.
Email	NVARCHAR(255)	No		Email đăng nhập, duy nhất.
PasswordHash	NVARCHAR(255)	No		Mật khẩu băm (hash).
Avatar	TEXT	Yes		URL hoặc đường dẫn avatar.
Role	NVARCHAR(20)	No		Vai trò: Guest/User/Admin.
IsLocked	BIT	Yes		Trạng thái khóa tài khoản (0/1).
CreatedAt	DATETIME2	Yes		Thời gian tạo, mặc định UTC now.

Bảng 13. Bảng User

Table: Comment				
Field Name	Data Type	Null	Key	Description
CommentId	UNIQUEIDENTIFIER	No	PK	Khóa chính bình luận (UUID).
UserId	UNIQUEIDENTIFIER	No	FK	Tham chiếu đến User.UserId.

² Cách đặt tên các thuộc tính trong db được lấy cảm hứng từ tài liệu của Mangadex API

MangaId	UNIQUEIDENTIFIER	No	FK	Tham chiếu đến Manga.MangaId.
ChapterId	UNIQUEIDENTIFIER	Yes	FK	Tham chiếu đến Chapter.ChapterId (bình luận chung manga nếu null).
Content	TEXT	No		Nội dung bình luận.
CreatedAt	DATETIME2	Yes		Thời gian tạo, mặc định UTC now.
UpdatedAt	DATETIME2	Yes		Thời gian cập nhật, mặc định UTC now.
IsDeleted	BIT	Yes		Trạng thái soft-delete (0/1), mặc định 0.
IsSpoiler	BIT	Yes		Có chứa nội dung spoiler (0/1), mặc định 0.
LikeCount	INT	Yes		Số lượng like, mặc định 0.
DislikeCount	INT	Yes		Số lượng dislike, mặc định 0.

Bảng 14. Bảng Comment

Table: CommentReaction				
Field Name	Data Type	Null	Key	Description
ReactionId	UNIQUEIDENTIFIER	No	PK	Khóa chính phản ứng (UUID).
CommentId	UNIQUEIDENTIFIER	No	FK	Tham chiếu đến Comment.CommentId.
UserId	UNIQUEIDENTIFIER	No	FK	Tham chiếu đến User.UserId.

Type	NVARCHAR(10)	No		Loại phản ứng: 'like' hoặc 'dislike'.
CreatedAt	DATETIME2	Yes		Thời gian tạo, mặc định UTC now.

Bảng 15. Bảng CommentReaction

Table: Rating				
Field Name	Data Type	Null	Key	Description
RatingId	UNIQUEIDENTIFIER	No	PK	Khóa chính đánh giá (UUID).
UserId	UNIQUEIDENTIFIER	No	FK	Tham chiếu đến User.UserId.
MangaId	UNIQUEIDENTIFIER	No	FK	Tham chiếu đến Manga.MangaId.
Score	INT	Yes		Điểm đánh giá của người dùng (1-10).

Bảng 16. Bảng Rating

Table: ReadingHistory				
Field Name	Data Type	Null	Key	Description
HistoryId	UNIQUEIDENTIFIER	No	PK	Khóa chính lịch sử (UUID).
UserId	UNIQUEIDENTIFIER	No	FK	Tham chiếu đến User.UserId.
MangaId	UNIQUEIDENTIFIER	No	FK	Tham chiếu đến Manga.MangaId.
ChapterId	UNIQUEIDENTIFIER	No	FK	Tham chiếu đến Chapter.ChapterId.
LastPageRead	INT	Yes		Vị trí trang đọc cuối cùng.
ReadAt	DATETIME2	Yes		Thời gian đọc gần nhất.

Bảng 17. Bảng ReadingHistory

Table: Report				
Field Name	Data Type	Null	Key	Description
ReportId	UNIQUEIDENTIFIER	No	PK	Khóa chính báo cáo (UUID).
UserId	UNIQUEIDENTIFIER	No	FK	Tham chiếu đến User.UserId (người báo cáo).
CommentId	UNIQUEIDENTIFIER	No	FK	Tham chiếu đến Comment.CommentId bị báo cáo.
Reason	TEXT	Yes		Lý do báo cáo vi phạm.
Status	NVARCHAR(50)	Yes		Trạng thái xử lý: Pending/Resolved/Rejected.
CreatedAt	DATETIME2	Yes		Thời gian tạo báo cáo.

Bảng 18. Bảng Report

Table: Manga				
Field Name	Data Type	Null	Key	Description
MangaId	UNIQUEIDENTIFIER	No	PK	Khóa chính thông tin truyện (UUID).
Type	NVARCHAR(50)	Yes		Loại truyện: manga/novel/one-shot.
TitleEn	NVARCHAR(500)	Yes		Tiêu đề truyện tiếng Anh.

ChapterNumbers- ResetOnNewVolume ³	BIT	Yes		Cờ reset số chapter theo volume (0/1).
ContentRating	NVARCHAR(50)	Yes		Phân loại độ tuổi: safe/suggestive/erotica/pornographic.
CreatedAt	DATETIME2	Yes		Thời gian tạo trên hệ thống.
UpdatedAt	DATETIME2	Yes		Thời gian cập nhật gần nhất.
IsLocked	BIT	Yes		Trạng thái khóa manga (0/1).
LastChapter	NVARCHAR(50)	Yes		Số chapter cuối cùng.
LastVolume	NVARCHAR(50)	Yes		Số volume cuối cùng.
LatestUploadedChapter	NVARCHAR(50)	Yes		ID Chapter upload gần nhất.
OriginalLanguage	NVARCHAR(10)	Yes		Mã ngôn ngữ gốc (ja/en/ko...).
PublicationDemographic	NVARCHAR(50)	Yes		Đối tượng độc giả: shounen/seinen/shoujo...
State	NVARCHAR(50)	Yes		Trạng thái hệ thống (draft/published).
Status	NVARCHAR(50)	Yes		Tình trạng phát hành: ongoing/completed.
Year	INT	Yes		Năm phát hành ban đầu.
OfficialLinks	TEXT	Yes		Chuỗi JSON chứa các links chính thức.

Bảng 19. Bảng Manga

³ Chỗ này tên bị dài quá nên phải ngắt

Table: MangaAltTitle				
Field Name	Data Type	Null	Key	Description
AltTitleId	INT	No	PK	Khóa chính tự động tăng.
MangaId	UNIQUEIDENTIFIER	No	FK	Tham chiếu đến Manga.MangaId.
LangCode	NVARCHAR(10)	Yes		Mã ngôn ngữ của tiêu đề (en/vi/ja...).
AltTitle	TEXT	Yes		Tiêu đề thay thế theo ngôn ngữ.

Bảng 20. Bảng MangaAltTitle

Table: MangaAvailableLanguage				
Field Name	Data Type	Null	Key	Description
LangId	INT	No	PK	Khóa chính tự động tăng.
MangaId	UNIQUEIDENTIFIER	No	FK	Tham chiếu đến Manga.MangaId.
LangCode	NVARCHAR(10)	Yes		Mã ngôn ngữ khả dụng cho truyện (vi/en).

Bảng 21. Bảng MangaAvailableLanguage

Table: MangaDescription				
Field Name	Data Type	Null	Key	Description
DescriptionId	INT	No	PK	Khóa chính tự động tăng.

MangaId	UNIQUEIDENTIFIER	No	FK	Tham chiếu đến Manga.MangaId.
LangCode	NVARCHAR(10)	Yes		Mã ngôn ngữ của đoạn mô tả.
Description	TEXT	Yes		Nội dung mô tả (tóm tắt) truyện.

Bảng 22. Bảng MangaDescription

Table: MangaLink				
Field Name	Data Type	Null	Key	Description
LinkId	INT	No	PK	Khóa chính tự động tăng.
MangaId	UNIQUEIDENTIFIER	No	FK	Tham chiếu đến Manga.MangaId.
Provider	NVARCHAR(20)	Yes		Tên nhà cung cấp link (AniList/MAL/Kitsu).
Url	TEXT	Yes		URL liên kết đến trang của provider.

Bảng 23. Bảng MangaLink

Table: MangaRelated				
Field Name	Data Type	Null	Key	Description
MangaId	UNIQUEIDENTIFIER	No	PK, FK	Tham chiếu đến Manga.MangaId gốc (Khóa chính tổ hợp).
RelatedId	UNIQUEIDENTIFIER	No	PK	ID của manga liên quan (Khóa chính tổ hợp).

Type	NVARCHAR(50)	No	PK	Loại quan hệ: manga/author/cover_art (Khóa chính tổ hợp).
Related	NVARCHAR(50)	Yes		Sub-type quan hệ: side_story/alternate/sequel.
FetchAt	DATETIME2	Yes		Thời gian đồng bộ dữ liệu.

Bảng 24. Bảng MangaRelated

Table: MangaStatistics				
Field Name	Data Type	Null	Key	Description
StatisticId	UNIQUEIDENTIFIER	No	PK	Khóa chính thống kê (UUID).
MangaId	UNIQUEIDENTIFIER	No	FK	Tham chiếu đến Manga.MangaId.
Source	NVARCHAR(50)	Yes		Nguồn dữ liệu thống kê.
Follows	INT	Yes		Tổng số lượt theo dõi truyện.
AverageRating	FLOAT	Yes		Rating trung bình.
BayesianRating	FLOAT	Yes		Rating tính theo trọng số Bayesian.
UnavailableChapters	INT	Yes		Số lượng chapter đang bị ẩn.
FetchAt	DATETIME2	Yes		Thời gian fetch dữ liệu

				thống kê.
--	--	--	--	-----------

Bảng 25. Bảng MangaStatistics

Table: Tag				
Field Name	Data Type	Null	Key	Description
TagId	UNIQUEIDENTIFIER	No	PK	Khóa chính của thể loại (UUID).
GroupName	NVARCHAR(100)	Yes		Nhóm phân loại của tag (genre/theme/format).
NameEn	NVARCHAR(200)	Yes		Tên thể loại bằng tiếng Anh.

Bảng 26. Bảng Tag

Table: MangaTag				
Field Name	Data Type	Null	Key	Description
MangaId	UNIQUEIDENTIFIER	No	PK, FK	Tham chiếu đến Manga.MangaId.
TagId	UNIQUEIDENTIFIER	No	PK, FK	Tham chiếu đến Tag.TagId.

Bảng 27. Bảng MangaTag

Table: Chapter				
Field Name	Data Type	Null	Key	Description
ChapterId	UNIQUEIDENTIFIER	No	PK	Khóa chính chương truyện

				(UUID).
MangaId	UNIQUEIDENTIFIER	No	FK	Tham chiếu đến Manga.MangaId.
Type	NVARCHAR(50)	Yes		Loại nội dung: regular/extra.
Volume	NVARCHAR(50)	Yes		Tên/số của Volume chứa chương.
ChapterNumber	NVARCHAR(50)	Yes		Số thứ tự của chương.
Title	TEXT	Yes		Tiêu đề của chương truyện.
TranslatedLang	NVARCHAR(10)	Yes		Ngôn ngữ dịch thuật của bản dịch (vi/en).
Pages	INT	Yes		Số lượng trang trong chương.
PublishAt	DATETIME2	Yes		Thời điểm phát hành bản dịch.
ReadableAt	DATETIME2	Yes		Thời điểm bắt đầu cho phép đọc.
IsUnavailable	BIT	Yes		Cờ đánh dấu chapter không khả dụng (0/1).
CreatedAt	DATETIME2	Yes		Thời gian tạo trên hệ thống.
UpdatedAt	DATETIME2	Yes		Thời gian cập nhật gần nhất.

Bảng 28. Bảng Chapter

Table: Creator				
Field Name	Data Type	Null	Key	Description
CreatorId	UNIQUEIDENTIFIER	No	PK	Khóa chính tác giả/họa sĩ

				(UUID).
Type	NVARCHAR(50)	Yes		Vai trò: author/artist.
Name	NVARCHAR(500)	Yes		Tên tác giả hoặc họa sĩ.
ImageUrl	TEXT	Yes		URL ảnh đại diện của tác giả.
BiographyEn	TEXT	Yes		Tiểu sử tiếng Anh.
BiographyJa	TEXT	Yes		Tiểu sử tiếng Nhật.
BiographyPtBr	TEXT	Yes		Tiểu sử tiếng Bồ Đào Nha.
CreatedAt	DATETIME2	Yes		Thời gian tạo profile.
UpdatedAt	DATETIME2	Yes		Thời gian cập nhật profile.

Bảng 29. Bảng Creator

Table: CreatorRelationship				
Field Name	Data Type	Null	Key	Description
Id	INT	No	PK	Khóa chính tự động tăng.
CreatorId	UNIQUEIDENTIFIER	No	FK	Tham chiếu đến Creator.CreatorId.
RelatedId	UNIQUEIDENTIFIER	Yes		ID của thực thể liên quan (Manga/Chapter).
RelatedType	NVARCHAR(50)	Yes		Loại thực thể liên kết: manga/chapter.

Bảng 30. Bảng CreatorRelationship

Table: List

Field Name	Data Type	Null	Key	Description
ListId	UNIQUEIDENTIFIER	No	PK	Khóa chính danh sách đọc (UUID).
UserId	UNIQUEIDENTIFIER	No	FK	Tham chiếu đến User.UserId sở hữu danh sách.
Name	NVARCHAR(200)	Yes		Tên danh sách do người dùng đặt.
Description	TEXT	Yes		Mô tả chi tiết về danh sách.
IsPublic	BIT	Yes		Đánh dấu tính công khai (0/1).
Slug	NVARCHAR(255)	Yes		Đường dẫn Slug thân thiện cho URL.
Visibility	NVARCHAR(20)	No		Tùy chọn hiển thị rõ ràng (public/private).
CreatedAt	DATETIME2	No		Thời gian khởi tạo danh sách.
UpdatedAt	DATETIME2	No		Thời gian cập nhật danh sách.
FollowerCount	INT	Yes		Tổng số người theo dõi danh sách, mặc định 0.
ItemCount	INT	Yes		Tổng số truyện có trong danh sách, mặc định 0.

Bảng 31. Bảng List

Table: ListManga				
Field Name	Data Type	Null	Key	Description

ListId	UNIQUEIDENTIFIER	No	PK, FK	Tham chiếu đến List.ListId.
MangaId	UNIQUEIDENTIFIER	No	PK, FK	Tham chiếu đến Manga.MangaId.
AddedAt	DATETIME2	Yes		Thời điểm thêm truyện vào danh sách.
Position	INT	Yes		Vị trí sắp xếp trong danh sách, mặc định 0.

Bảng 32. Bảng ListManga

Table: ListFollower				
Field Name	Data Type	Null	Key	Description
ListId	UNIQUEIDENTIFIER	No	PK, FK	Tham chiếu đến List.ListId.
UserId	UNIQUEIDENTIFIER	No	PK, FK	Tham chiếu đến User.UserId của người theo dõi.
FollowedAt	DATETIME2	Yes		Thời điểm bắt đầu theo dõi, mặc định UTC now.

Bảng 33. Bảng ListFollower

Table: Covers				
Field	Data Type	Null	Key	Description

Name				
cover_id	UNIQUEIDENTIFIER	No	PK	Khóa chính cover (UUID).
manga_id	UNIQUEIDENTIFIER	No	FK	Tham chiếu đến Manga.MangaId.
type	NVARCHAR(50)	Yes		Loại phân loại ảnh bìa.
description	TEXT	Yes		Mô tả ảnh bìa.
volume	NVARCHAR(50)	Yes		Tên/số của volume sử dụng ảnh bìa này.
fileName	NVARCHAR(255)	Yes		Tên file ảnh bìa.
locale	NVARCHAR(10)	Yes		Ngôn ngữ của ảnh bìa.
createdAt	DATETIMEOFFSET	Yes		Thời gian tạo (bao gồm múi giờ).
updatedAt	DATETIMEOFFSET	Yes		Thời gian cập nhật (bao gồm múi giờ).
version	INT	Yes		Phiên bản của cover.
rel_user_id	UNIQUEIDENTIFIER	Yes		ID người dùng liên quan đã upload (từ API).
url	NVARCHAR(500)	Yes		URL gốc của ảnh bìa.
image_data	VARBINARY(MAX)	Yes		Dữ liệu binary ảnh. Cột này để phục vụ test

Bảng 34. Bảng Covers

Table: MangaCovers (Bảng Staging)				
Field Name	Data Type	Null	Key	Description

cover_id	NVARCHAR(50)	No	PK	Khóa chính dạng chuỗi UUID.
manga_id	UNIQUEIDENTIFIER	No	FK	Tham chiếu đến Manga.MangaId.
file_name	NVARCHAR(300)	Yes		Tên file ảnh bìa.
description	TEXT	Yes		Đoạn mô tả.
volume	NVARCHAR(50)	Yes		Số volume áp dụng.
locale	NVARCHAR(20)	Yes		Ngôn ngữ quy định.
created_at	DATETIMEOFFSET	Yes		Thời gian tạo (từ hệ thống nguồn).
updated_at	DATETIMEOFFSET	Yes		Thời gian cập nhật (từ hệ thống nguồn).
version	INT	Yes		Số hiệu phiên bản.
image_url	NVARCHAR(1000)	Yes		URL lấy ảnh.
file_path	NVARCHAR(1000)	Yes		Đường dẫn lưu file tương đối.
downloaded_at	DATETIMEOFFSET	Yes		Thời điểm tiến hành tải ảnh.
raw_json	TEXT	Yes		Chứa nội dung JSON thô nguyên bản từ API.
inserted_at	DATETIMEOFFSET	Yes		Thời gian insert dữ liệu vào DB, mặc định now.

Bảng 35. Bảng MangaCovers

Table: ChatMessage				
Field Name	Data Type	Null	Key	Description

MessageId	UNIQUEIDENTIFIER	No	PK	Khóa chính định danh duy nhất cho tin nhắn.
RoomId	UNIQUEIDENTIFIER	No	FK	Tham chiếu đến ChatRoom.RoomId.
SenderId	UNIQUEIDENTIFIER	No	FK	Tham chiếu đến User.UserId.
Content	NVARCHAR(MAX)	Yes		Nội dung văn bản của tin nhắn.
MessageType	NVARCHAR(20)	Yes		Loại tin nhắn (text, image, file, video, ...), mặc định là text.
MediaUrl	NVARCHAR(500)	Yes		URL của file đa phương tiện đính kèm.
ReplyToId	UNIQUEIDENTIFIER	Yes	FK	Tham chiếu đến tin nhắn được trả lời.
Status	NVARCHAR(20)	Yes		Trạng thái tin nhắn (sent, delivered, seen, ...), mặc định là sent.
CreatedAt	DATETIME2(7)	Yes		Thời điểm tạo tin nhắn, mặc định UTC hiện tại.
EditedAt	DATETIME2(7)	Yes		Thời điểm chỉnh sửa tin nhắn gần nhất.
IsDeleted	BIT	Yes		Đánh dấu trạng thái xóa mềm, mặc định 0.

Bảng 36. Bảng ChatMessage

Table: ChatRoom				
Field Name	Data Type	Null	Key	Description

RoomId	UNIQUEIDENTIFIER	No	PK	Khóa chính định danh duy nhất cho phòng chat.
Type	NVARCHAR(20)	No		Loại phòng chat (direct, group, ...), mặc định là direct.
Name	NVARCHAR(200)	Yes		Tên hiển thị của phòng chat hoặc nhóm chat.
AvatarUrl	NVARCHAR(500)	Yes		URL ảnh đại diện của phòng chat.
CreatedBy	UNIQUEIDENTIFIER	Yes	FK	Tham chiếu đến User.UserId của người tạo phòng.
CreatedAt	DATETIME2(7)	Yes		Thời điểm tạo phòng chat, mặc định UTC hiện tại.
UpdatedAt	DATETIME2(7)	Yes		Thời điểm cập nhật gần nhất của phòng chat.

Bảng 37. Bảng ChatRoom

Table: ChatRoomMember				
Field Name	Data Type	Null	Key	Description
RoomId	UNIQUEIDENTIFIER	No	PK, FK	Tham chiếu đến ChatRoom.RoomId.
UserId	UNIQUEIDENTIFIER	No	PK, FK	Tham chiếu đến User.UserId.
JoinedAt	DATETIME2(7)	Yes		Thời điểm người dùng tham gia phòng chat.

Role	NVARCHAR(20)	Yes		Vai trò trong phòng chat (member, admin, owner, ...), mặc định là member.
LastReadAt	DATETIME2(7)	Yes		Thời điểm đọc tin nhắn gần nhất.
Nickname	NVARCHAR(100)	Yes		Biệt danh của người dùng trong phòng chat.
IsMuted	BIT	Yes		Trạng thái tắt thông báo của thành viên, mặc định 0.

Bảng 38. Bảng ChatRoomMember

Table: Friendship				
Field Name	Data Type	Null	Key	Description
UserId	UNIQUEIDENTIFIER	No	PK, FK	Tham chiếu đến User.UserId.
FriendId	UNIQUEIDENTIFIER	No	PK, FK	Tham chiếu đến User.UserId của người được kết bạn.
Status	NVARCHAR(20)	No		Trạng thái quan hệ bạn bè (pending, accepted, blocked, ...), mặc định là pending.
CreatedAt	DATETIME2(7)	Yes		Thời điểm tạo yêu cầu kết bạn.

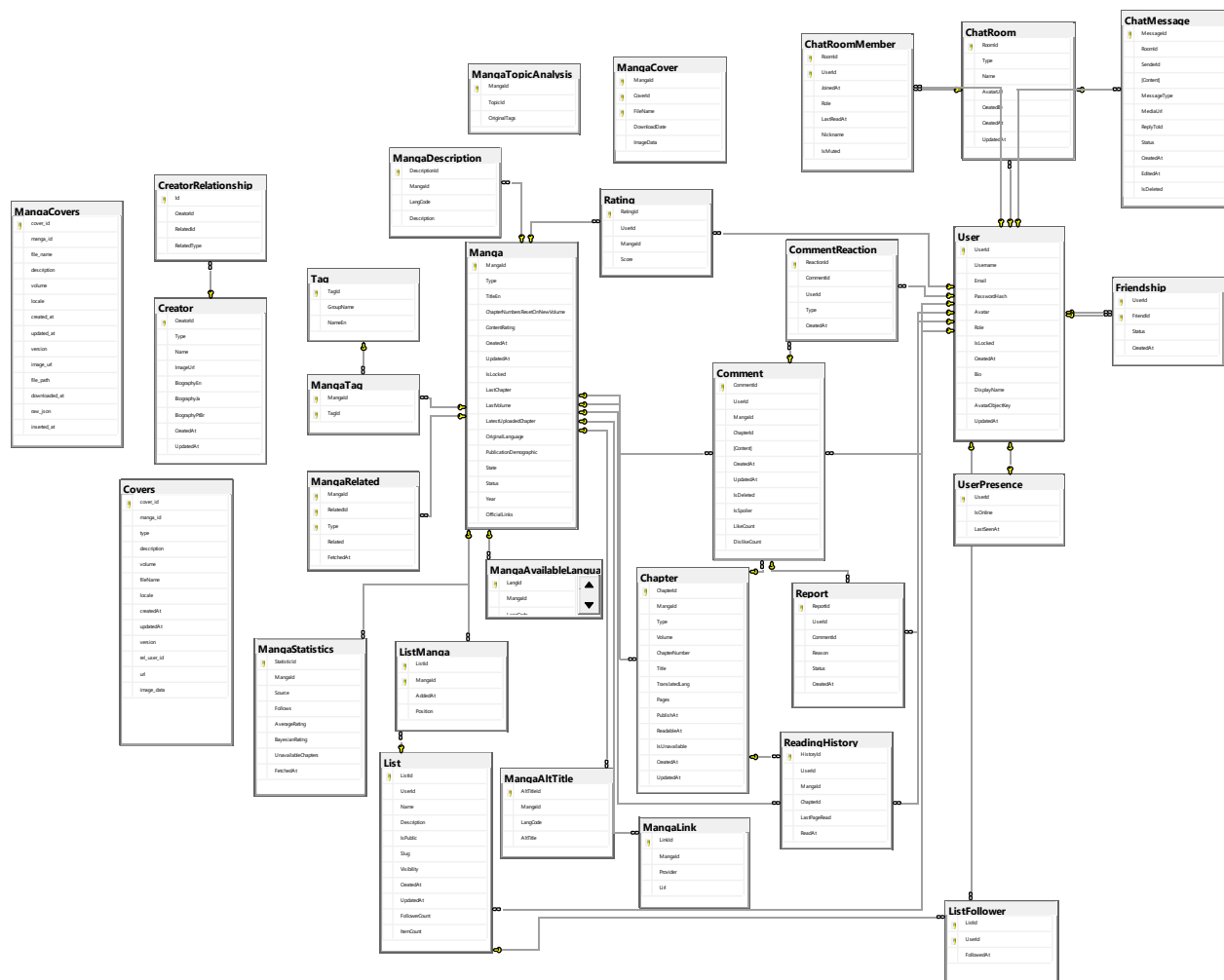
Bảng 39. Bảng Friendship

Table: UserPresence

Field Name	Data Type	Null	Key	Description
UserId	UNIQUEIDENTIFIER	No	PK, FK	Tham chiếu đến User.UserId.
IsOnline	BIT	Yes		Trạng thái trực tuyến của người dùng, mặc định 0.
LastSeenAt	DATETIME2(7)	Yes		Thời điểm hoạt động cuối cùng của người dùng.

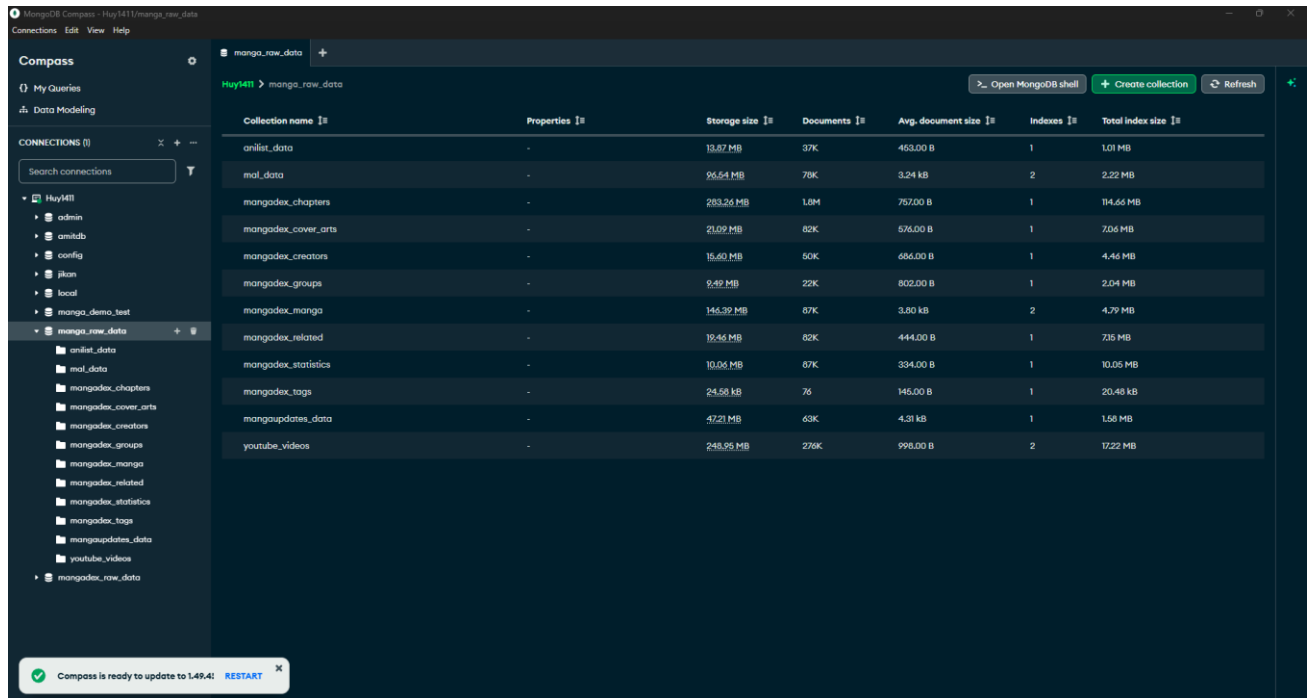
Bảng 40. Bảng UserPresence

Minh họa cho lược đồ quan hệ:



Hình 26. ERD của DB nghiệp vụ

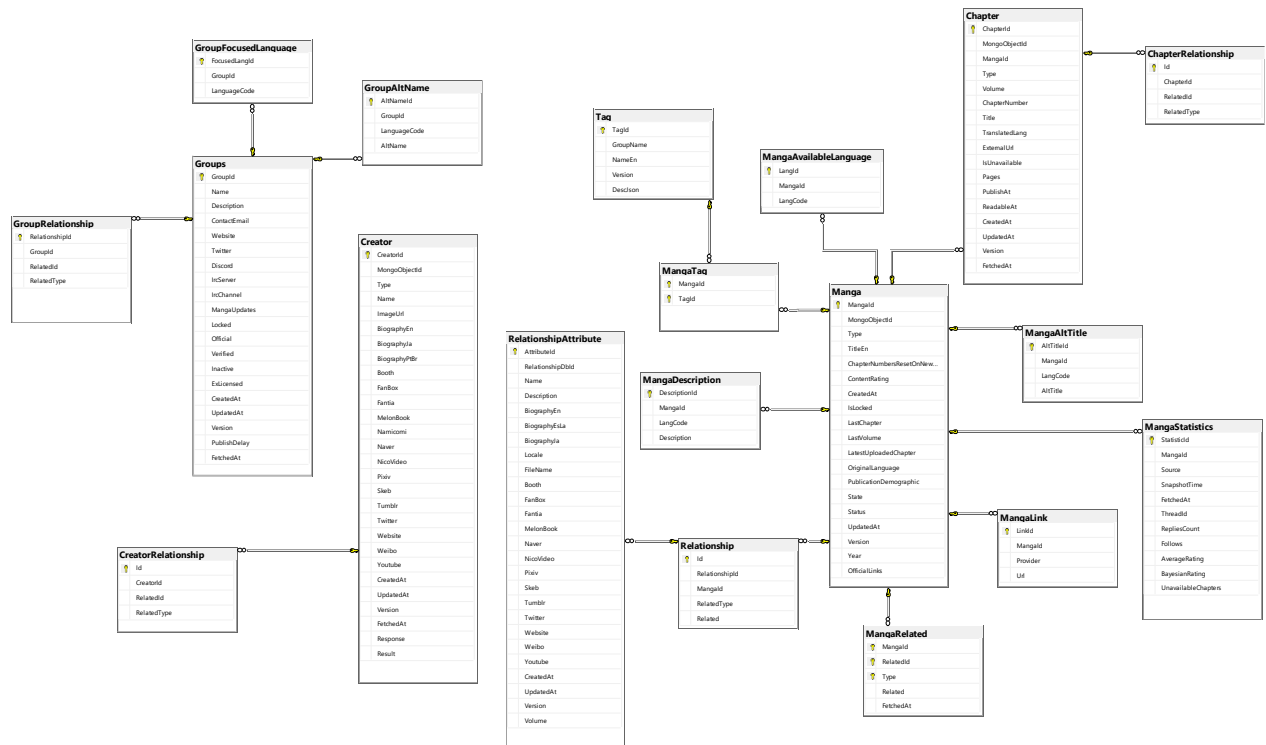
Bên trên là dữ liệu dạng bảng trong hệ thống (nghiệp vụ), nhưng như đã đề cập trong các kiến trúc trước nhóm em có sử dụng kiến trúc Pipeline ETL, mà trong đó làm việc với dữ liệu kiểu JSON trong MongoDB, Các bảng dữ liệu trên phần lớn đều được ETL từ các document thu thập được từ API nằm trong MongoDB:



Collection name	Properties	Storage size	Documents	Avg. document size	Indexes	Total index size
anlist_data	-	13.87 MB	37K	453.00 B	1	1.01 MB
mal_data	-	95.54 MB	78K	3.24 kB	2	2.22 MB
mangadex_chapters	-	283.26 MB	1.0M	757.00 B	1	114.46 MB
mangadex_cover_arts	-	21.02 MB	82K	576.00 B	1	7.06 MB
mangadex_creators	-	16.60 MB	50K	486.00 B	1	4.46 MB
mangadex_groups	-	9.49 MB	22K	802.00 B	1	2.04 MB
mangadex_manga	-	146.39 MB	87K	3.80 kB	2	4.79 MB
mangadex_related	-	19.46 MB	82K	444.00 B	1	7.15 MB
mangadex_statistics	-	10.04 MB	87K	334.00 B	1	10.05 MB
mangadex_tags	-	24.58 kB	76	145.00 B	1	20.48 kB
mangaupdates_data	-	47.21 MB	63K	4.31 kB	1	1.58 MB
youtube_videos	-	248.95 MB	276K	998.00 B	2	17.22 MB

Hình 27. Danh sách các document sau khi thu thập từ Mangadex API

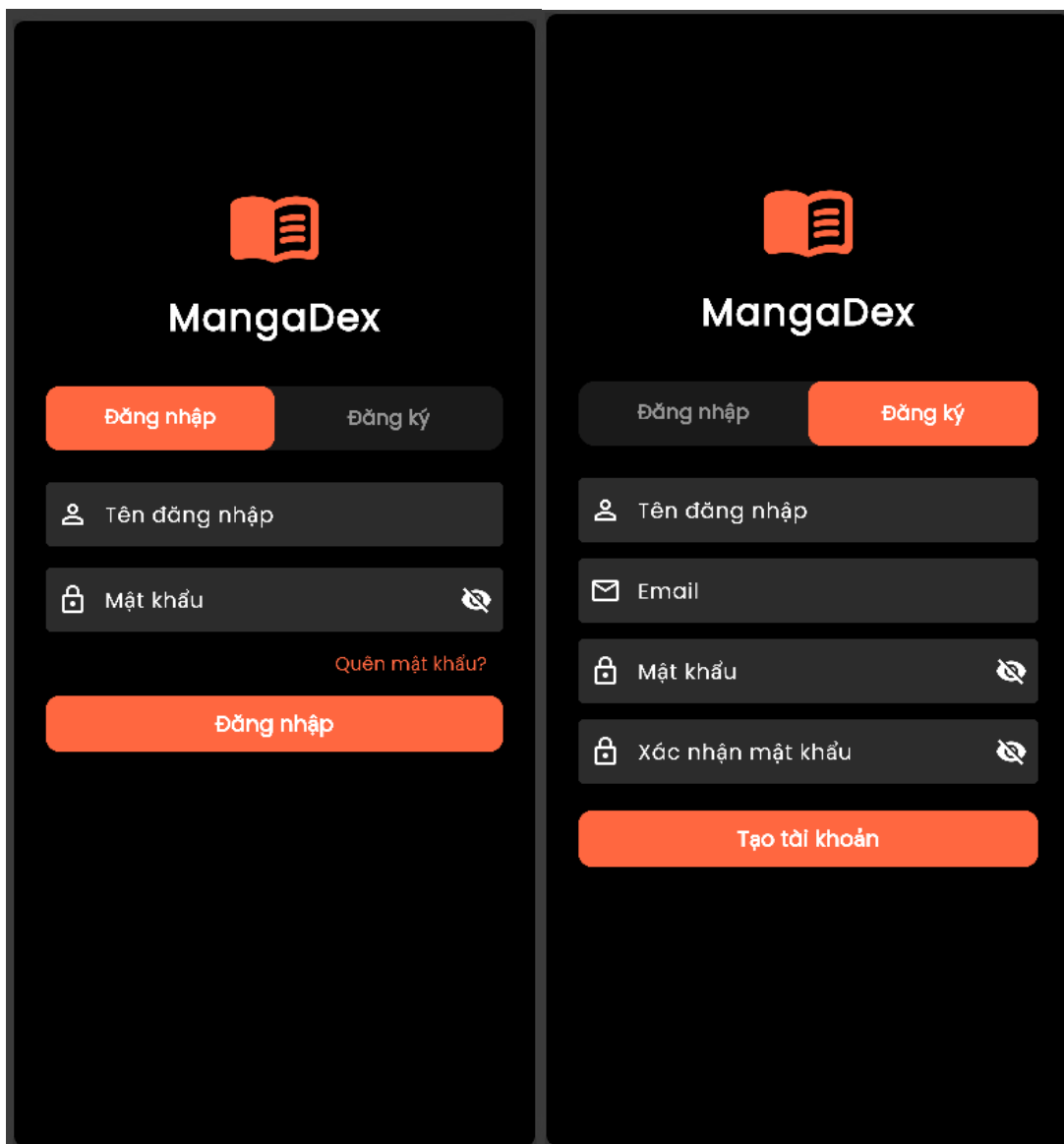
Sau khi thu thập xong dữ liệu sẽ tiến hành ETL lần 1 vào một Staging DB, tại đây sẽ tái cấu trúc từ dữ liệu JSON thành các bảng dữ liệu đã được chia nhỏ. Có thể xem như dữ liệu thô có cấu trúc tabular.



Hình 28. ERD của DB Staging

2.2. Thiết kế giao diện của hệ thống⁴

2.2.1. Phân Hệ User

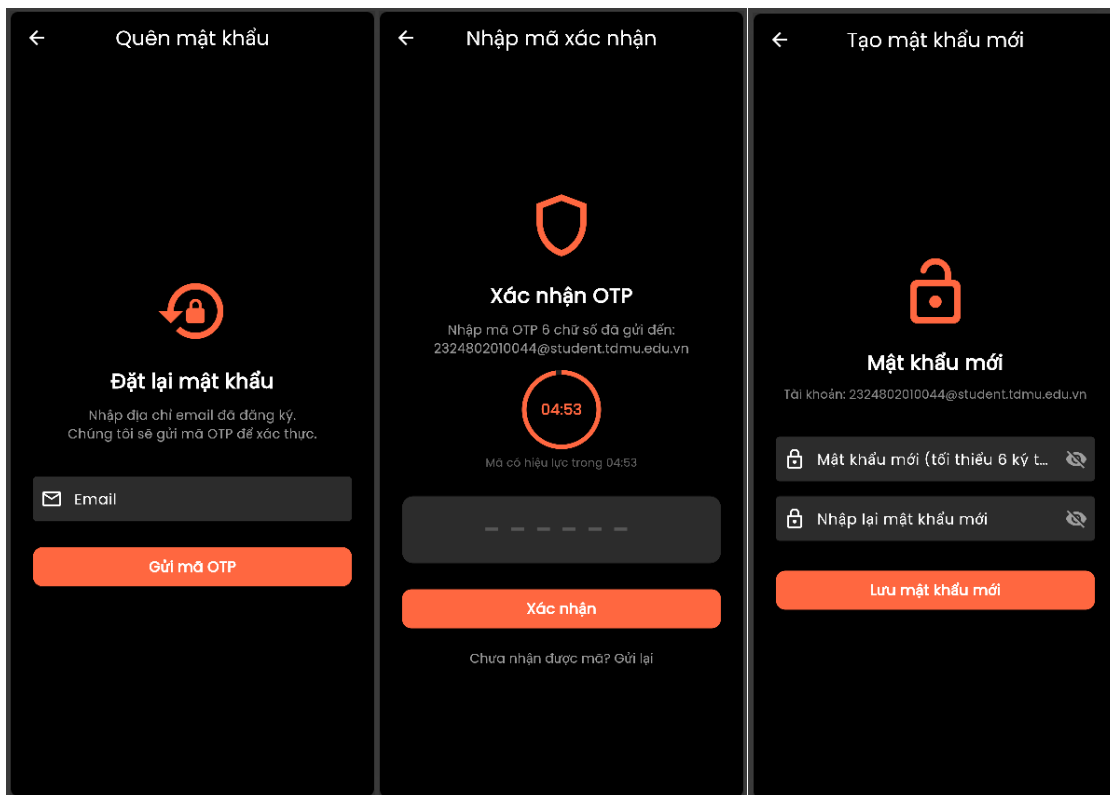


Hình 29. Giao diện authentication của app

⁴ Toàn bộ source code dự án đã được đẩy lên github ở:

<https://github.com/huynxuantruong1411/coursework-KTTKPM-2026>

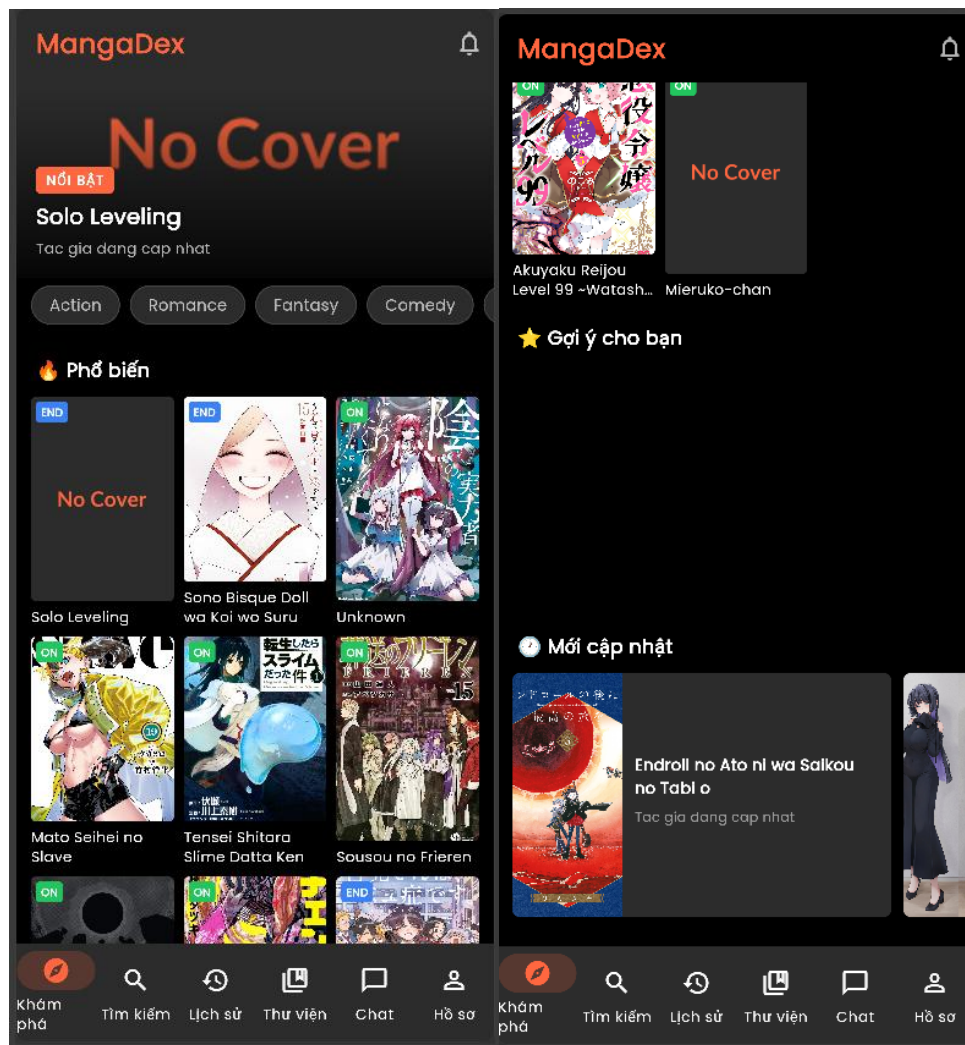
Do backend API đã có sẵn nên việc dùng client rất linh hoạt, thay vì đi vào lỗi mòn chỉ build web, nhóm em quyết định build thành một app Flutter di động đa nền tảng (có thể xem như một web – app trên giao diện điện thoại).



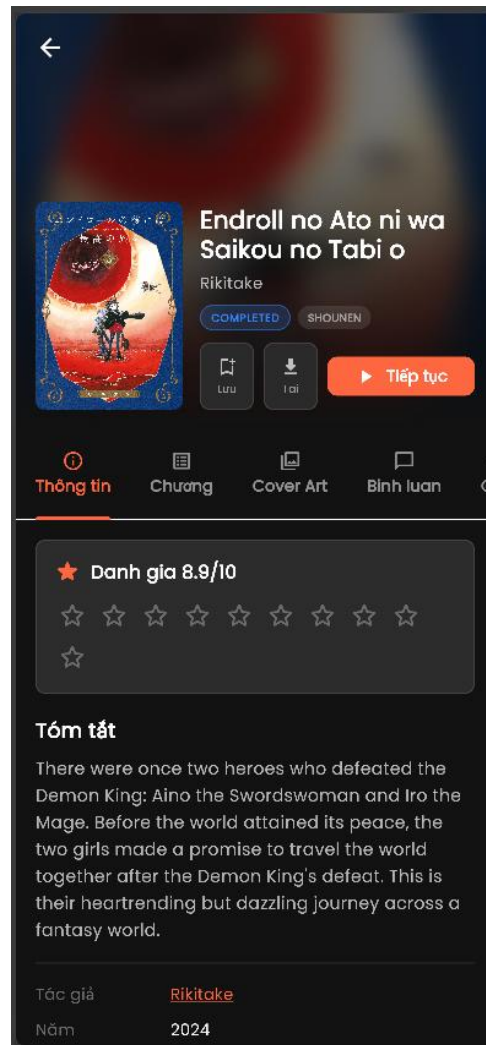
Hình 30. Quy trình Dùng OTP cho xác thực tài khoản



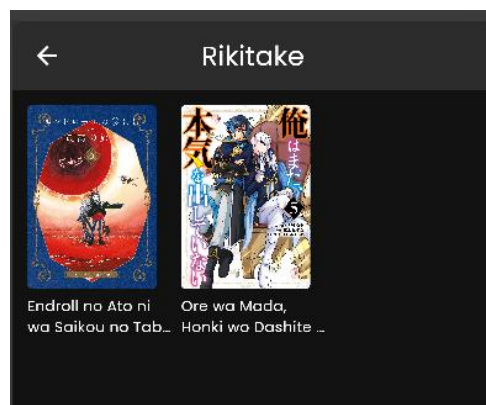
Hình 31. Hệ thống được setup SMTP service thủ công



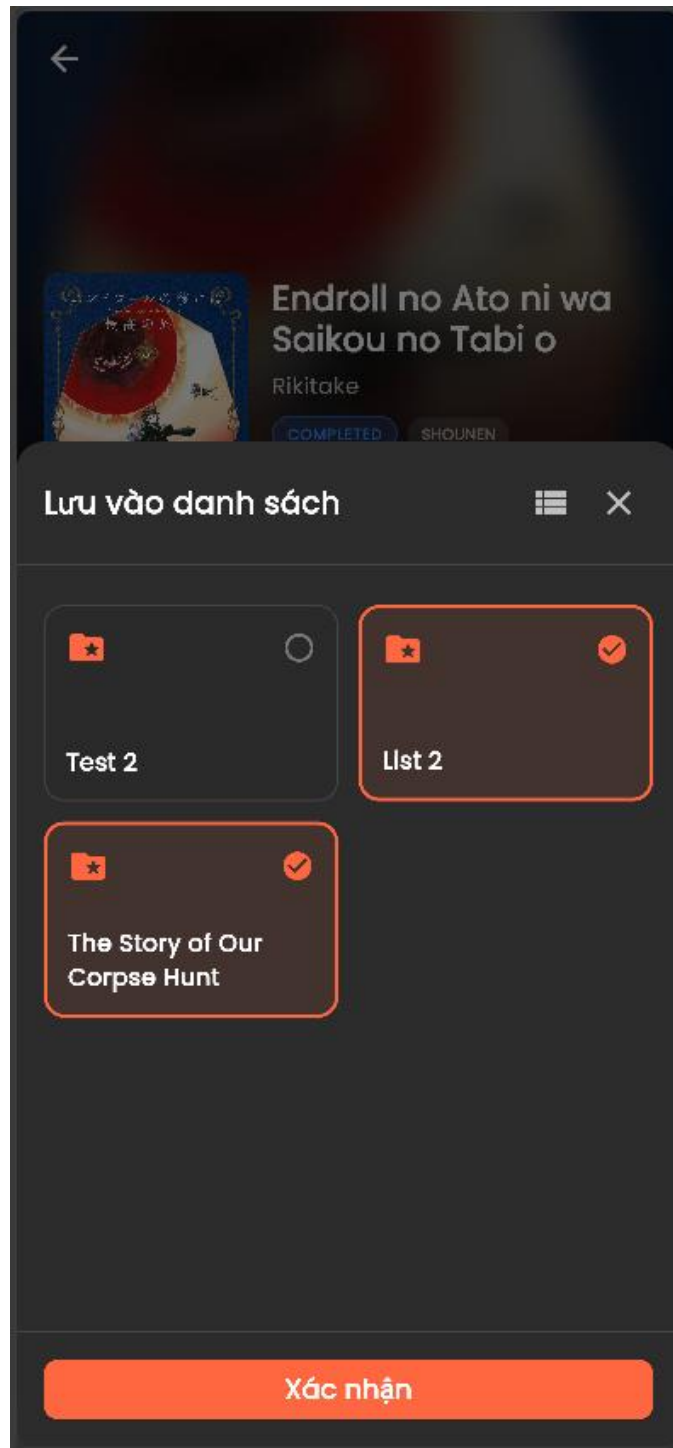
Hình 32. Giao diện trang chủ



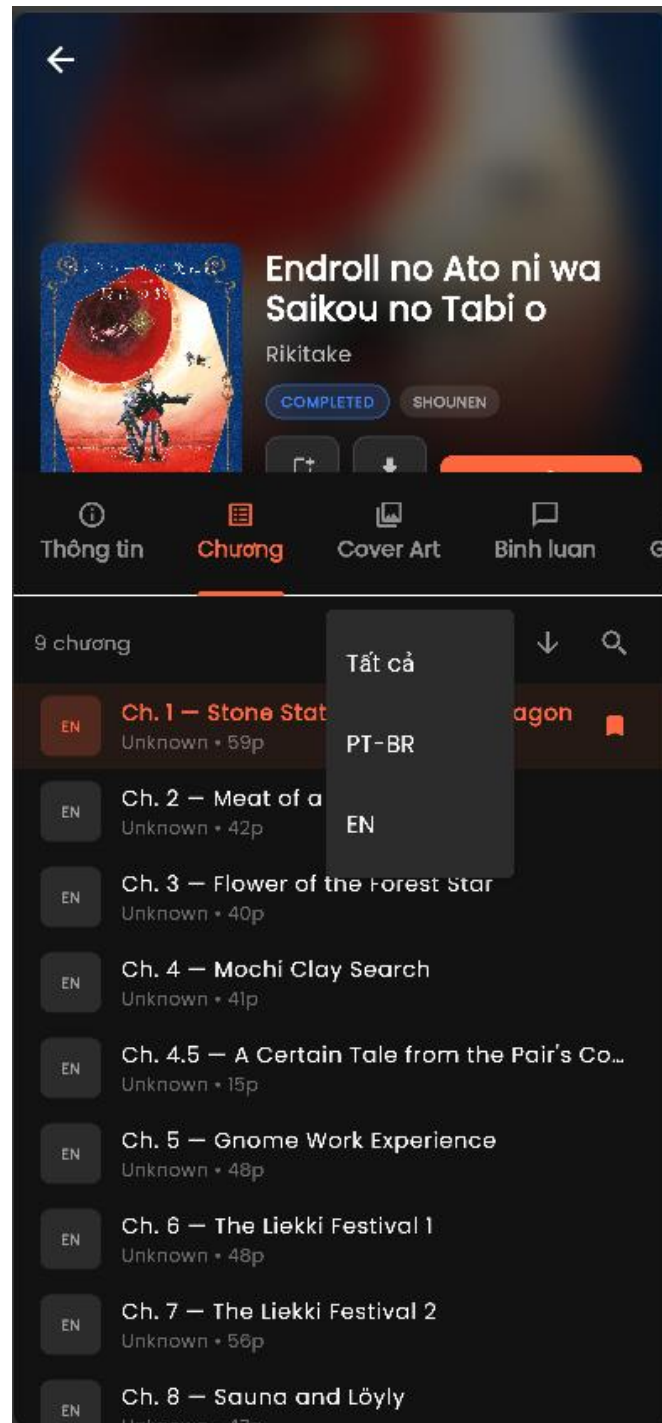
Hình 33. Giao diện xem chi tiết manga



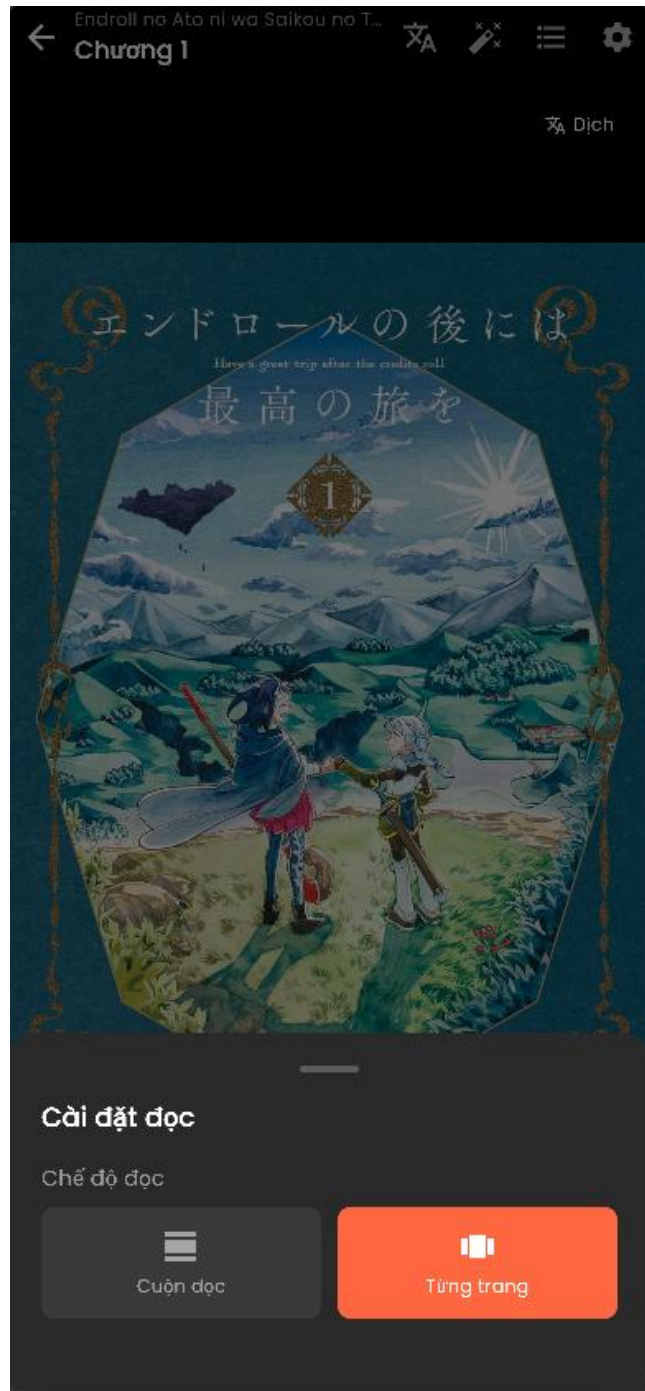
Hình 34. Giao diện xem danh sách manga theo Author/Artist



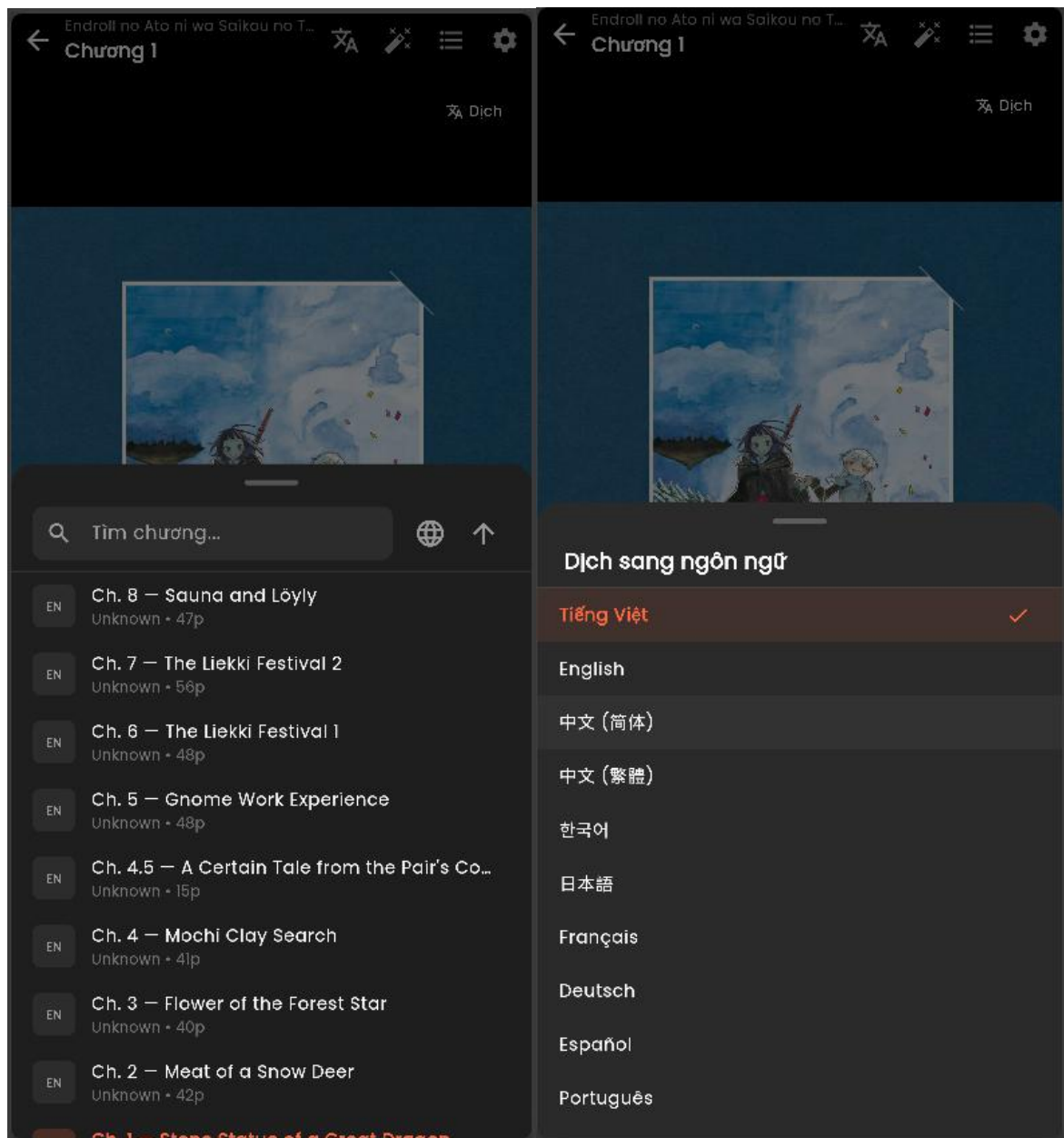
Hình 35. Thêm/Xóa manga trong list tại trang chi tiết manga



Hình 36. Giao diện xem danh sách chapter có tùy chỉnh ngôn ngữ



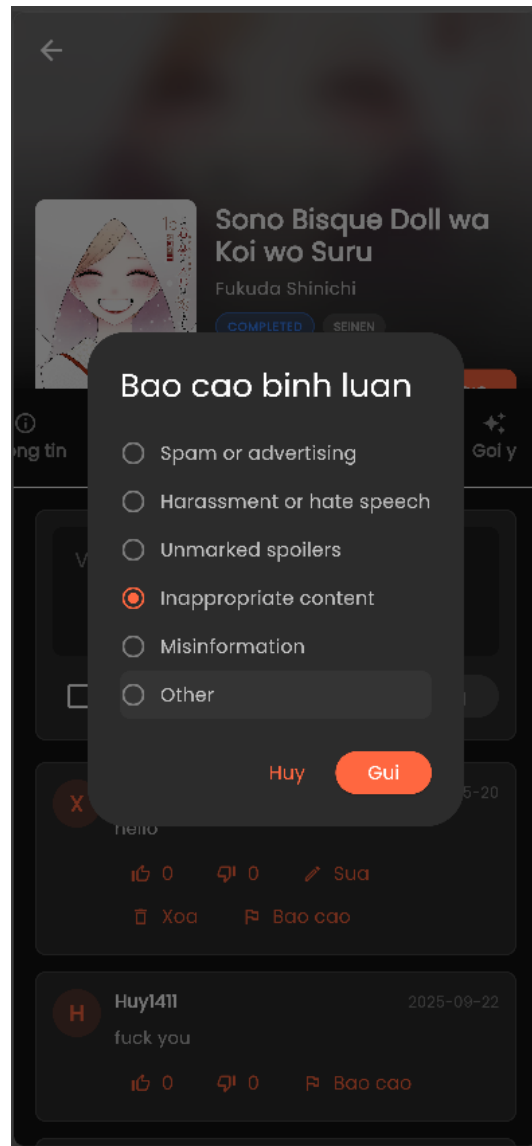
Hình 37. Giao diện đọc truyện có thể tùy chỉnh layout



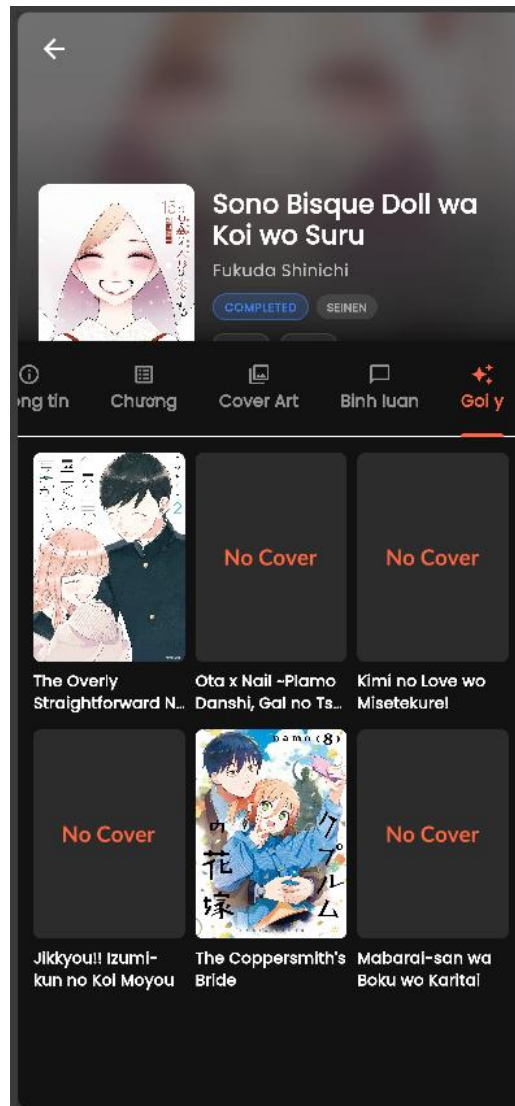
Hình 38. Các setting cơ bản trong lúc đọc truyện



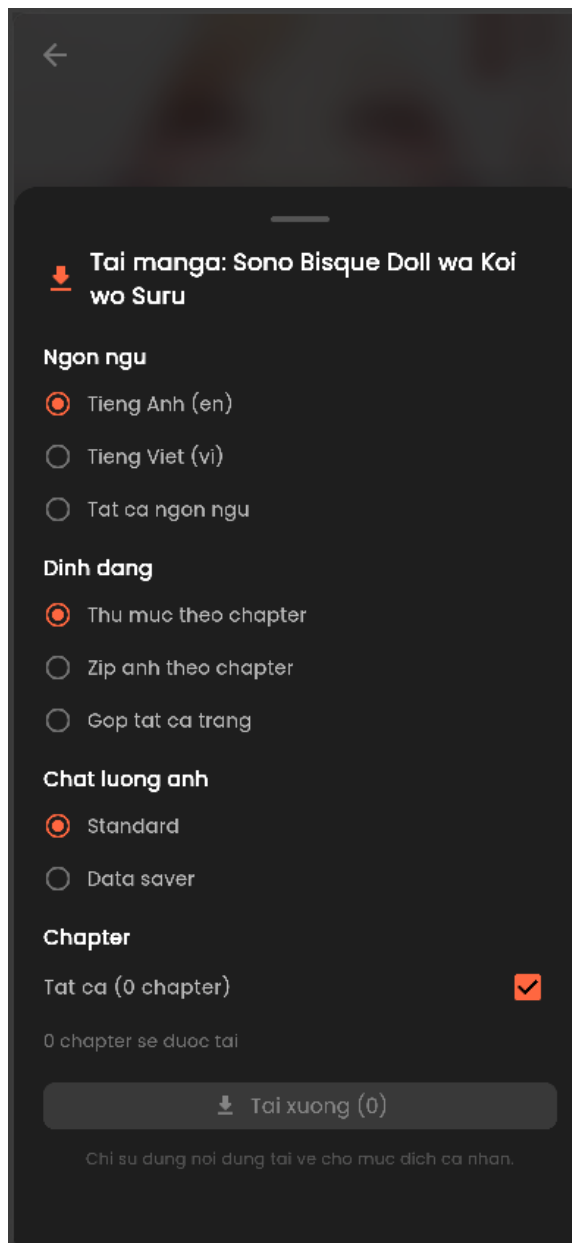
Hình 39. Demo chức năng dịch trang truyện



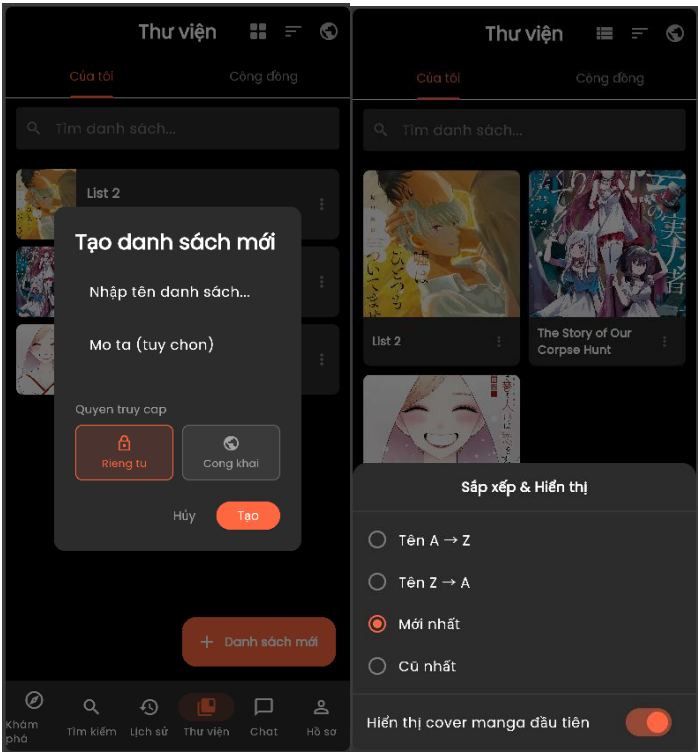
Hình 40. Giao diện comment và report



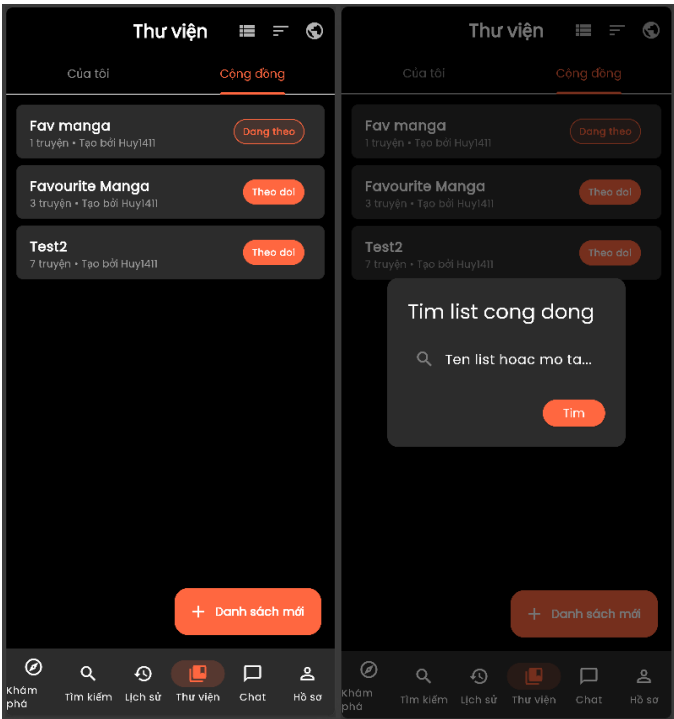
Hình 41. Giao diện trang Related Manga (Hệ tư vấn)



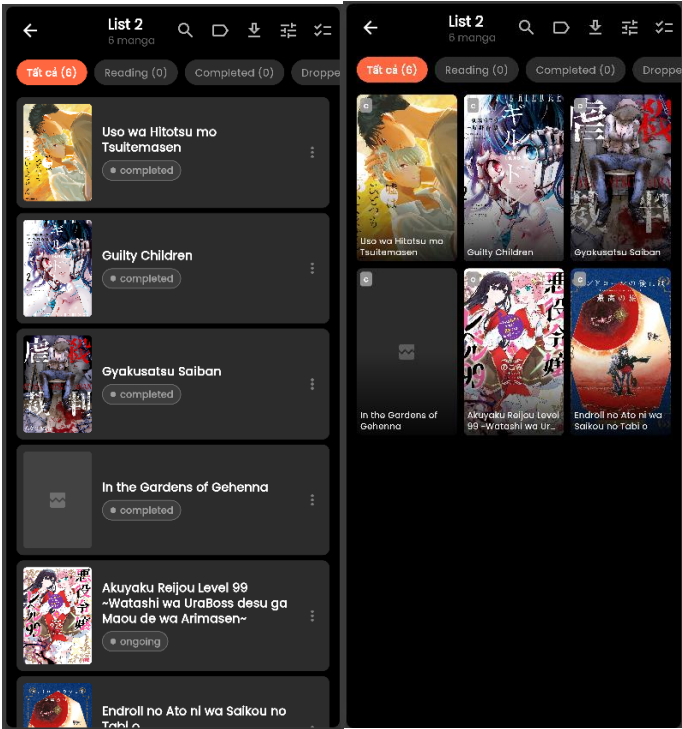
Hình 42. Giao diện cho phép tải manga về máy



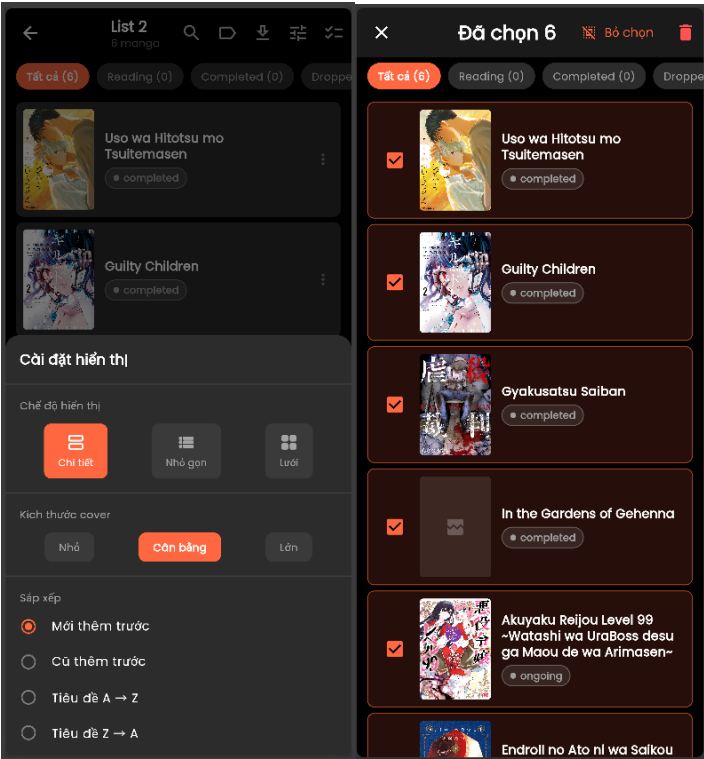
Hình 43. Giao diện quản lý danh sách cá nhân của người đọc



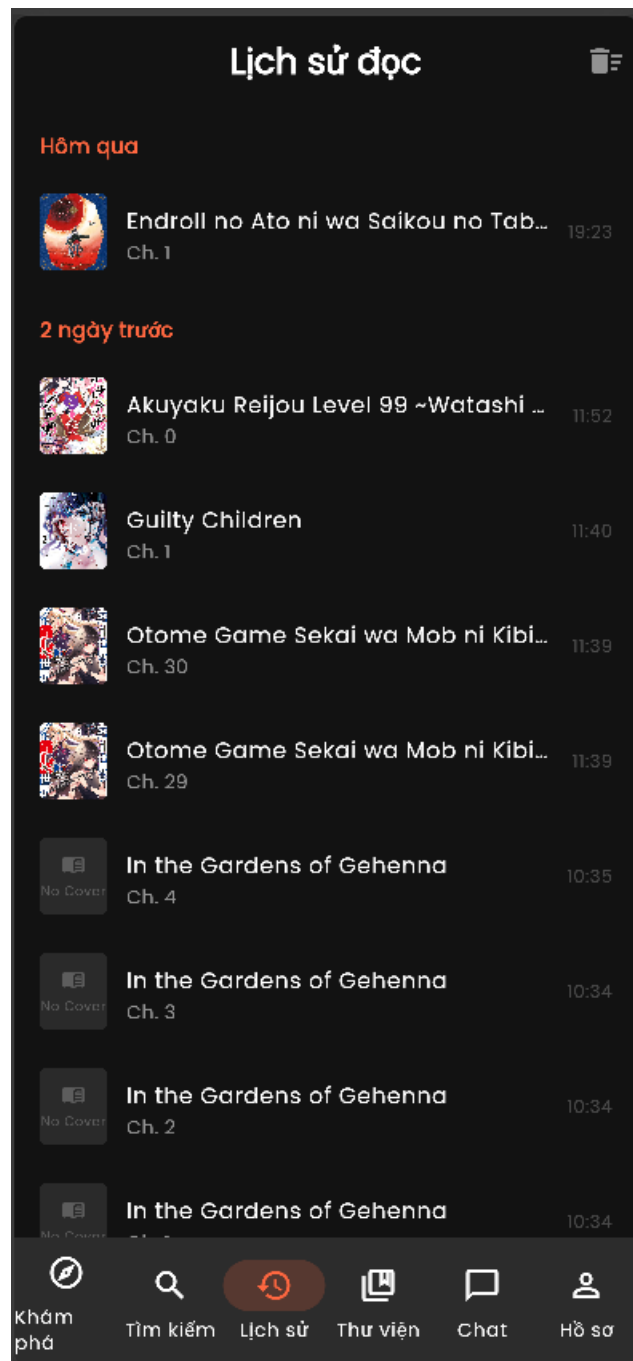
Hình 44. Giao diện search/follow list của user khác



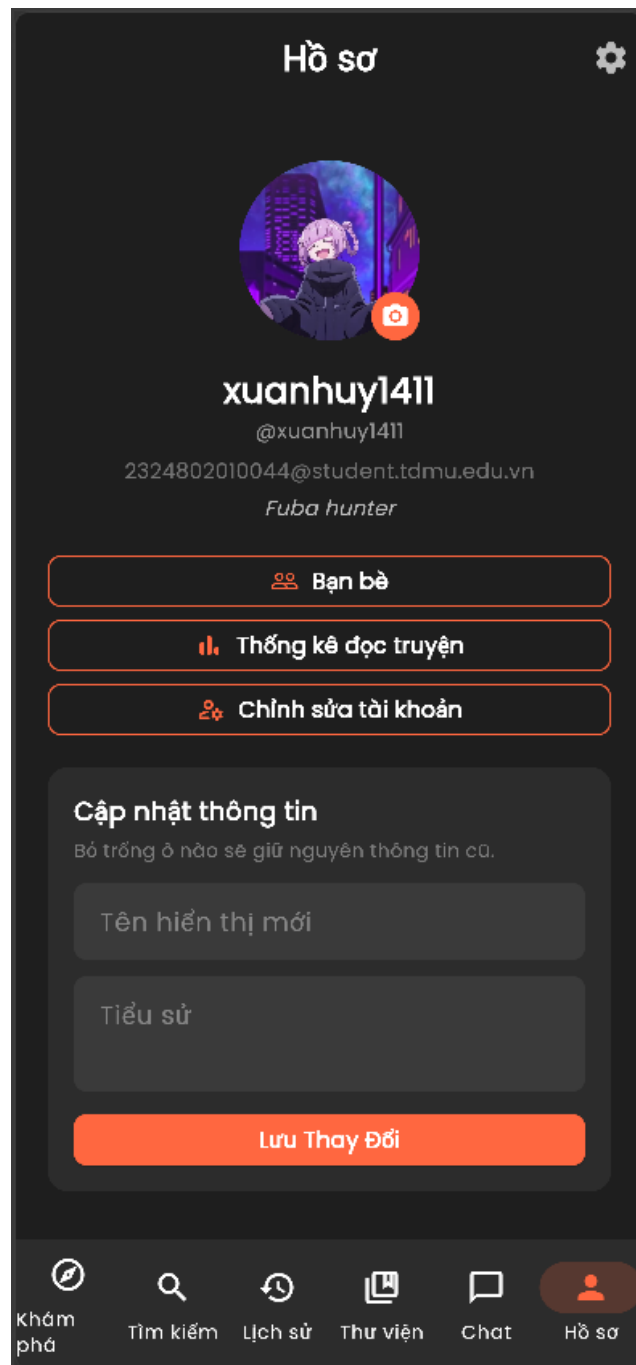
Hình 45. Giao diện quản lý manga trong list



Hình 46. Các tiện ích hỗ trợ quản lý manga trong list



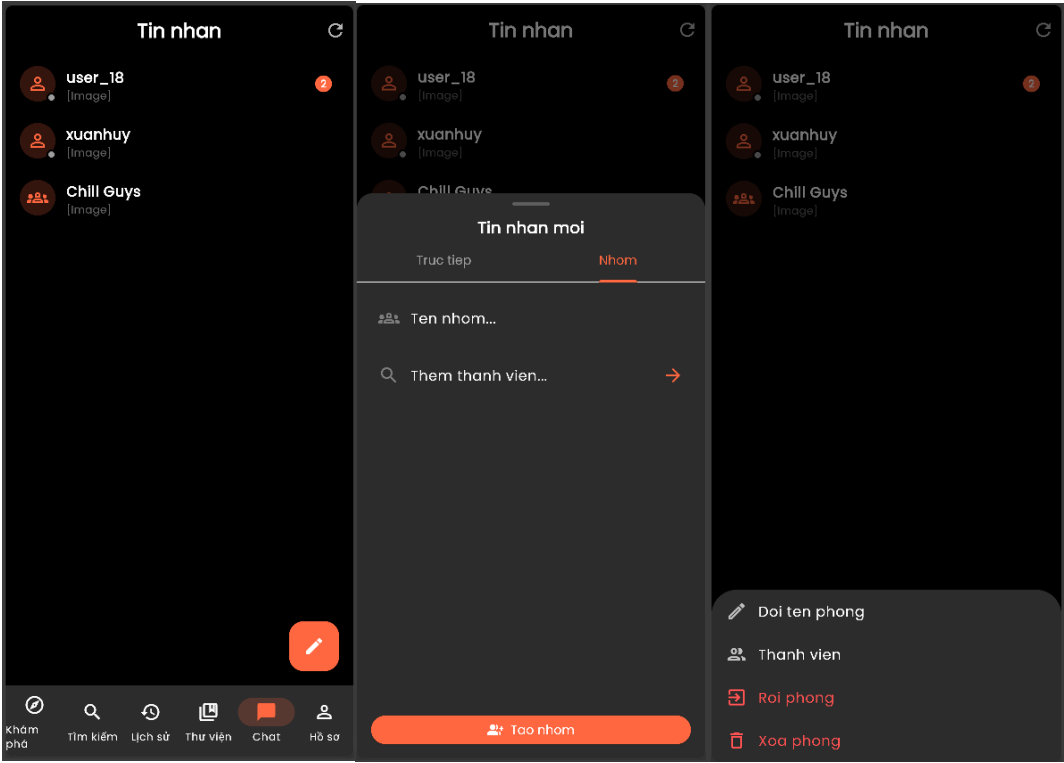
Hình 47. Giao diện xem lịch sử đọc



Hình 48. Giao diện trang cá nhân user, có thể thay đổi avatar



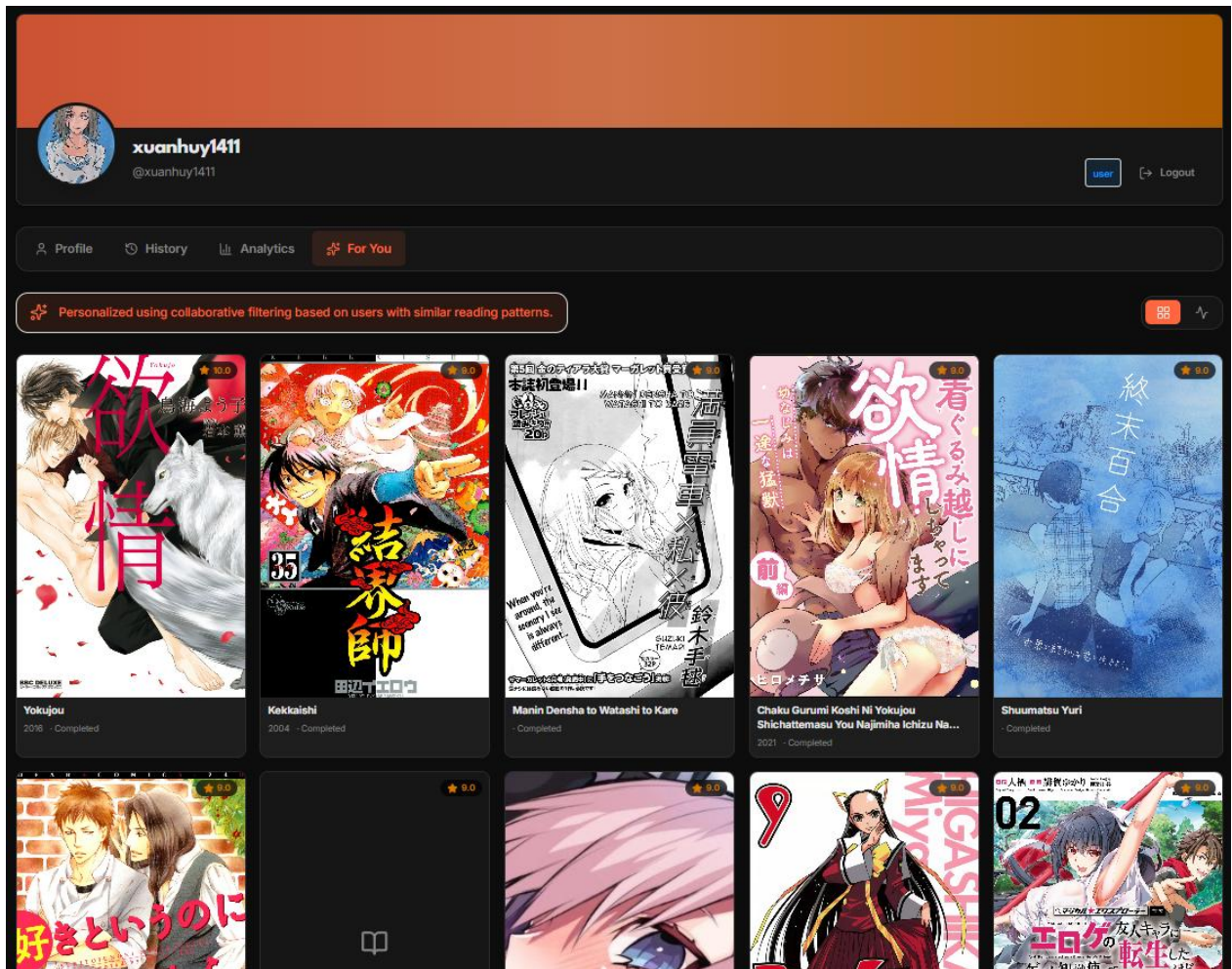
Hình 49. Giao diện phân tích thói quen đọc dựa trên dữ liệu lịch sử



Hình 50. Giao diện quản lý các dịch vụ chat



Hình 51. Giao diện trang chat có thể gửi/xem/tải ảnh



Hình 52. Giao diện hệ tư vấn theo cá nhân sử dụng lọc cộng tác

Cơ chế gợi ý nội dung được triển khai dựa trên mô hình User-based Collaborative Filtering – một trong những phương pháp kinh điển và phổ biến nhất trong lĩnh vực Recommender Systems. Phương pháp này khai thác hành vi đánh giá của cộng đồng người dùng nhằm xác định các mẫu sở thích tương đồng, từ đó đưa ra các đề xuất manga phù hợp cho từng cá nhân. Khác với các hệ thống dựa trên nội dung (Content-based Filtering), mô hình Collaborative Filtering không phụ thuộc vào mô tả, thể loại hay metadata của manga, mà tập trung hoàn toàn vào hành vi đánh giá thực tế của người dùng.

Về mặt nguyên lý, hệ thống giả định rằng những người dùng có xu hướng đánh giá giống nhau trong quá khứ sẽ có khả năng yêu thích các manga tương tự trong tương lai. Nói cách khác, nếu hai người dùng cùng đánh giá cao một tập hợp manga giống nhau, hệ thống sẽ xem họ là những người có “gu đọc” tương đồng. Dựa trên mức độ tương đồng đó, hệ thống có thể suy luận và đề xuất các manga mà một người dùng chưa đọc nhưng lại được đánh giá cao bởi các người dùng tương tự khác.

Toàn bộ dữ liệu đánh giá được biểu diễn dưới dạng một ma trận User–Item Rating Matrix. Trong ma trận này, mỗi hàng đại diện cho một người dùng, mỗi cột đại diện cho một manga, còn giá trị tại giao điểm giữa hàng và cột tương ứng chính là điểm đánh giá mà người dùng dành cho manga đó. Nếu người dùng chưa từng đánh giá manga, giá trị tại ô tương ứng sẽ bằng 0 hoặc được xem là dữ liệu khuyết.

Ví dụ, giả sử hệ thống có ba người dùng và bốn manga khác nhau, ma trận đánh giá có thể được biểu diễn như sau:

User	Naruto	One Piece	Berserk	Vagabond
User A	10	9	0	8
User B	9	10	0	7
User C	2	1	10	9

Bảng 41. Ví dụ ma trận User – Item

Trong ví dụ trên, User A và User B có xu hướng đánh giá rất giống nhau đối với Naruto, One Piece và Vagabond, trong khi User C có hành vi đánh giá khác biệt đáng kể. Hệ thống sẽ sử dụng dữ liệu này để xác định mức độ tương đồng giữa các người dùng.

Để xây dựng ma trận đánh giá, hệ thống trước tiên thu thập toàn bộ dữ liệu rating từ cơ sở dữ liệu. Mỗi bản ghi bao gồm định danh người dùng, định danh manga và điểm đánh giá tương ứng. Sau đó, hệ thống ánh xạ tập người dùng và tập manga sang các chỉ số

số nguyên nhằm tối ưu quá trình truy cập dữ liệu trong bộ nhớ. Việc ánh xạ này giúp chuyển đổi các UUID phức tạp thành chỉ số hàng và cột trong ma trận hai chiều, từ đó cho phép thực hiện các phép toán đại số tuyến tính hiệu quả hơn.

Sau khi ma trận được khởi tạo, hệ thống tiến hành bước quan trọng nhất: Xác định độ tương đồng giữa người dùng. Trong mô hình hiện tại, độ tương đồng được tính bằng Cosine Similarity – một phương pháp phổ biến trong khai phá dữ liệu và hệ gợi ý. Cosine Similarity đo góc giữa hai vector đánh giá trong không gian nhiều chiều, từ đó phản ánh mức độ giống nhau trong hành vi rating giữa hai người dùng. Công thức tính Cosine Similarity được biểu diễn như sau:

$$\text{sim}(u, v) = \frac{u \cdot v}{||u|| \times ||v||}$$

Trong đó:

- u và v là hai vector đánh giá của hai người dùng;
- $u \cdot v$ là tích vô hướng giữa hai vector;
- $||u||$ và $||v||$ là độ dài Euclidean của các vector tương ứng.

Giá trị similarity nằm trong khoảng từ -1 đến 1:

- Giá trị gần 1 cho thấy hai người dùng có sở thích rất giống nhau;
- Giá trị gần 0 biểu thị không có mối liên hệ rõ ràng;
- Giá trị âm thể hiện xu hướng đánh giá đối lập.

Tuy nhiên, trong hệ thống recommendation thực tế, việc tính toán similarity trên toàn bộ vector thường gây sai lệch do tồn tại nhiều giá trị chưa đánh giá. Vì lý do đó, hệ thống chỉ xét các manga mà cả hai người dùng đều đã rating. Cơ chế này giúp loại bỏ nhiễu và phản ánh chính xác hơn mức độ tương đồng thực tế giữa các người dùng.

Ví dụ, nếu User A và User B đều đánh giá Naruto và One Piece, hệ thống chỉ sử dụng hai manga này để tính similarity, thay vì toàn bộ tập manga tồn tại trong cơ sở dữ liệu. Điều này đặc biệt quan trọng trong các hệ thống recommendation quy mô lớn, nơi dữ liệu rating thường rất thưa.

Sau khi tính similarity giữa người dùng hiện tại với toàn bộ người dùng còn lại trong hệ thống, thuật toán tiến hành lựa chọn tập K Nearest Neighbors – tức K người dùng có độ tương đồng cao nhất. Đây là bước cốt lõi của User-based Collaborative Filtering, bởi toàn bộ quá trình dự đoán sau đó đều dựa trên hành vi của nhóm người dùng tương tự này.

Ví dụ, nếu hệ thống xác định rằng User B, User D và User E có similarity cao nhất đối với User A, thì mọi recommendation dành cho User A sẽ chủ yếu dựa trên các manga được yêu thích bởi ba người dùng nói trên.

Tiếp theo, hệ thống thực hiện dự đoán điểm đánh giá cho các manga mà người dùng hiện tại chưa đọc. Quá trình này được thực hiện thông qua phương pháp Weighted Average Prediction. Ý tưởng chính là: nếu nhiều người dùng tương tự đều đánh giá cao một manga, thì khả năng cao người dùng hiện tại cũng sẽ yêu thích manga đó.

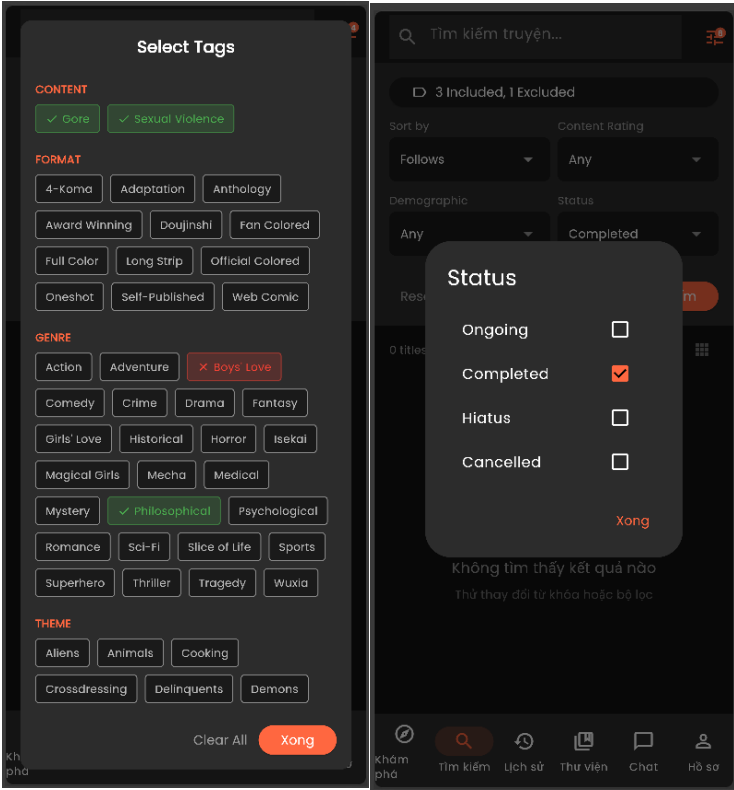
Điểm dự đoán được tính bằng công thức:

$$\hat{r}_{u,i} = \frac{\sum_{v \in N(u)} sim(u, v) \times r_{v,i}}{\sum_{v \in N(u)} |sim(u, v)|}$$

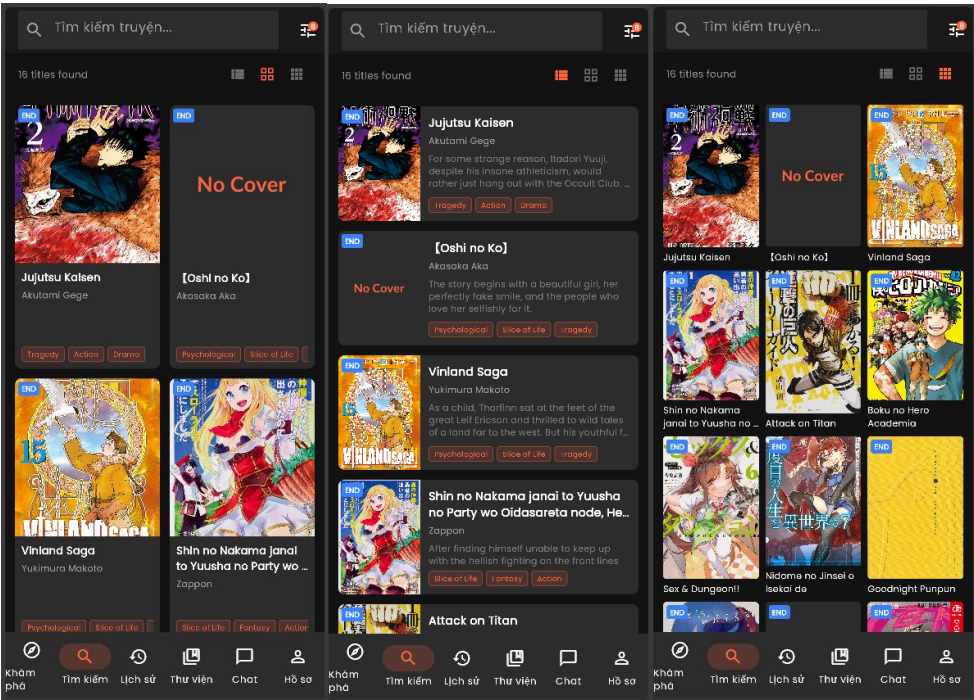
Trong đó:

- $\hat{r}_{u,i}$ là điểm dự đoán của người dùng u đối với manga i ;
- $N(u)$ là tập người dùng tương đồng với u ;
- $sim(u, v)$ là độ tương đồng giữa người dùng u và v ;
- $r_{v,i}$ là điểm đánh giá mà người dùng v dành cho manga i .

Công thức trên thực chất là một dạng trung bình cộng có trọng số, trong đó các người dùng có similarity cao hơn sẽ có ảnh hưởng lớn hơn đến kết quả cuối cùng. Điều này giúp recommendation phản ánh đúng sở thích cá nhân thay vì chỉ dựa trên xu hướng số đông.



Hình 53. Hỗ trợ nhiều option linh hoạt cho chức năng search

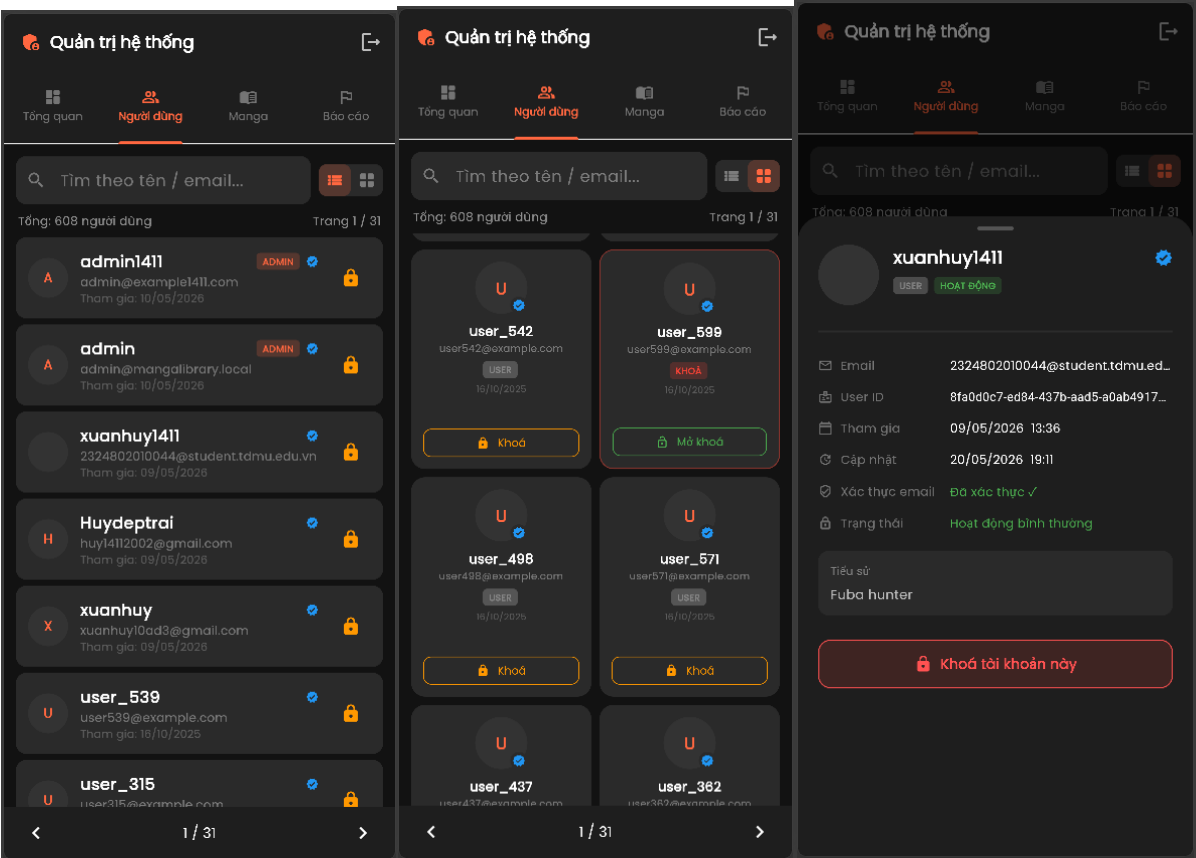


Hình 54. Hỗ trợ nhiều layout hiển thị cho kết quả search

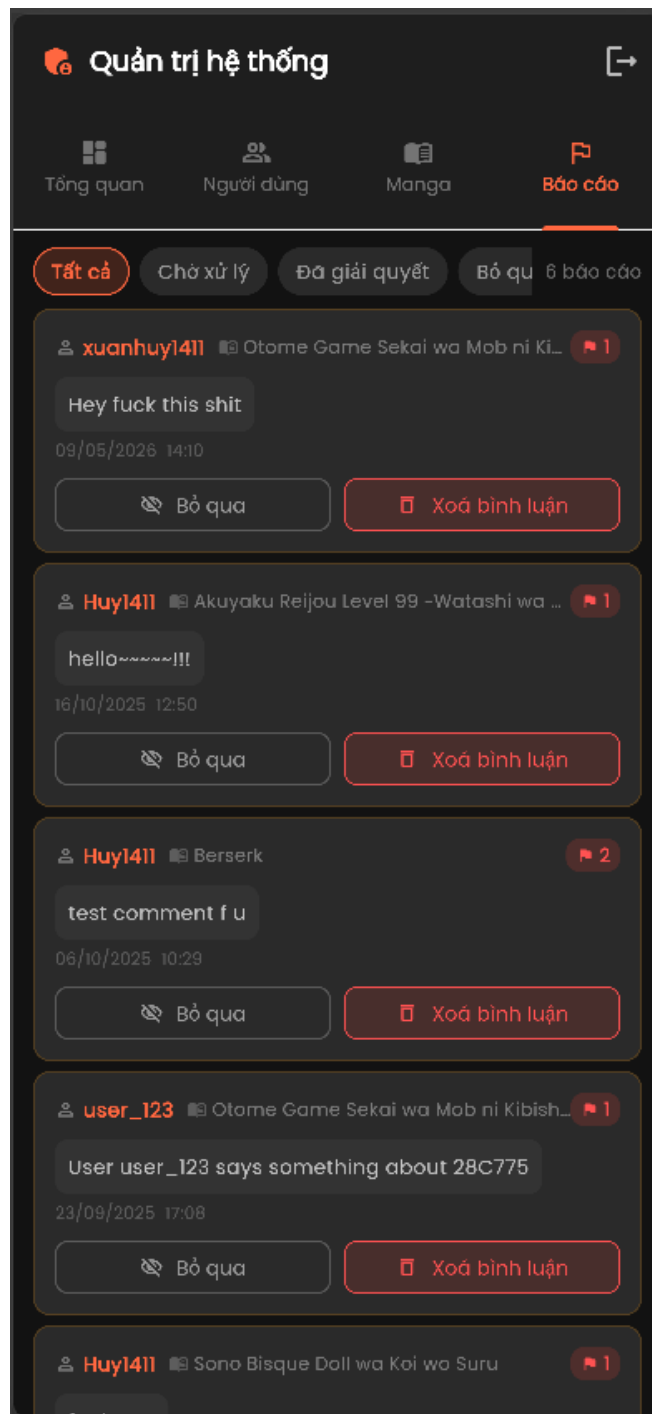
2.2.2. Phân Hệ Admin



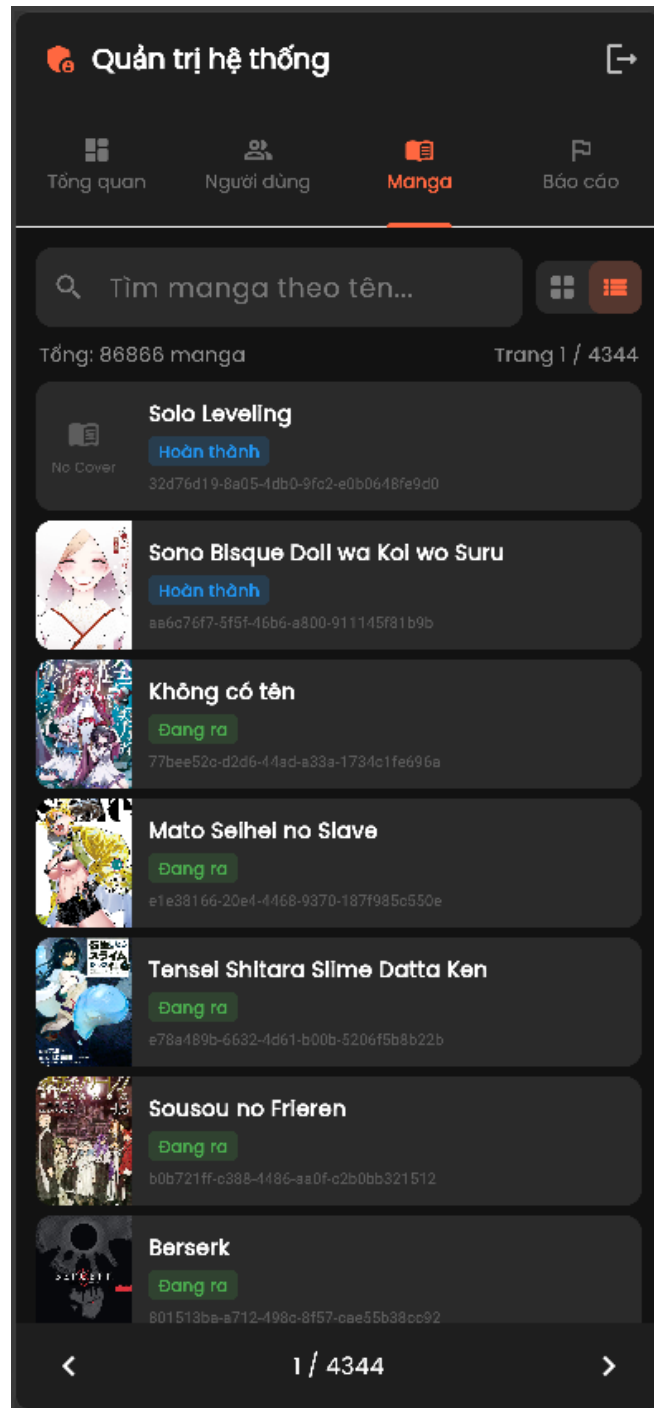
Hình 55. Giao diện dashboard của Admin



Hình 56. Giao diện quản lý độc giả



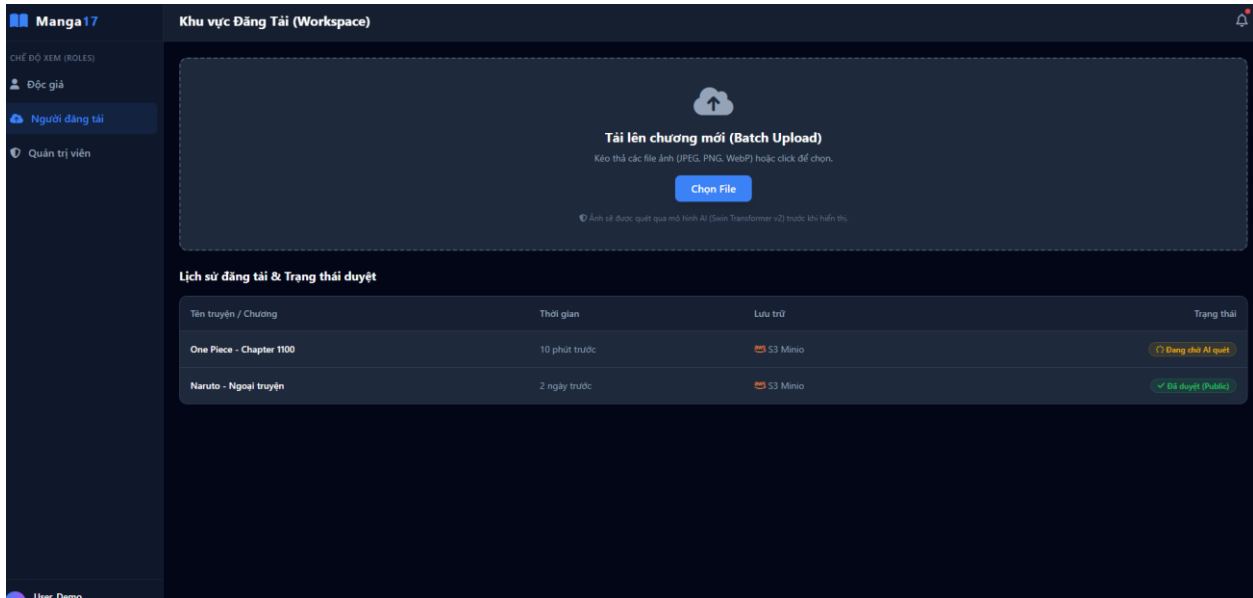
Hình 57. Giao diện quản lý comment



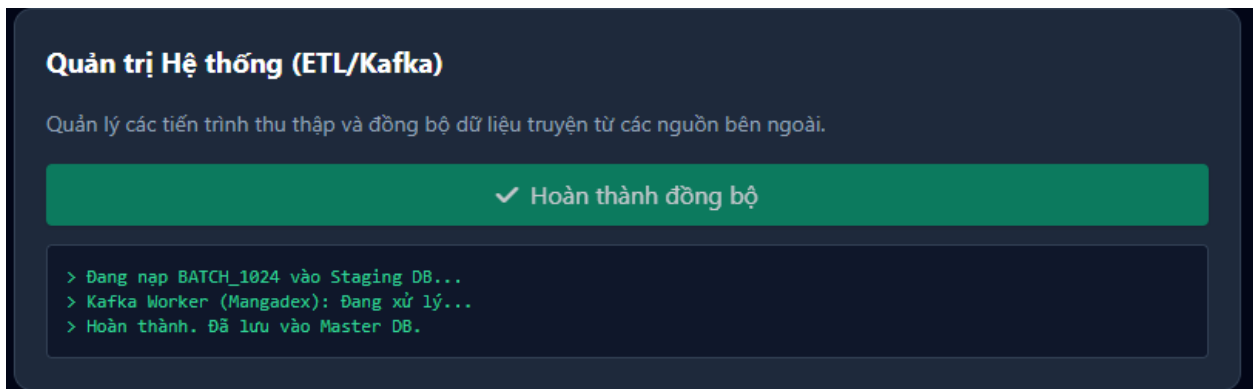
Hình 58. Giao diện quản lý manga của hệ thống

2.2.3. Phân hệ bổ sung

Bên trên là những gì nhóm đã code được, còn dưới đây sẽ là bổ sung thêm các giao diện còn thiếu ứng với các chức năng:



Hình 59. Giao diện của người đăng tải



Hình 60. Giao diện quản lý log của admin

CHƯƠNG 3: ĐÁNH GIÁ VÀ HƯỚNG PHÁT TRIỂN

3.1 Các nội dung đã thực hiện được

Xây dựng bản thiết kế kiến trúc tổng thể: Đề xuất thành công kiến trúc phần mềm cho nền tảng Web - App đọc truyện tranh trực tuyến giải quyết được bài toán về hiệu suất và tự động hóa.

Áp dụng đa dạng các mẫu kiến trúc: Sử dụng Microservices làm kiến trúc nền tảng giúp phân tách các dịch vụ độc lập, kết hợp cùng các mẫu hỗ trợ như Client - Server, Broker (với Apache Kafka) để xử lý bất đồng bộ và Pipe - Filter cho luồng ETL và AI.

Thiết kế cơ sở hạ tầng chịu tải cao: Hoàn thiện mô hình triển khai với các công nghệ tối ưu hiện đại như FastAPI, cơ sở dữ liệu PostgreSQL theo mô hình Master - Slave (tách biệt luồng Đọc/Ghi), Redis Cluster để phân tán cache và MinIO Object Storage để xử lý độc lập luồng tải ảnh lớn.

Tích hợp hệ sinh thái AI vào vận hành: Thiết kế chi tiết các tiến trình Worker ngầm áp dụng AI vào quản lý như: Swin Transformer v2 để kiểm duyệt ảnh nhạy cảm (NSFW), Tesseract OCR kết hợp với AI để nhận diện và dịch thuật, và mô hình BERTopic để phân cụm bình luận phục vụ gợi ý cá nhân hóa.

Xác thực kiến trúc chặt chẽ: Đã lên kịch bản và chứng minh được khả năng đáp ứng của hệ thống đối với 5 thuộc tính chất lượng (Tính sẵn dùng, Khả năng cập nhật, Tính bảo mật, Khả năng mở rộng, Độ tin cậy) trong các tình huống khó khăn như bị DDoS, sập Database Master, sập máy chủ chứa Catalog hay khi tải lượng truy cập tăng x20 lần.

Hoàn thiện và tích hợp nền tảng Mobile App đa nền tảng: Dựa trên kiến trúc Backend mạnh mẽ và hệ thống RESTful API đã được thiết kế mở, nhóm đã xây dựng một ứng dụng di động Flutter (đa nền tảng chạy được trên cả iOS và Android) để mang lại trải nghiệm đọc gọn nhẹ, thuận tiện và mượt mà nhất cho người dùng.

3.2 Những hạn chế trong việc cụ thể hóa cài đặt so với bản thiết kế

Hạn chế về hạ tầng triển khai (Infrastructure): Theo bản thiết kế, lý tưởng nhất là lưu trữ dữ liệu ảnh và backup trên môi trường Cloud (như AWS S3). Tuy nhiên, do hạn chế về kinh phí trong quá trình phát triển, nhóm bắt buộc phải tự triển khai host cục bộ bằng mã nguồn mở (MinIO).

Công cụ điều phối (Orchestration): Hệ thống hiện tại mới chỉ dừng lại ở việc thiết kế chạy các cụm container thông qua trình điều phối Docker Swarm. Nhóm chưa có đủ thời gian và nguồn lực cài đặt hệ thống trên môi trường Kubernetes (K8s) phức tạp hơn để tối ưu hóa triệt để tài nguyên.

Rào cản phần cứng cho AI Worker: Để mô hình AI (Pytorch) chạy trơn tru cần các máy chủ chuyên dụng có trang bị phần cứng Card đồ họa (AI GPU Nodes). Đây là một hạn chế lớn khi triển khai thực tế đối với một đề án do chi phí thuê máy chủ GPU rất đắt đỏ.

Hạn chế trong nghiệp vụ Dịch thuật AI: Việc kết hợp OCR Scan và dịch thuật hiện tại vẫn phụ thuộc vào việc dùng tạm Google Translate API (hoặc cào thủ công qua Playwright). Nhóm chưa thể tích hợp sâu hoàn toàn vào hệ sinh thái Google AI Mode do đòi hỏi kỹ thuật thiết lập phức tạp và giới hạn về API Key.

3.3 Hướng phát triển của của đề tài

Nâng cấp toàn diện hạ tầng lên Kubernetes (K8s): Thay thế Docker Swarm bằng Kubernetes để quản lý Application Cluster, từ đó cung cấp khả năng tự động hóa mở rộng (Auto-scaling), phục hồi lỗi và quản lý cấu hình các Microservices chuyên nghiệp hơn trong môi trường Production.

Tích hợp cổng thanh toán thương mại: Triển khai thêm một Microservice mới là Payment Service hoàn toàn độc lập với các dịch vụ cũ. Dịch vụ này sẽ tích hợp các cổng thanh toán như Momo/ZaloPay để phát hành tính năng mua/thuê các chương truyện Premium nhằm tạo doanh thu bảo trì hệ thống.

Triển khai hệ thống giám sát và ELK Stack: Tích hợp sâu bộ công cụ ELK Stack để tối ưu hóa việc phân tích logs và truy xuất các biểu đồ Analytics phức tạp (đảm bảo hoàn thành dưới 15 giây), giúp Quản trị viên theo dõi sát sao lưu lượng, tốc độ tăng trưởng (User growth) của nền tảng.

Tự động hóa luồng huấn luyện AI (MLOps): Xây dựng các luồng Pipeline tự động hóa định kỳ (6 tháng 1 lần) thu thập dữ liệu người dùng mới để train lại các mô hình AI (Swin NSFW, BERTopic), nhằm nâng cao độ chính xác của tính năng gợi ý và kiểm duyệt theo thời gian. Bổ sung thêm các chỉ số độ tin cậy mới (như R^2) để đo lường năng lực của AI.

TÀI LIỆU THAM KHẢO

Các tài liệu dạng sách

- [1] Cole Nussbaumer Knaflitz. (2021). *Storytelling With Data - Kể Chuyện Thông Qua Dữ Liệu*. Nhà Xuất Bản Thế Giới.
- [2] Ian Sommerville. (2016). *Software Engineering* (10th ed.). Pearson Education.

Các tài liệu trên internet

- [3] Chart.js Contributors. (n.d.). *Chart.js: Simple yet flexible JavaScript charting for designers & developers*. Retrieved November 20, 2025, from <https://www.chartjs.org/>
- [4] SQLAlchemy authors. (n.d.). *SQLAlchemy 1.4 Documentation*. Retrieved November 20, 2025, from <https://www.sqlalchemy.org/>